

Robot Perception and Learning

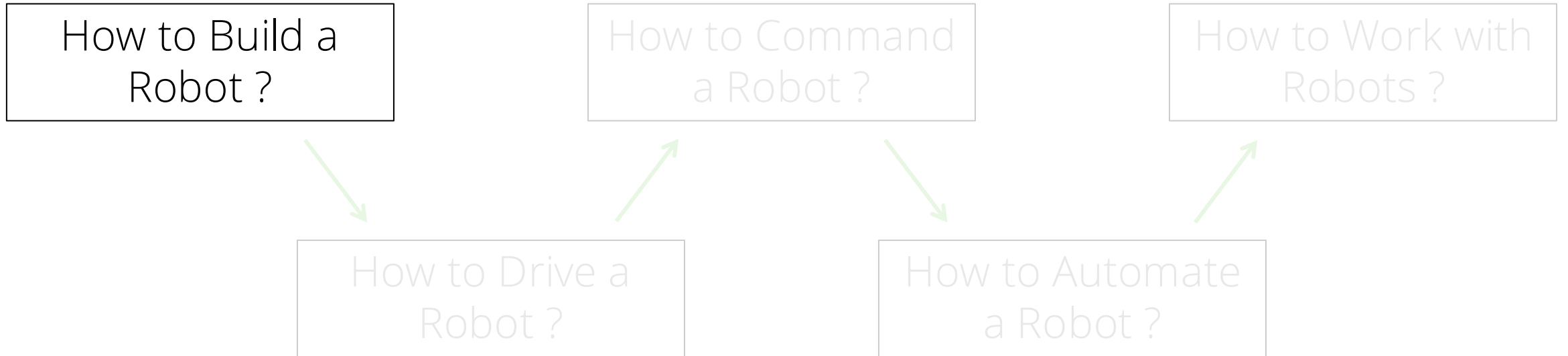
Motion planning

Tsung-Wei Ke

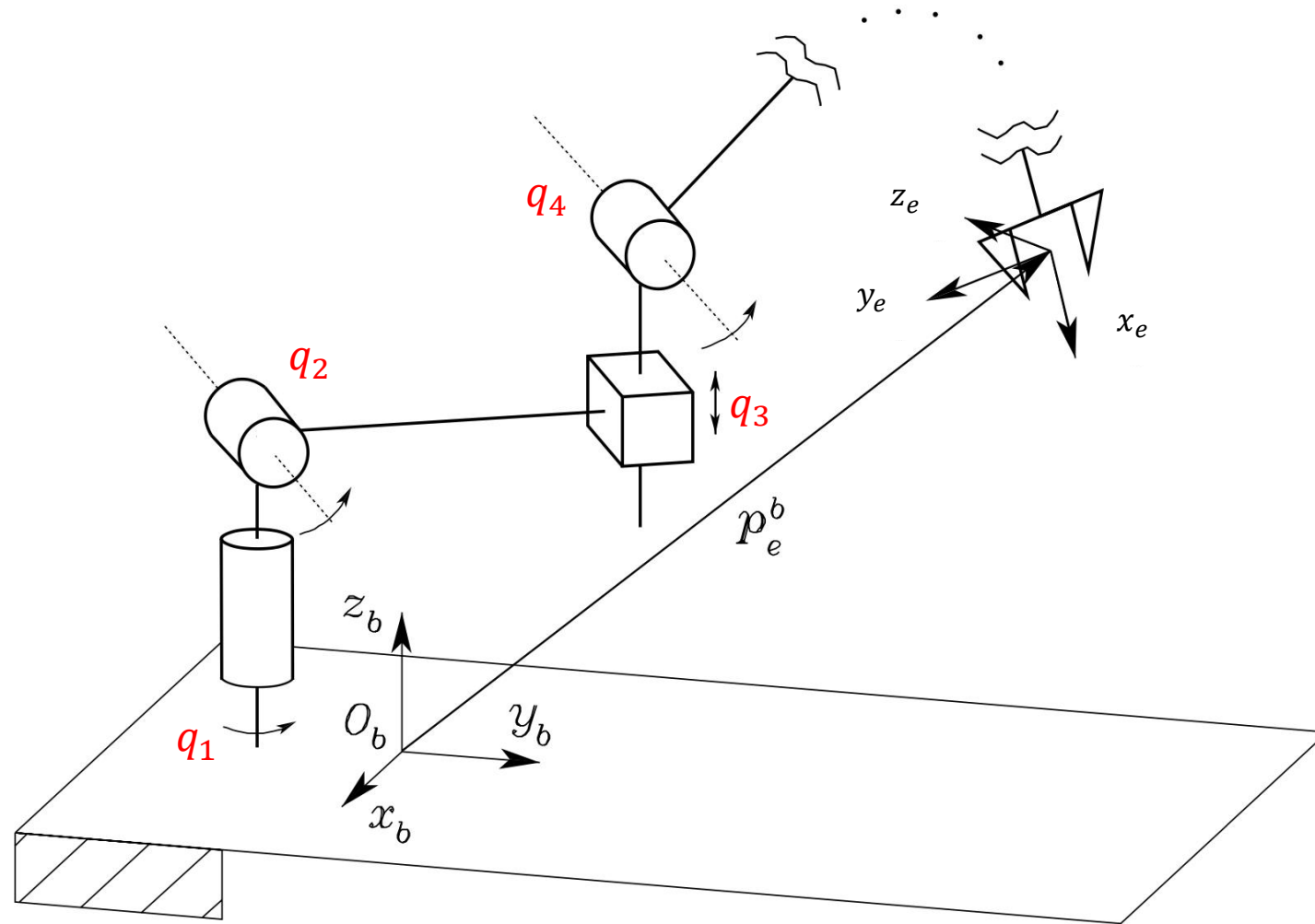
Fall 2026



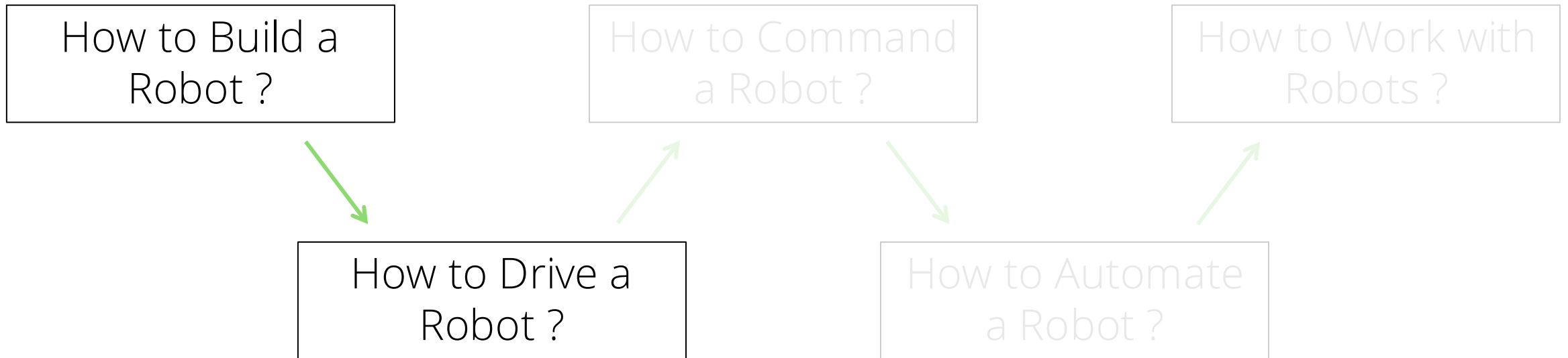
Recap: Robotics is a System



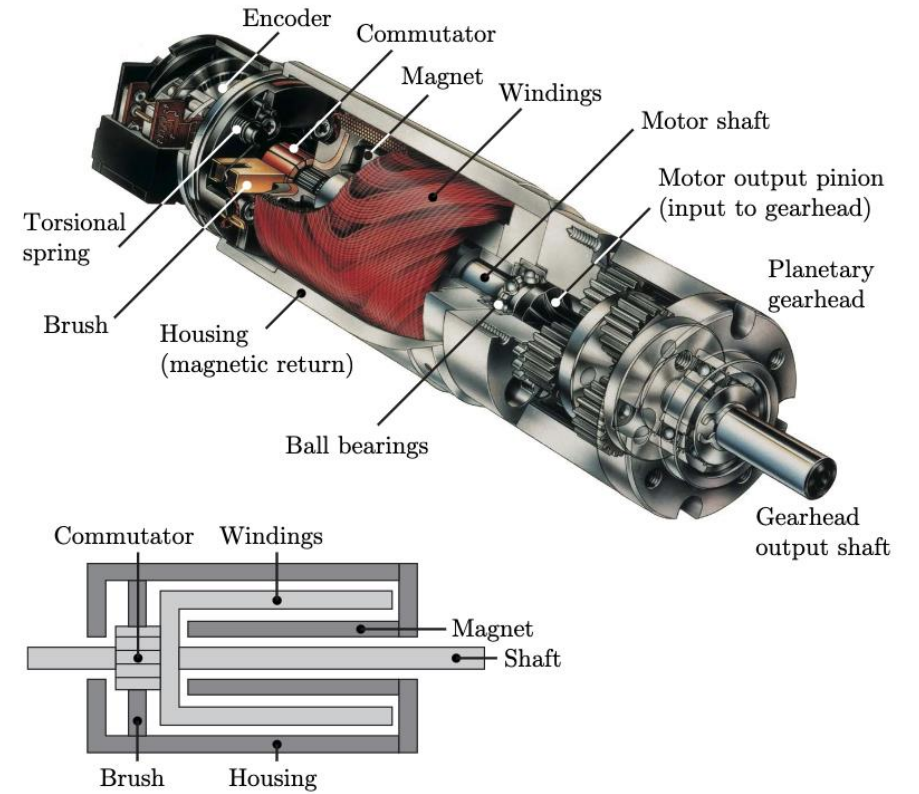
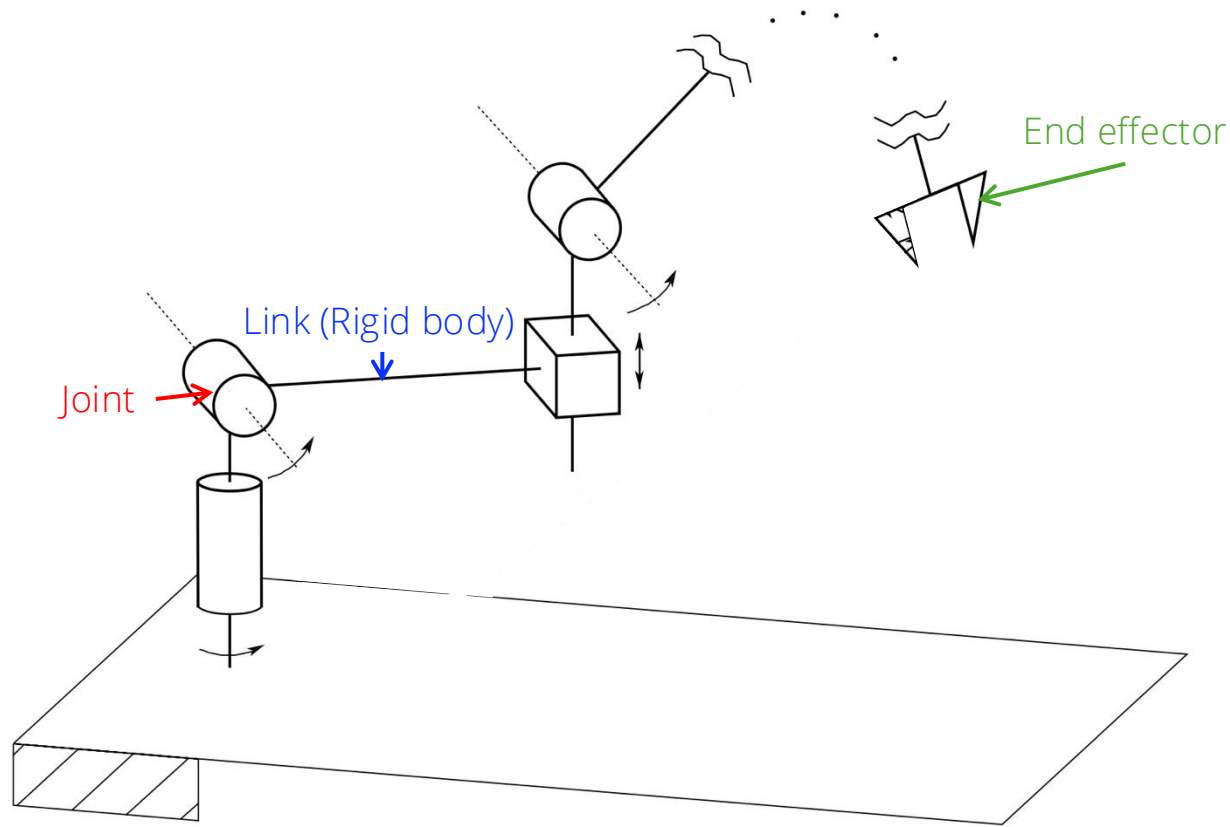
Recap: Robotic Kinematics



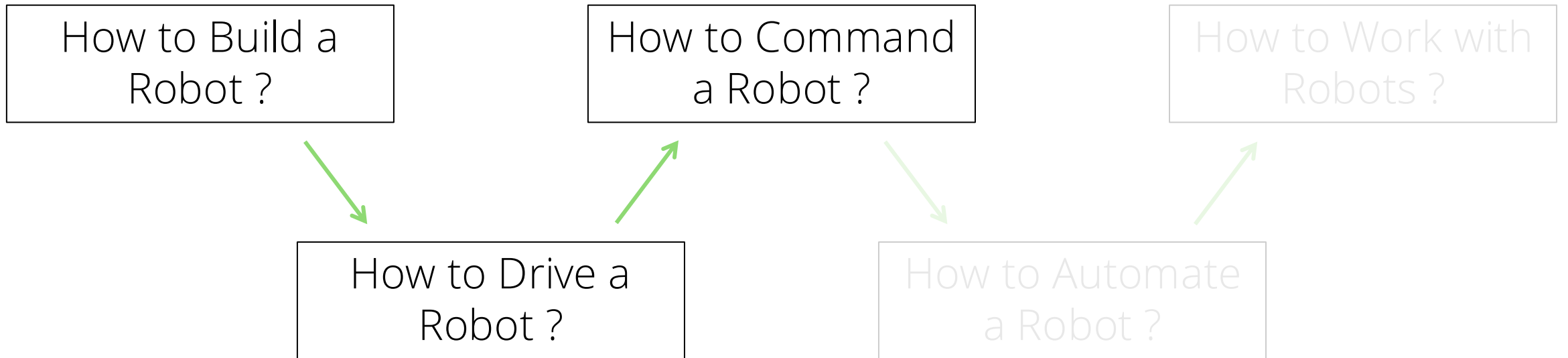
Recap: Robotics is a System

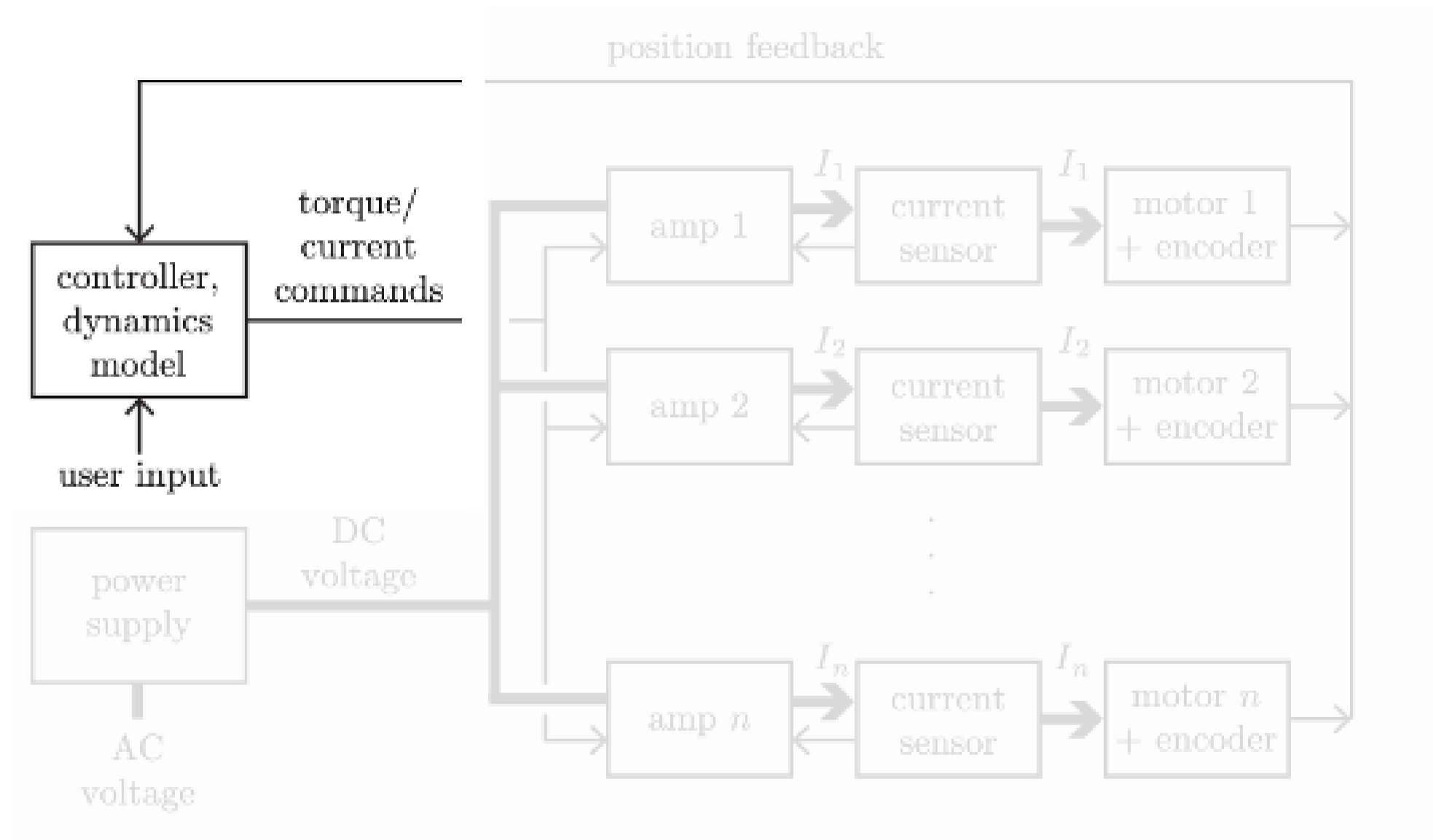


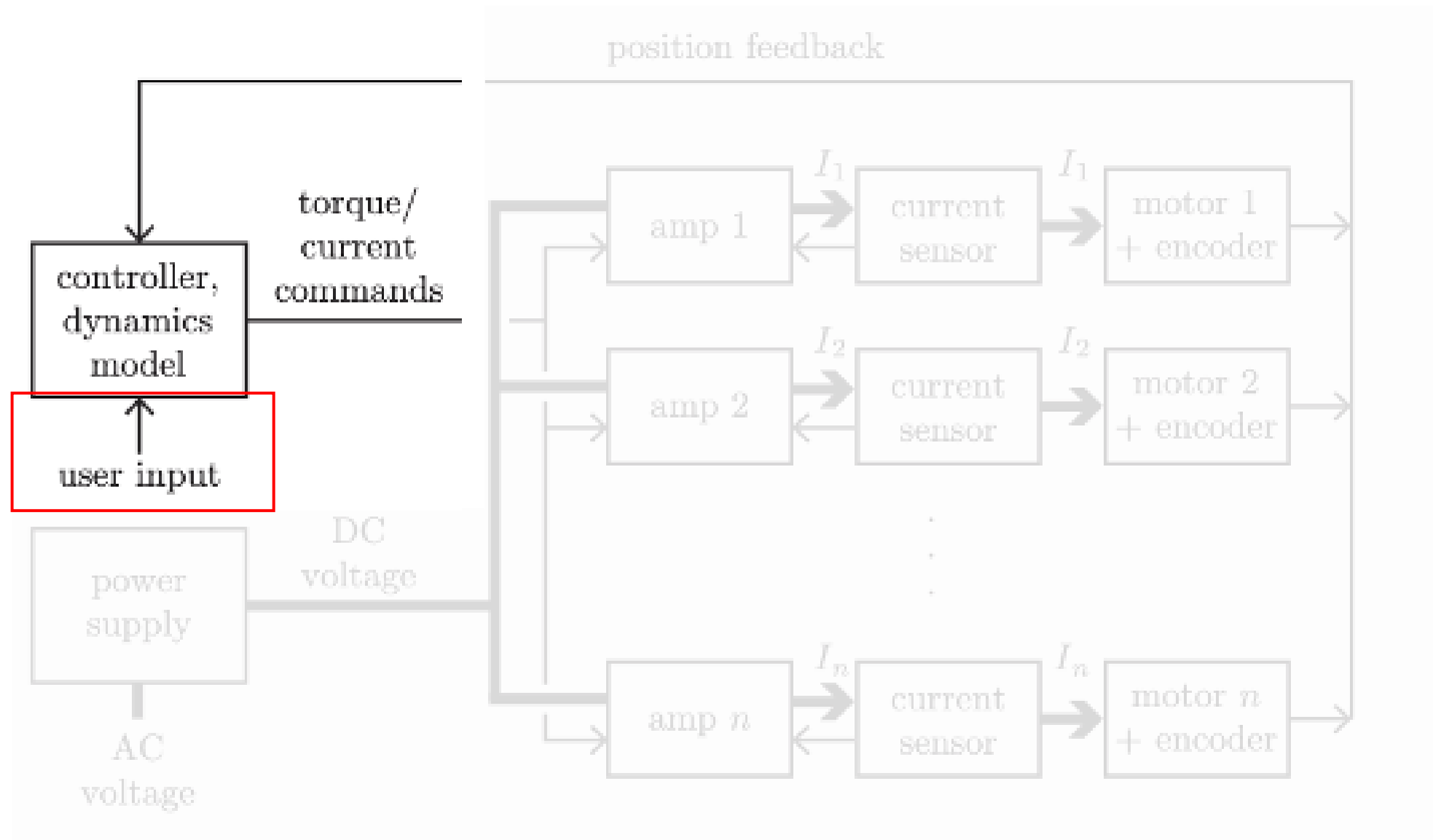
Recap: Motors Drive Robot Joints



Robotics is a System







We want a Robot that can See and Act



First, We Need to Define Goals



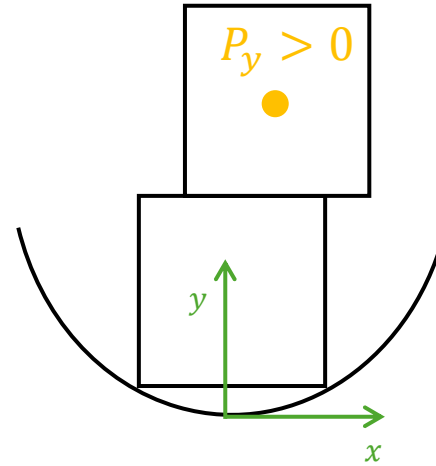
Language instruction

Stack the blocks on the empty bowl.

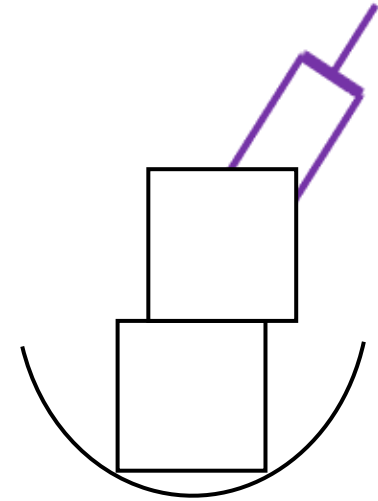
User



Object configuration

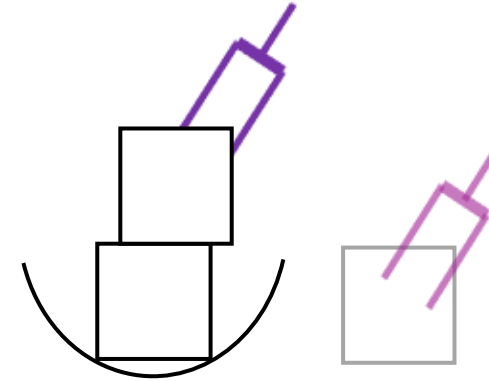


Robot pose

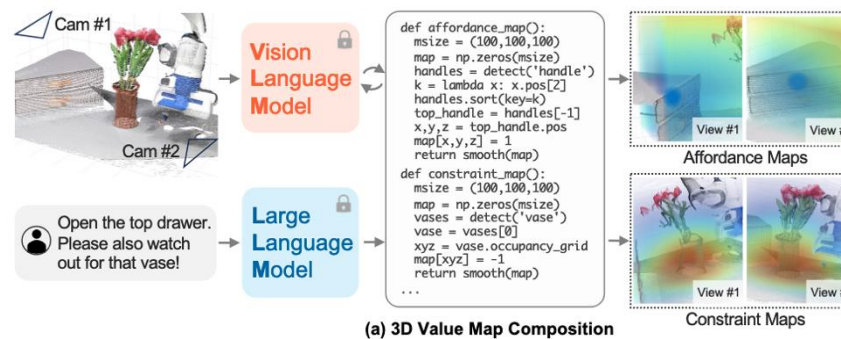


A Robot's End-Effector Poses are Common Representations of Goals

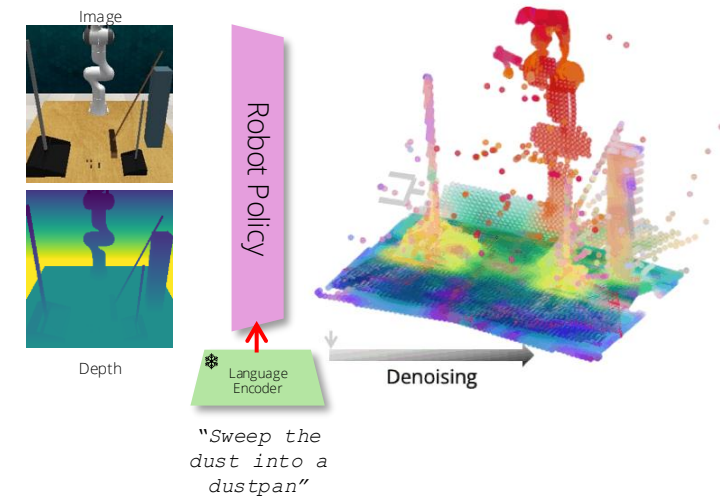
User
Stack the blocks on the empty bowl.



Neural Descriptor Fields: SE(3)-Equivariant Object Representations for Manipulation. Simeonov et al.

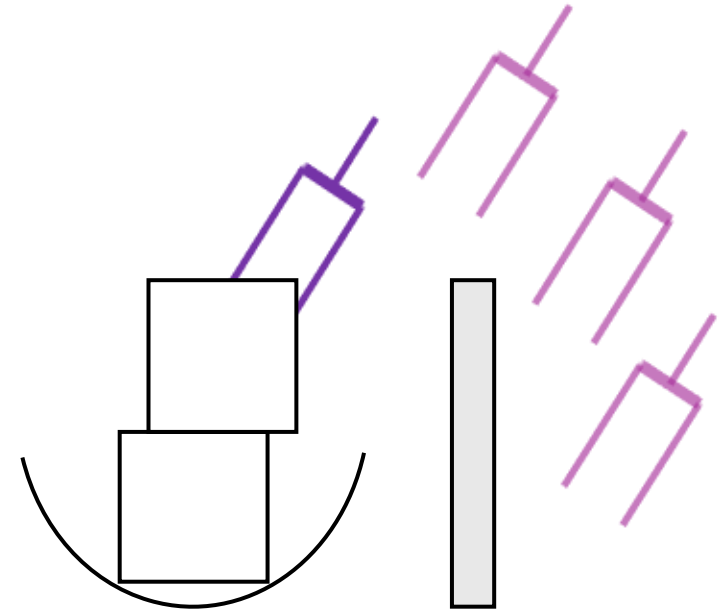
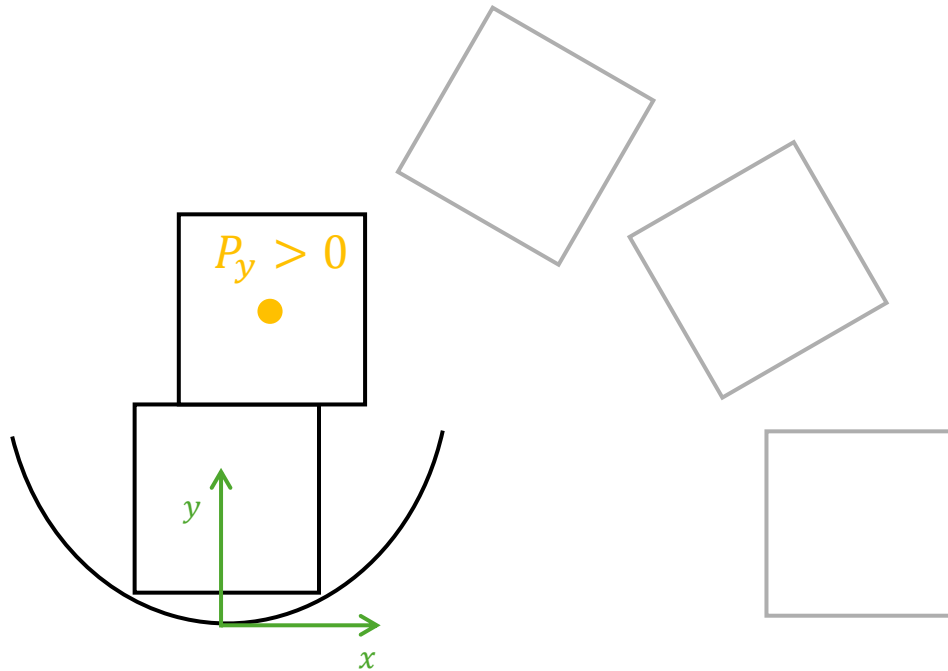
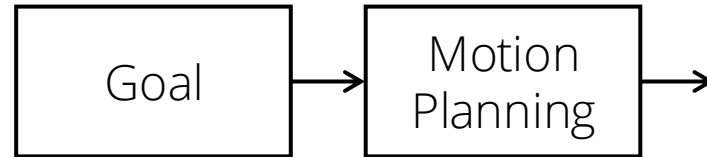


VoxPoser: Composable 3D Value Maps for Robotic Manipulation with Language Models. Huang et al

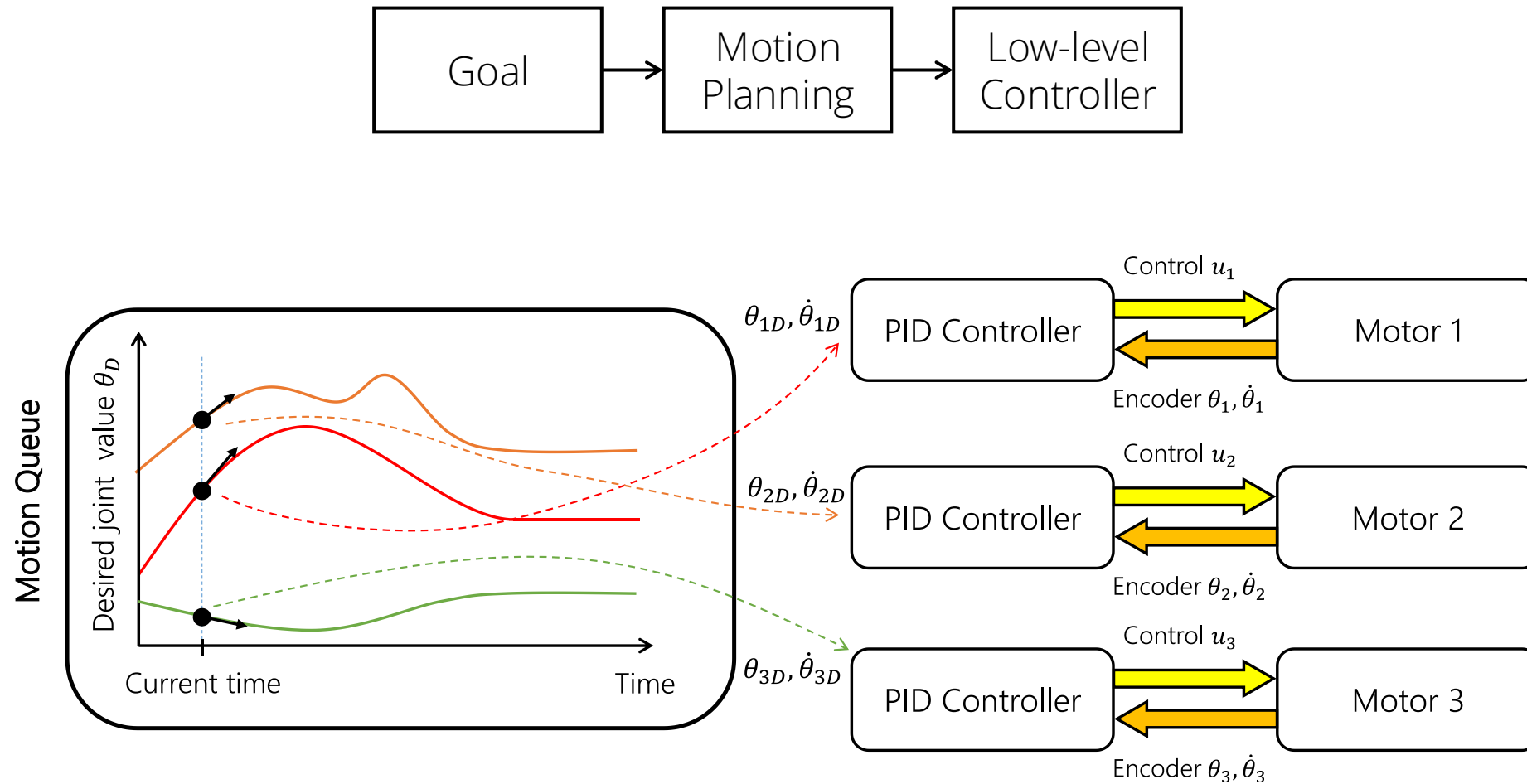


3D Diffuser Actor: Policy Diffusion with 3D Scene Representations. Ke et al.

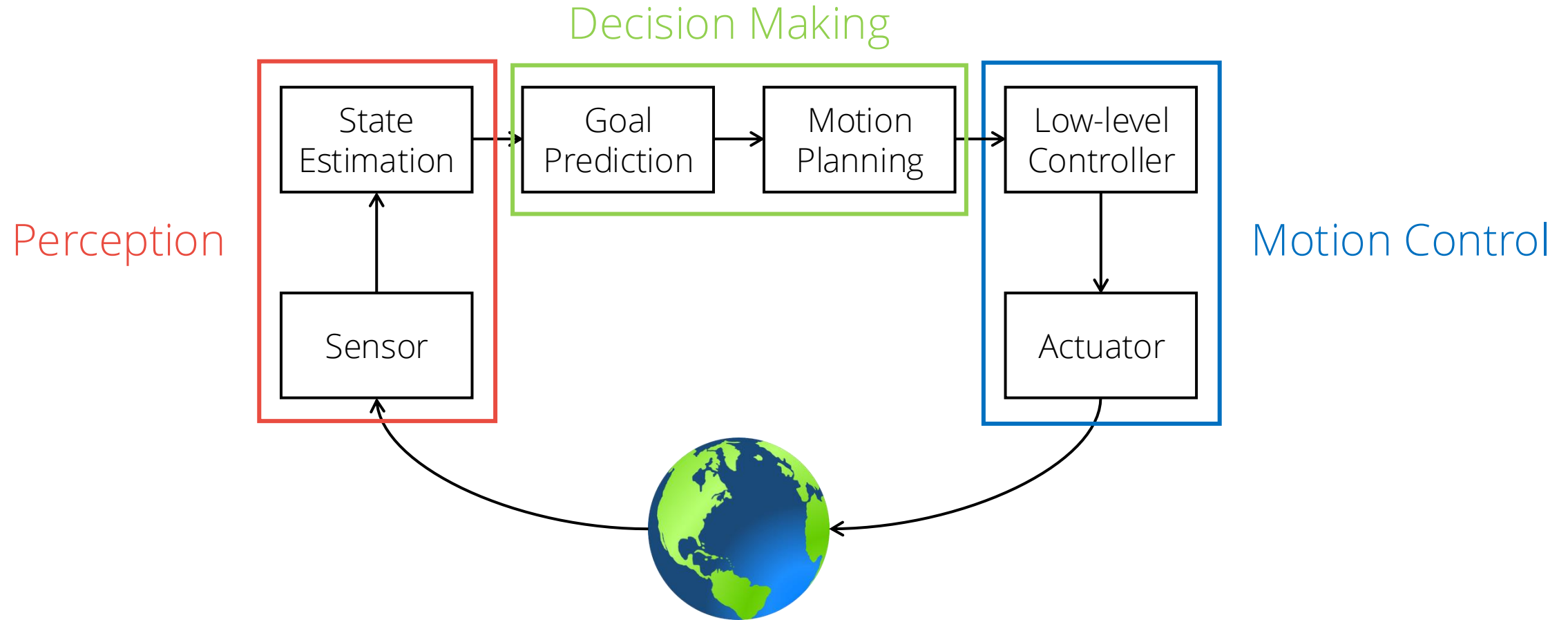
Second, We Need to Decide Paths



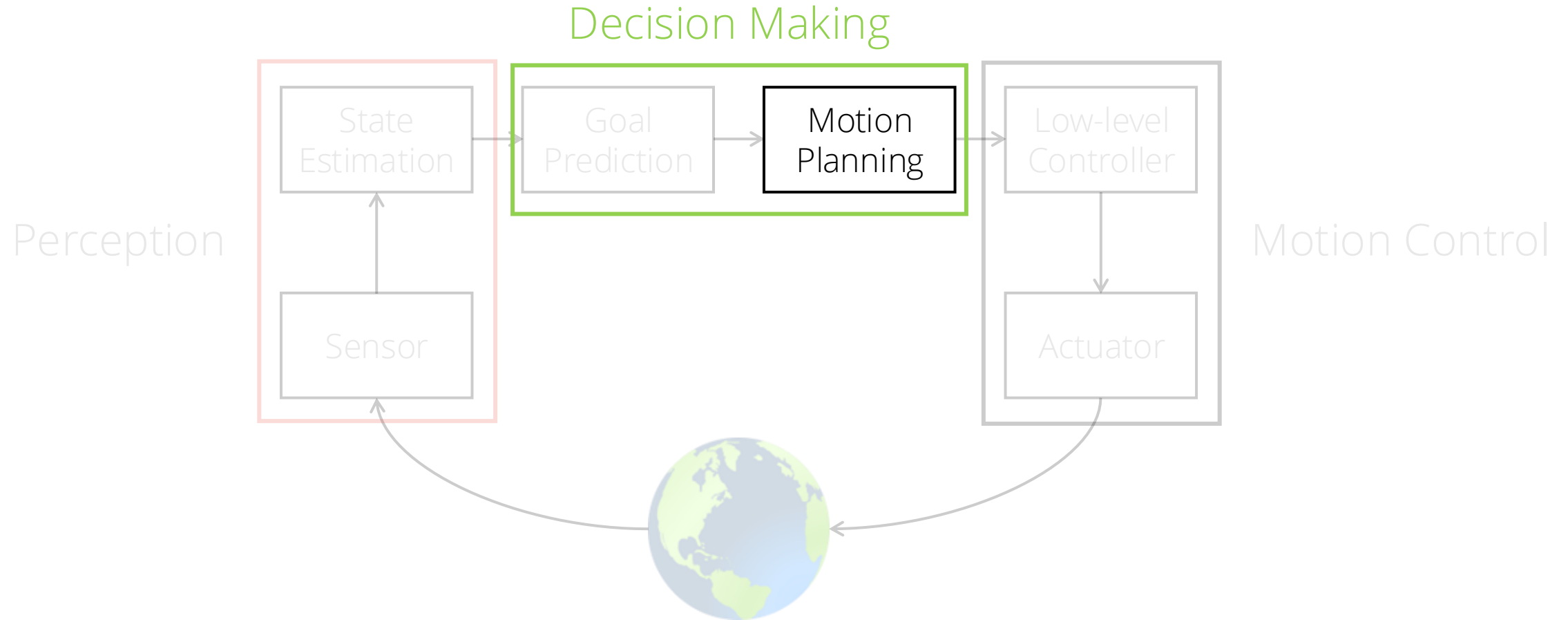
Last, We Need to Direct Movements



Action Requires Deciding a Goal, Planning to Achieve the Goal, and Controlling Motion to Follow the Plan

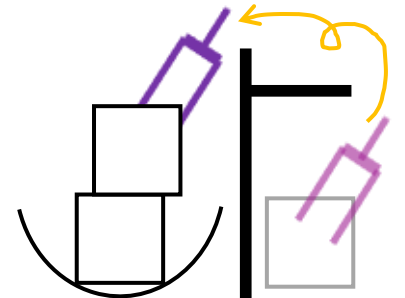


Today's Class Focuses on Motion Planning



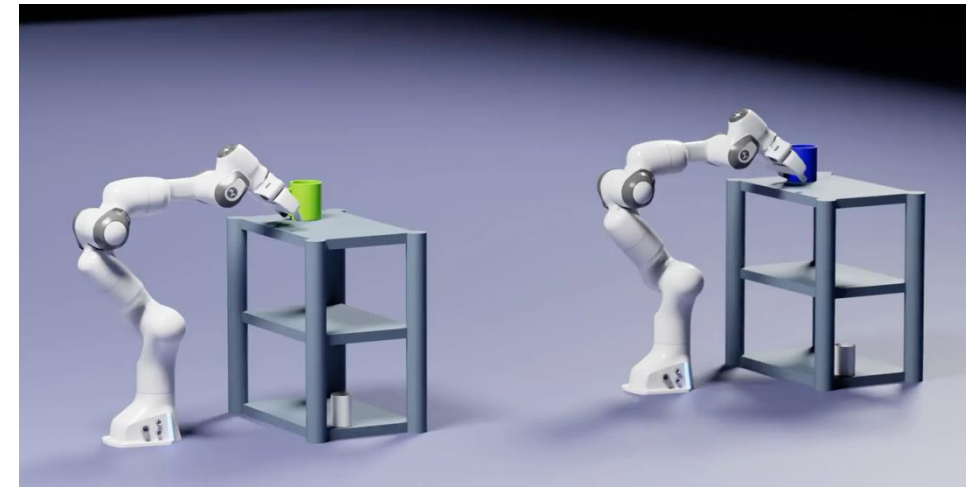
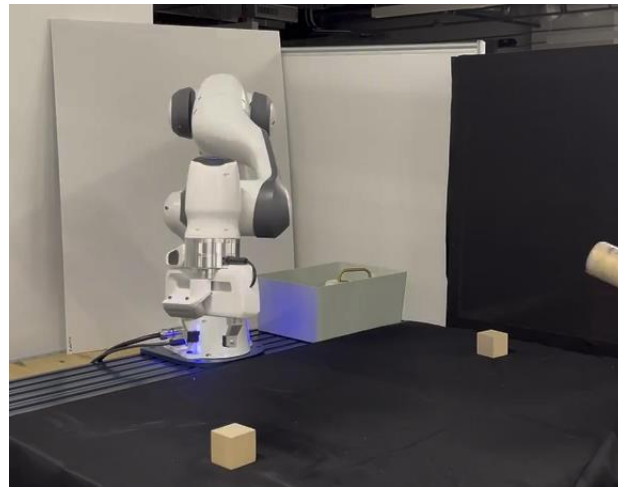
Motion Planning

- Task: Find a **feasible** (and **optimal**) path/motion from the current configuration/pose of the robot to the goal configuration/pose
- Feasibility: The proposed plan should follow the given constraints
 - Environmental constraints (obstacles)
 - Efficiency constraints (dynamics/kinematics)
- Completeness: Report whether or not a feasible path exist in **finite time**
- Optimality: Return the best solution in **finite time** that minimizes the cost: time, energy, risk ...

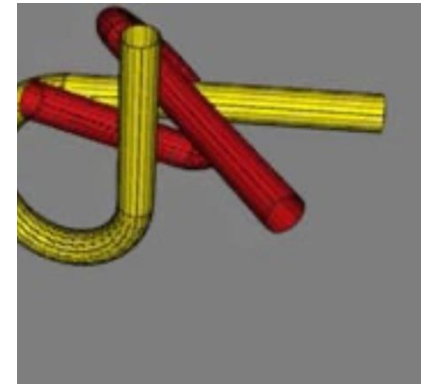
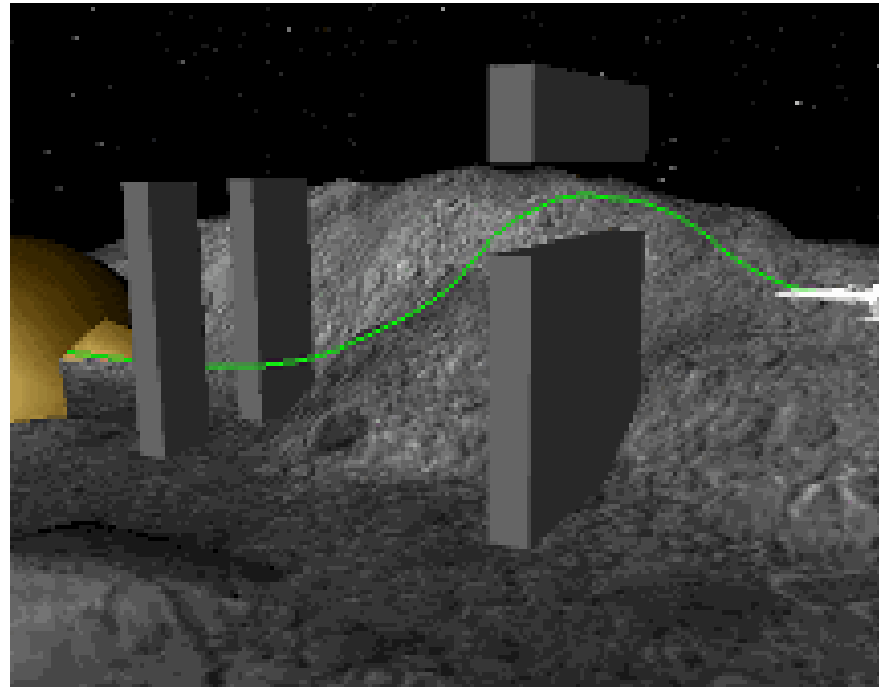
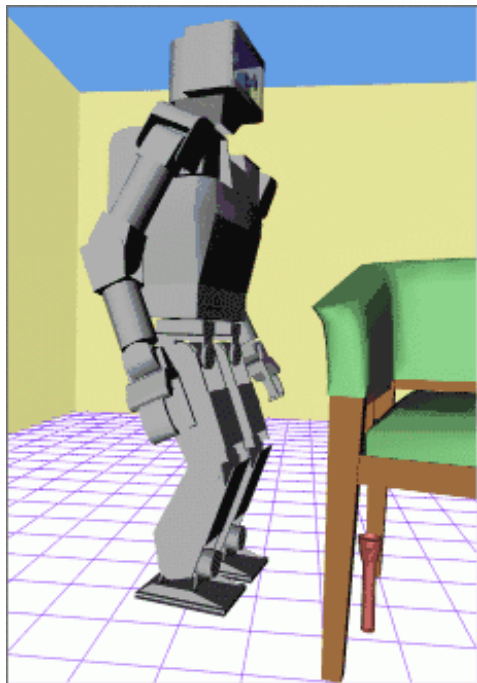


Motion Planning: Piano Mover's Problem

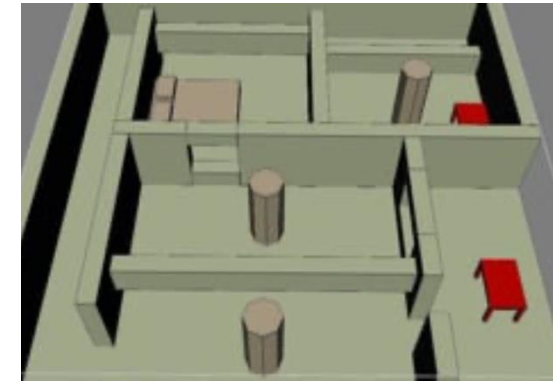




Video credit ccuRobo from Nvidia

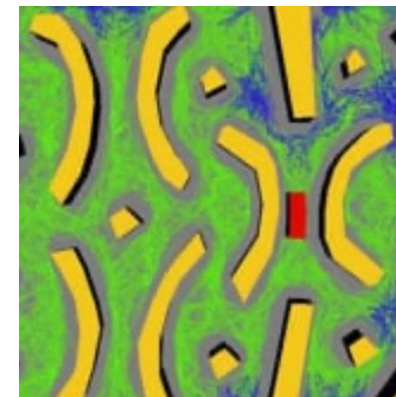


<https://vimeo.com/58709484>

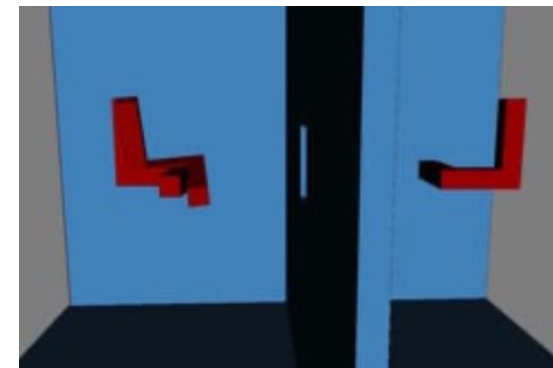


<https://vimeo.com/58686593>

Video credit J.J. Kuffner



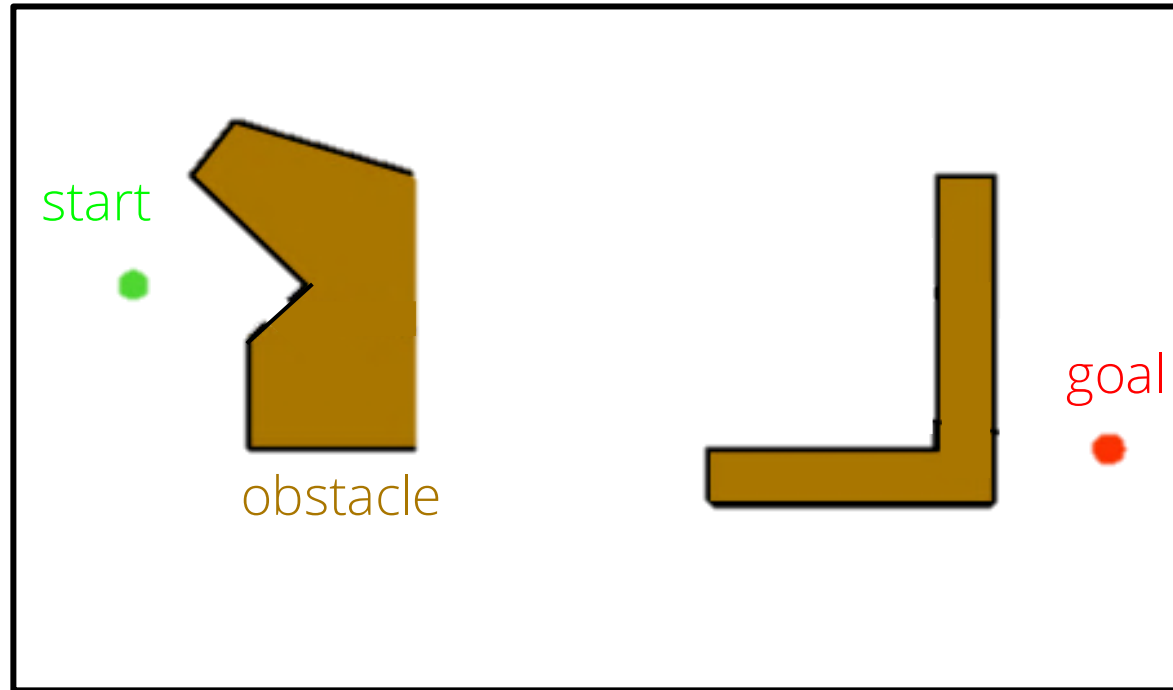
<https://vimeo.com/58686594>



<https://vimeo.com/58709589>

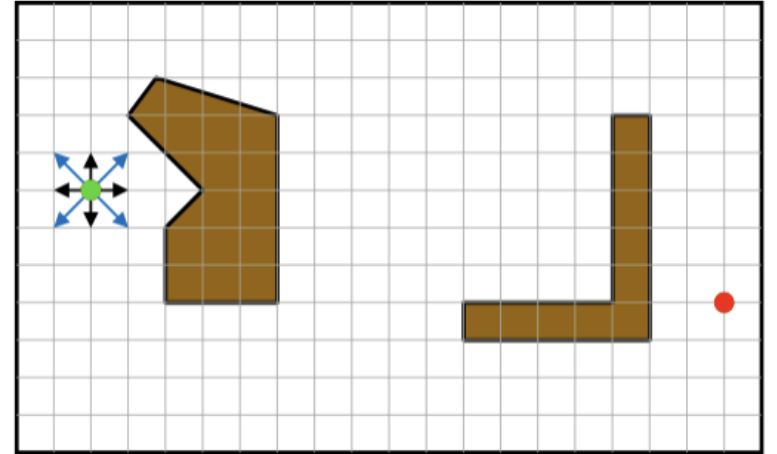
Toy Example: 2D Path Planning with Point Robot

The robot is a point, that can only translate in 2D without rotation.
How can it reach for the red location from the green location?



Solution 1: Roadmap Planner Algorithm

- Discretize the 2D space into grids.
- Robot can only travel to neighboring grids
 - cost=1 to horizontal/vertical neighbors
 - cost= $\sqrt{2}$ to diagonal neighbors
 - cost= ∞ to neighbors including obstacles



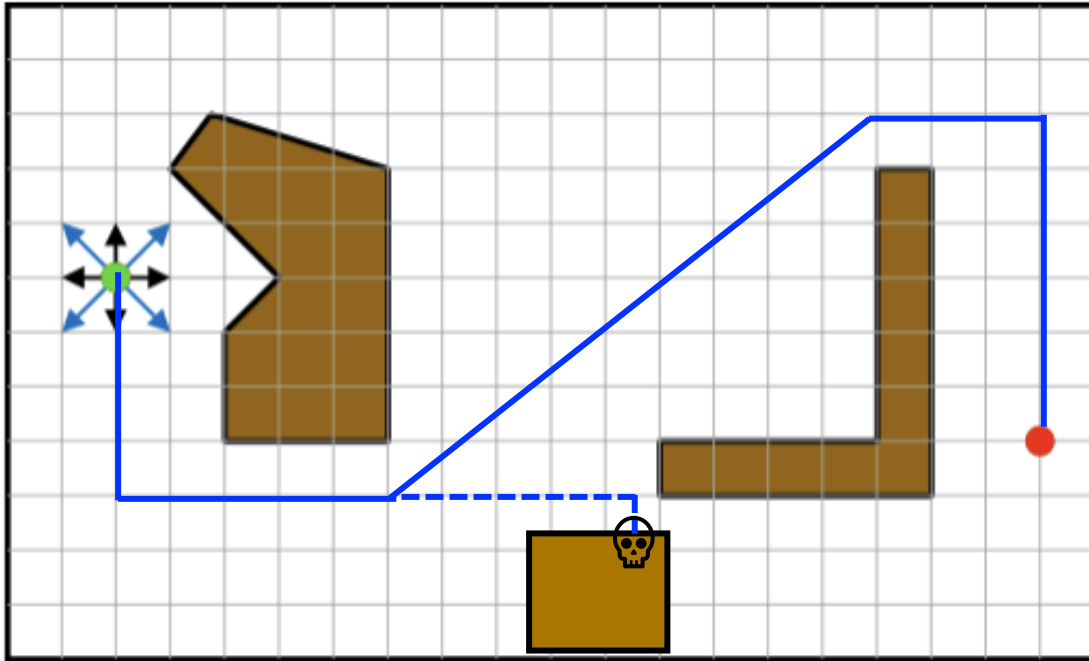
Solution 1: Roadmap Planner Algorithm

- Discretize the 2D space into grids.
 - Robot can only travel to neighboring grids
 - cost=1 to horizontal/vertical neighbors
 - cost= $\sqrt{2}$ to diagonal neighbors
 - cost= ∞ to neighbors including obstacles
 - Search the best path (e.g. Dijkstra's algorithm)
- Discretize the 2D space into grids.
 - Robot can only travel to neighboring grids
 - cost=1 to horizontal/vertical neighbors
 - cost= $\sqrt{2}$ to diagonal neighbors
 - cost= ∞ to neighbors including obstacles
 - Search the best path (e.g. Dijkstra's algorithm)

Roadmap Planner Algorithm Has Resolution Problems

Coarse-resolution grids:

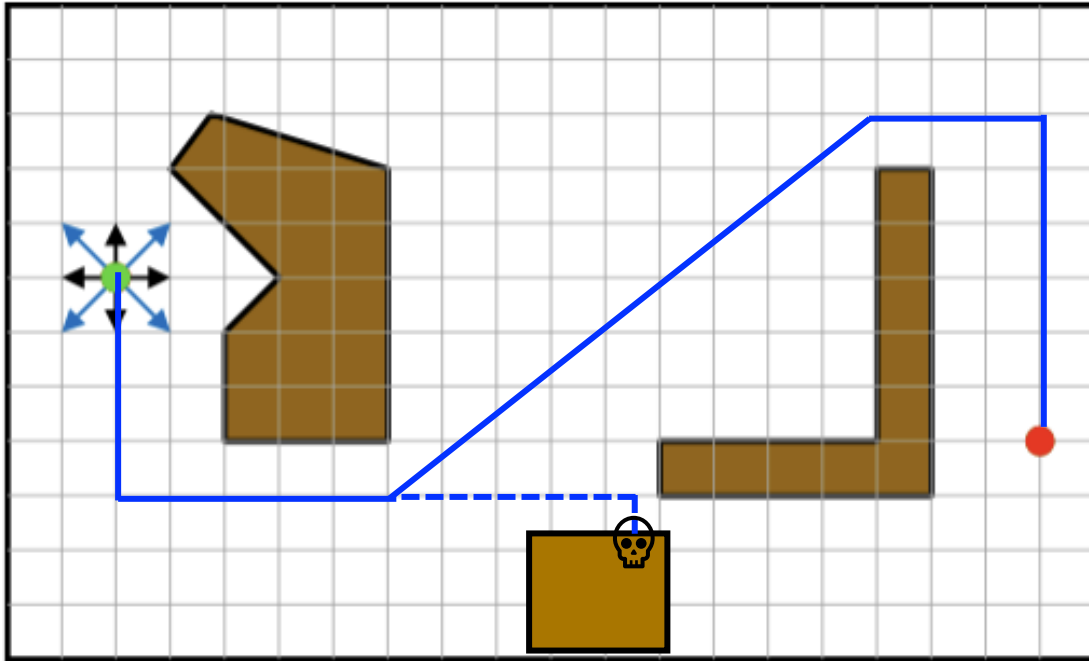
- Computationally efficient
- May not be optimal



Roadmap Planner Algorithm Has Resolution Problems

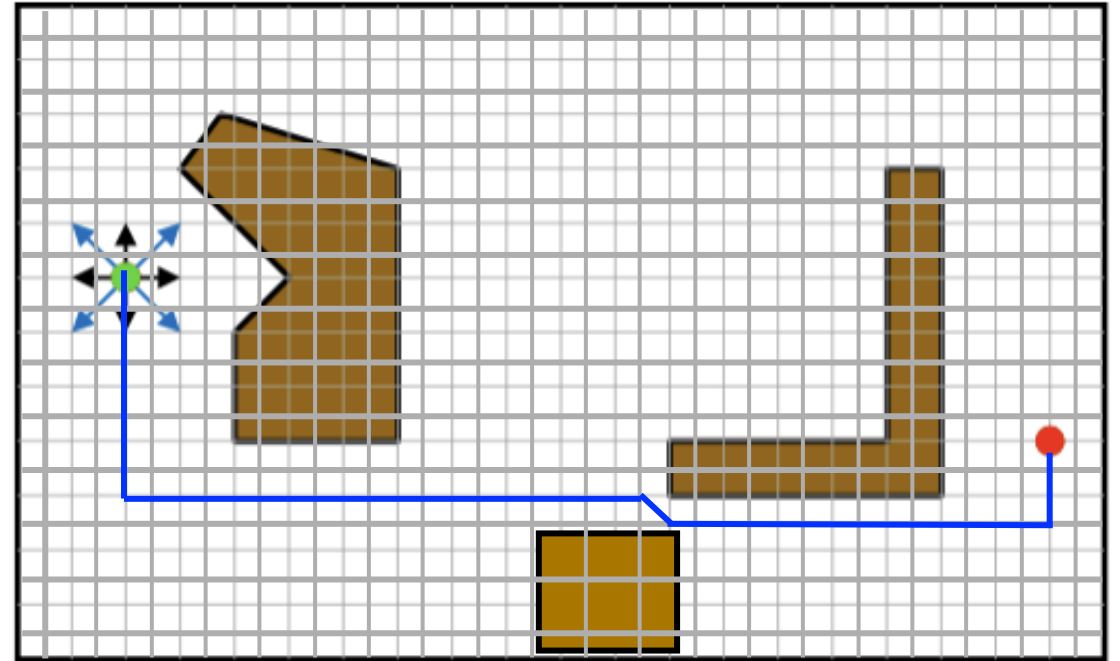
Coarse-resolution grids:

- Computationally efficient
- May not be optimal



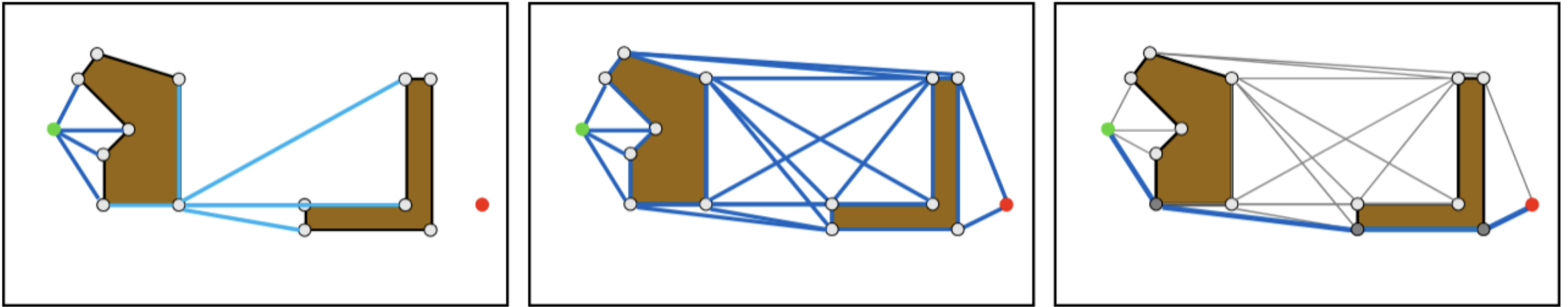
Fine-resolution grids:

- Computationally inefficient
- More likely to be optimal



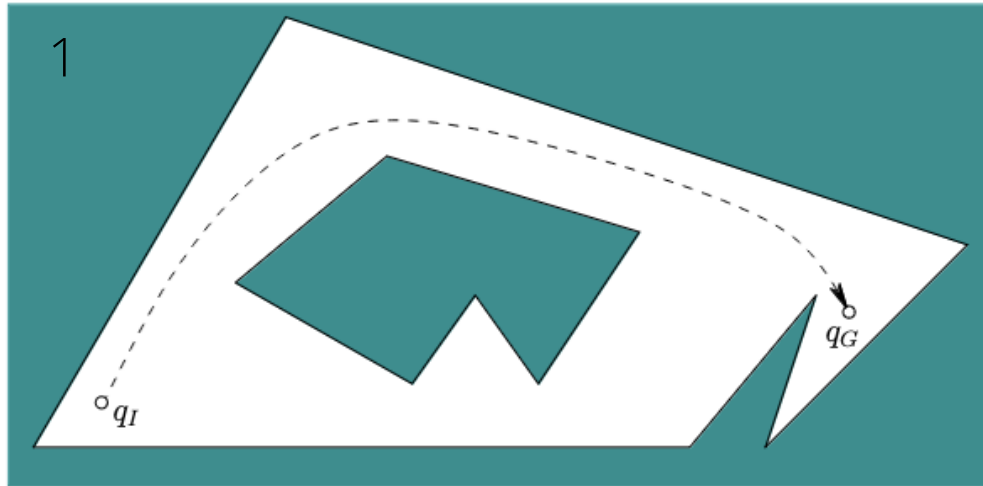
Solution 2: Visibility Graph Algorithm

We don't need grids, but vertices of the start location, goal location and obstacle corners

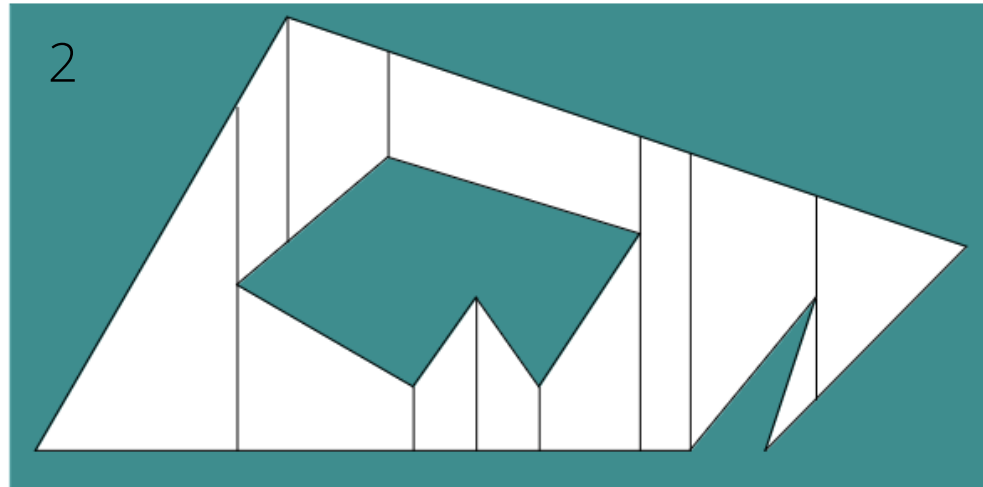
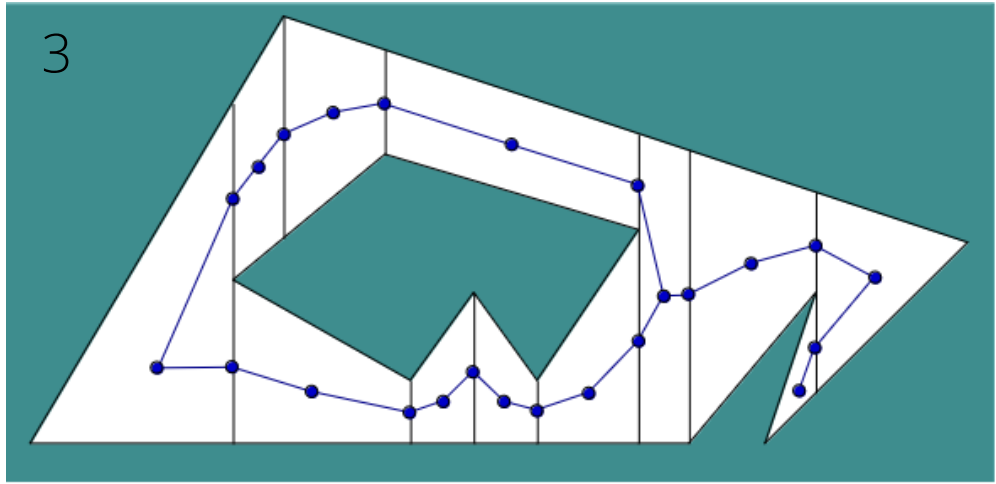


1. Let V be the union of the start, the goal and all obstacle vertices.
2. $E \leftarrow \{\}$
3. **for** all pairs of distinct vertices $u, v \in V$
4. **if** $\overline{uv}$ is an obstacle edge
5. Add (u, v) to E
6. **else if** $\overline{uv}$ is collision free
7. Add (u, v) to E
8. Search $G=(V,E)$, with Cartesian distance as the edge cost, to connect the start and goal
9. **return** the path if one is found.

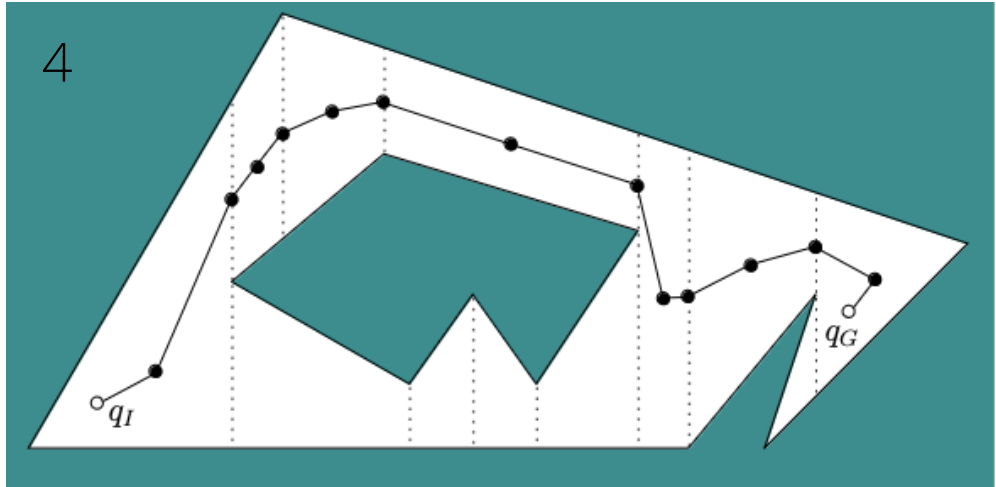
Solution 3: Cell Decomposition Algorithm



Connect neighboring free-space cells



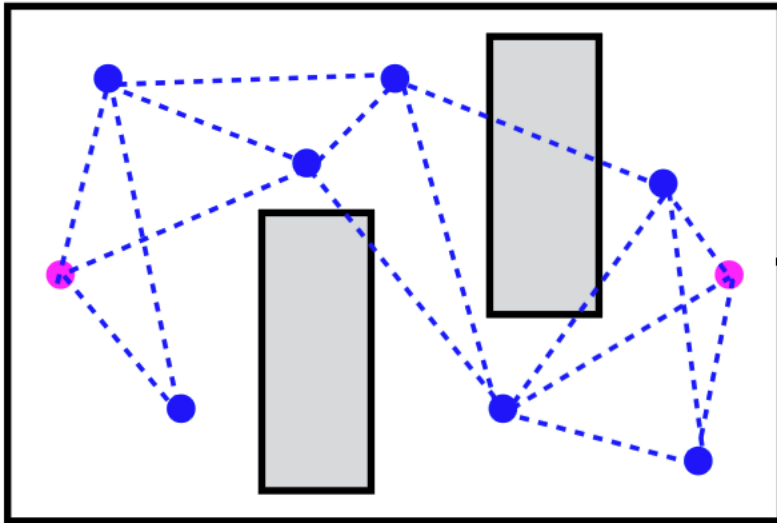
Segment the free space into cells by vertices



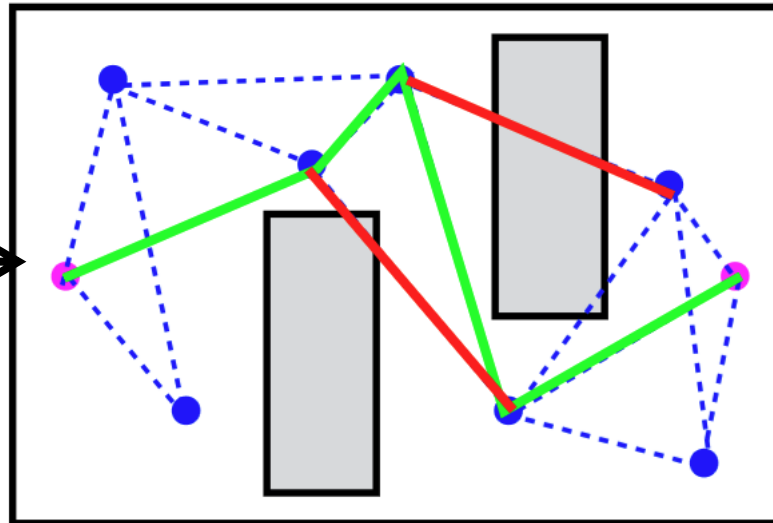
Search a path that traverses from the start cell to the goal cell

A General Framework for Motion/Path Planning

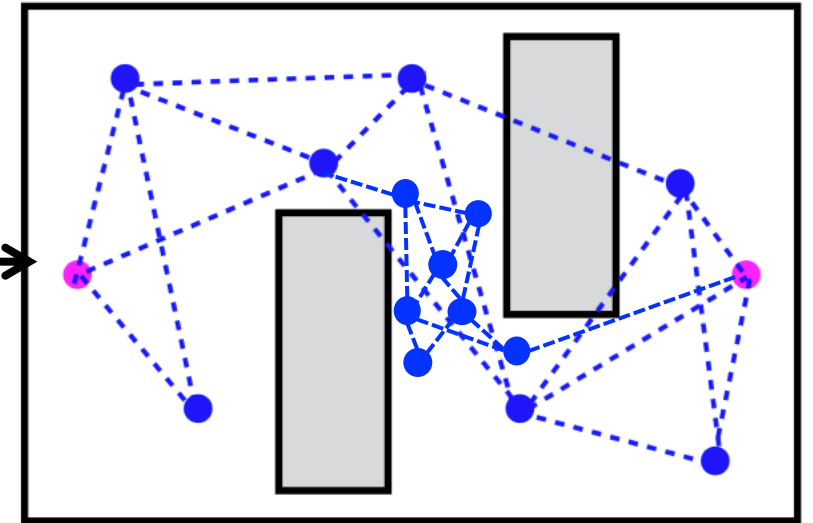
Create a graph



Search the graph

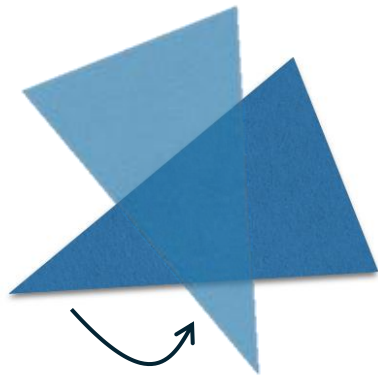


Densify the graph



But Planning is Hard...

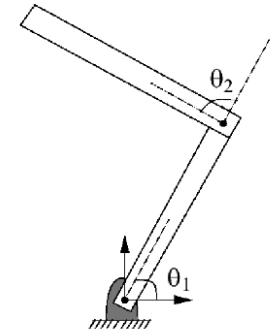
- Previously, we only consider 2D point robots. What if the robot shape is triangle that can both translate and rotate in 2D?
- What if the robot is a car which cannot make pure left/right movement?
- What if the robot is a 2-joint planar arm?



Triangle



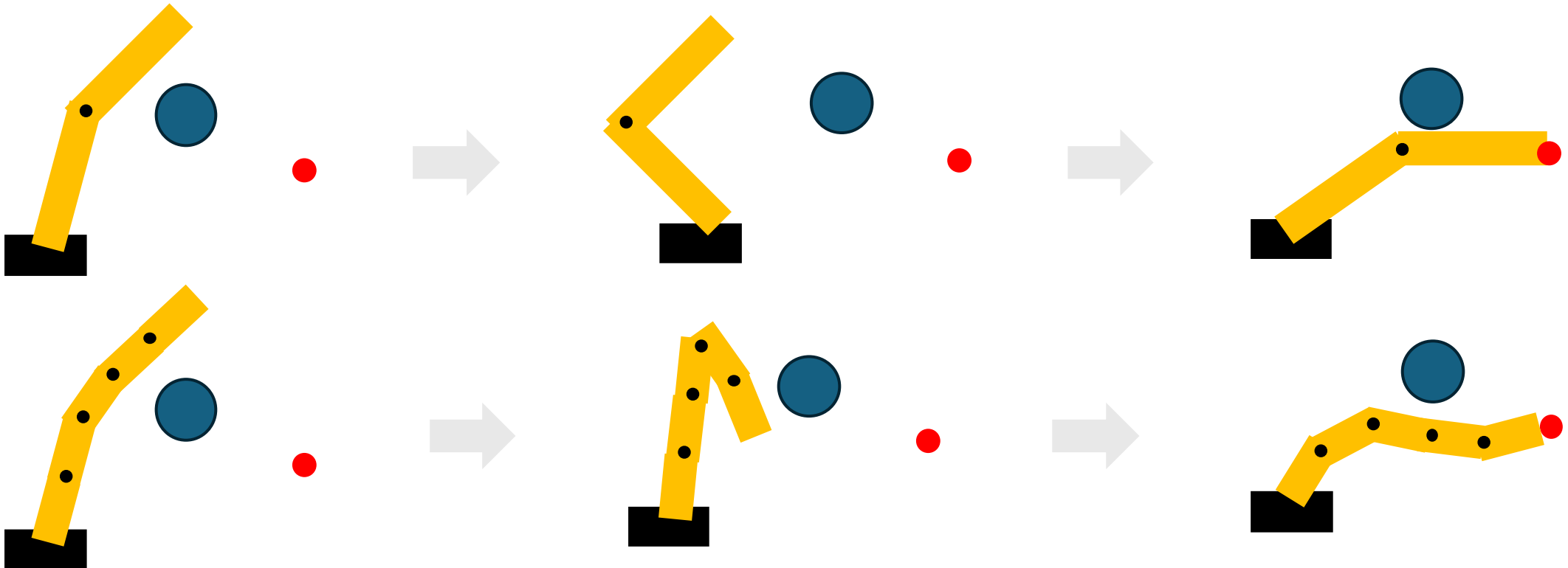
Racecar



2-joint planar arm

But Planning is Hard...

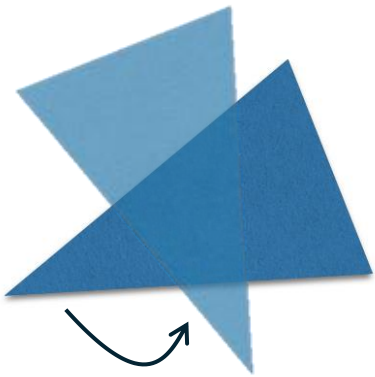
- Previously, we only consider 2D point robots. What if the robot shape is triangle that can both translate and rotate in 2D?
- What if the robot is a car which cannot make pure left/right movement?
- What if the robot is a 2-joint planar arm?



But Planning is Hard...

- Previously, we only consider 2D point robots. What if the robot shape is triangle that can both translate and rotate in 2D?
- What if the robot is a car which cannot make pure left/right movement?
- What if the robot is a 2-joint planar arm?
- What if the robot is a 7-joint planar arm?
- What if the robot works in 3D?

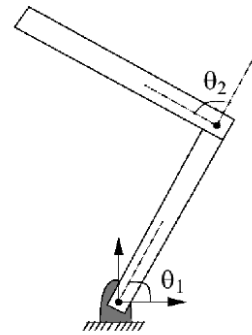
What is the unified planning formulation for robots with different mechanisms?



Triangle



Racecar



2-joint planar arm

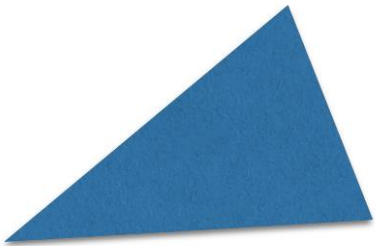


Manipulator

The Configuration Space

- The configuration of a robot is a complete specification of the position of every point of the robot
- Configuration space (C-space) is the n-dimensional space containing all possible configurations of the robot. In other words, C-space includes the set of all rigid-body transformations that can be applied to the robot.

$\mathbb{R}^2$



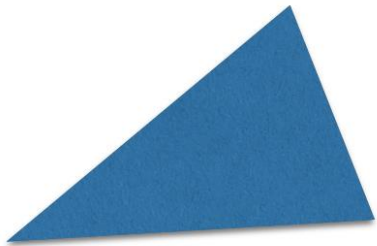
Translating Triangle

The Configuration Space

- The configuration of a robot is a complete specification of the position of every point of the robot
- Configuration space (C-space) is the n-dimensional space containing all possible configurations of the robot. In other words, C-space includes the set of all rigid-body transformations that can be applied to the robot.

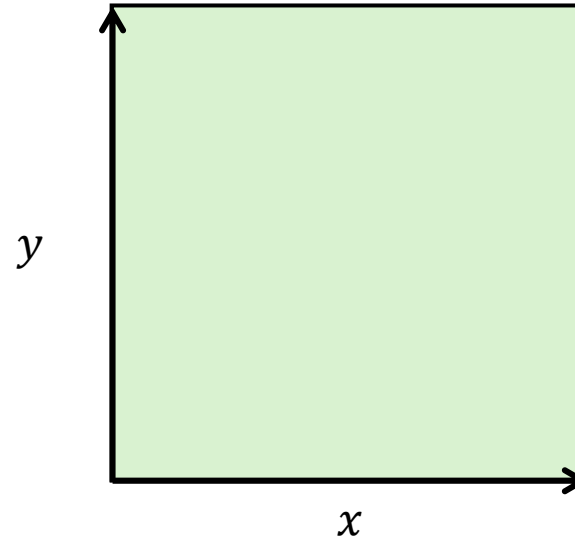
We can represent it with coordinates x, y

$\mathbb{R}^2$



Translating Triangle

C-space

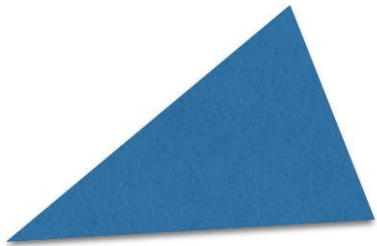


The Configuration Space

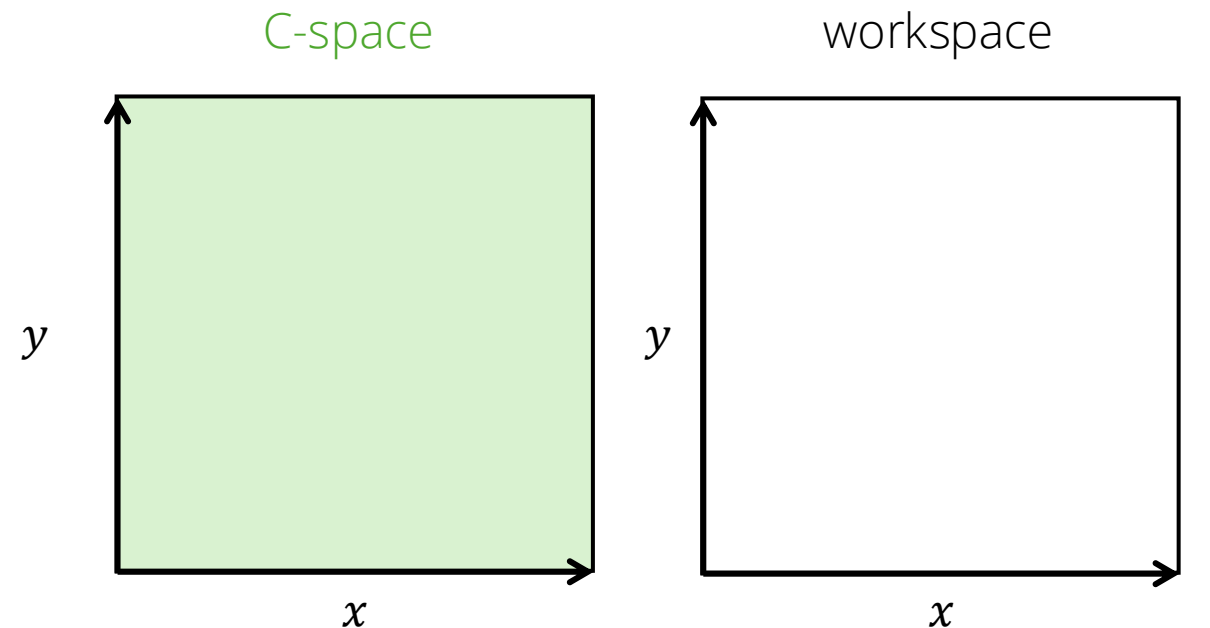
- The configuration of a robot is a complete specification of the position of every point of the robot
- Configuration space (C-space) is the n-dimensional space containing all possible configurations of the robot. In other words, C-space includes the set of all rigid-body transformations that can be applied to the robot.

We can represent it with coordinates x, y

$\mathbb{R}^2$



Translating Triangle

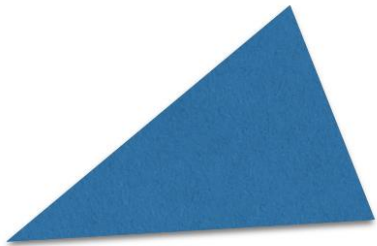


The Configuration Space

- The configuration of a robot is a complete specification of the position of every point of the robot
- Configuration space (C-space) is the n-dimensional space containing all possible configurations of the robot. In other words, C-space includes the set of all rigid-body transformations that can be applied to the robot.

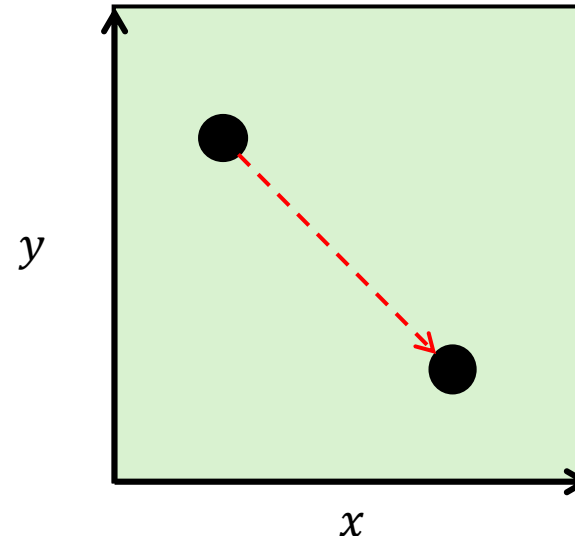
We can represent it with coordinates x, y

$\mathbb{R}^2$



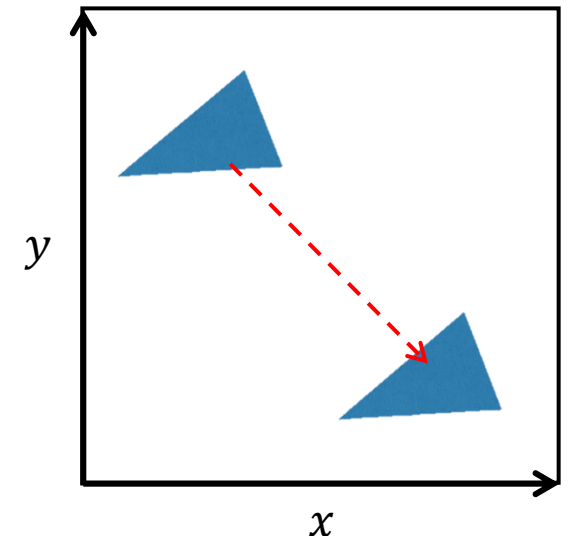
Translating Triangle

C-space



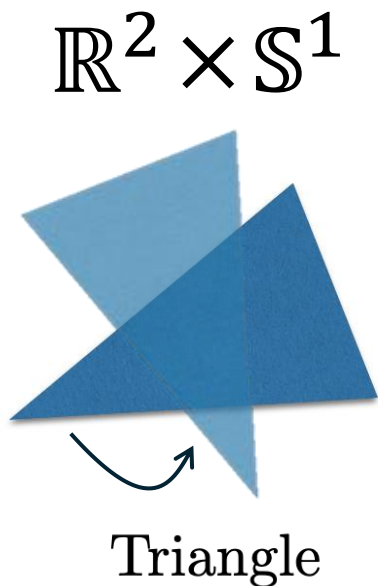
The configuration that the robot can reach

workspace



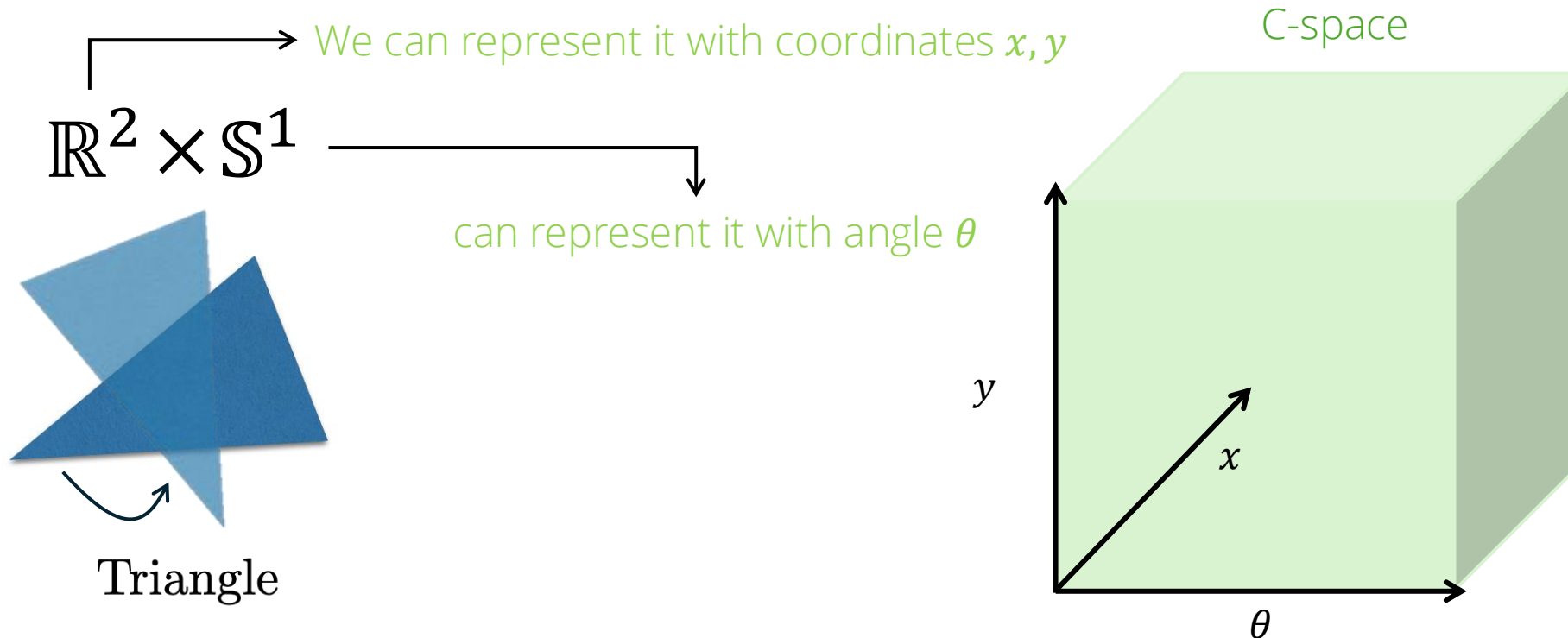
The Configuration Space

- The configuration of a robot is a complete specification of the position of every point of the robot
- Configuration space (C-space) is the n-dimensional space containing all possible configurations of the robot. In other words, C-space includes the set of all rigid-body transformations that can be applied to the robot.



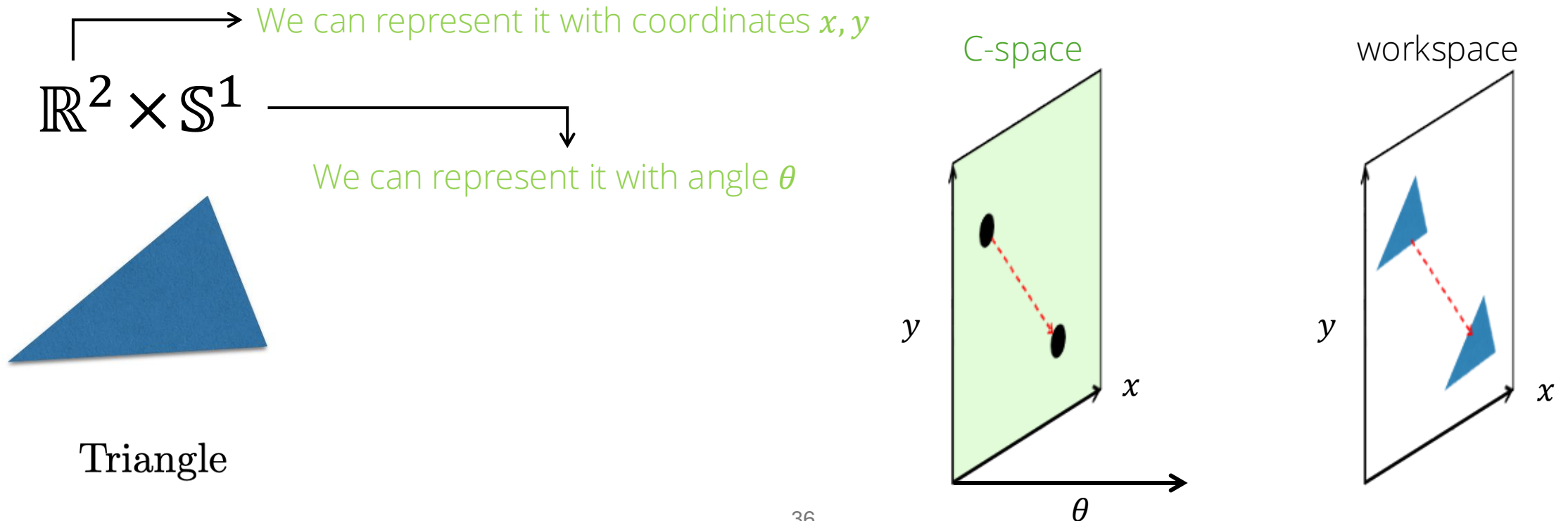
The Configuration Space

- The configuration of a robot is a complete specification of the position of every point of the robot
- Configuration space (C-space) is the n-dimensional space containing all possible configurations of the robot. In other words, C-space includes the set of all rigid-body transformations that can be applied to the robot.



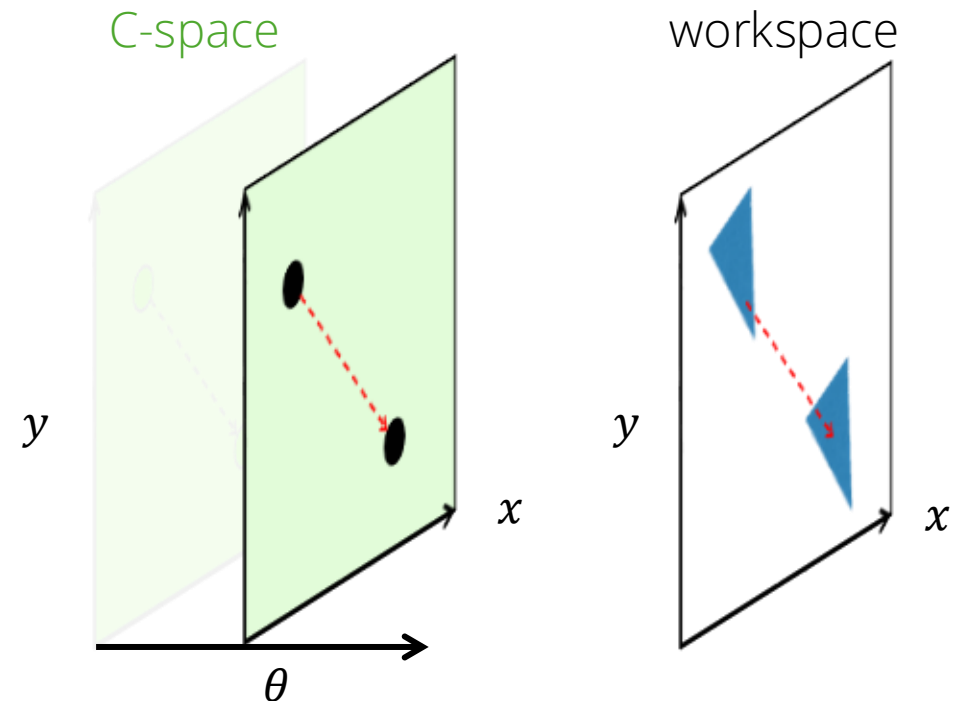
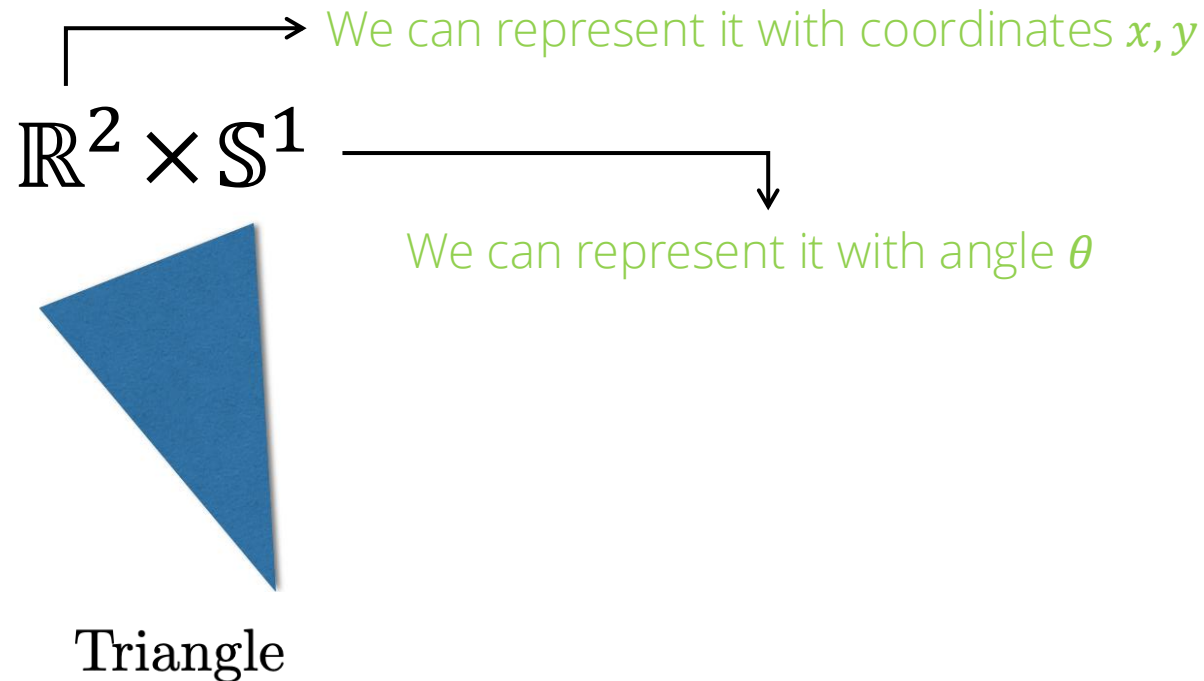
The Configuration Space

- The configuration of a robot is a complete specification of the position of every point of the robot
- Configuration space (C-space) is the n-dimensional space containing all possible configurations of the robot. In other words, C-space includes the set of all rigid-body transformations that can be applied to the robot.



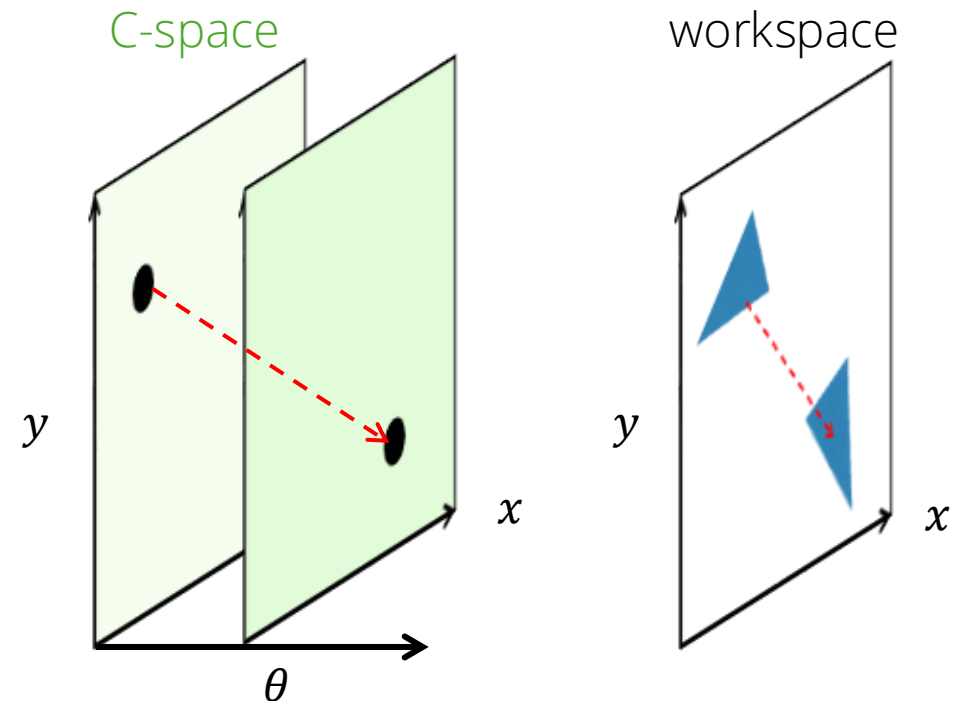
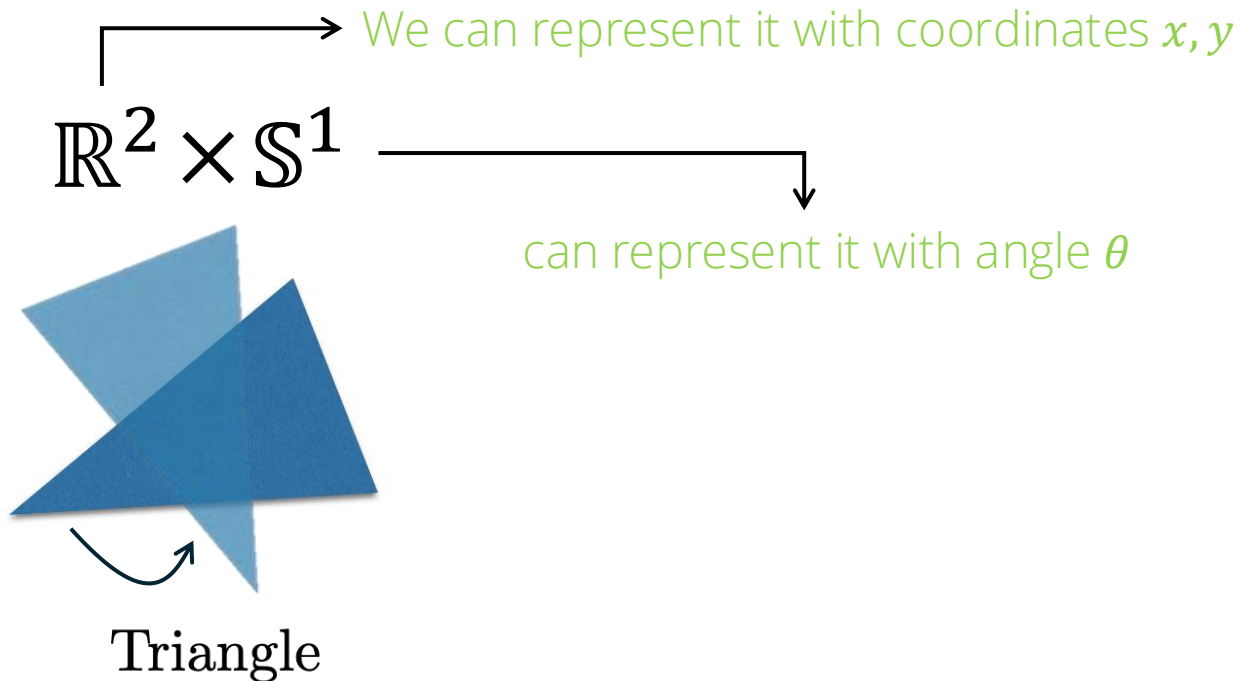
The Configuration Space

- The configuration of a robot is a complete specification of the position of every point of the robot
- Configuration space (C-space) is the n-dimensional space containing all possible configurations of the robot. In other words, C-space includes the set of all rigid-body transformations that can be applied to the robot.



The Configuration Space

- The configuration of a robot is a complete specification of the position of every point of the robot
- Configuration space (C-space) is the n-dimensional space containing all possible configurations of the robot. In other words, C-space includes the set of all rigid-body transformations that can be applied to the robot.



The Configuration Space

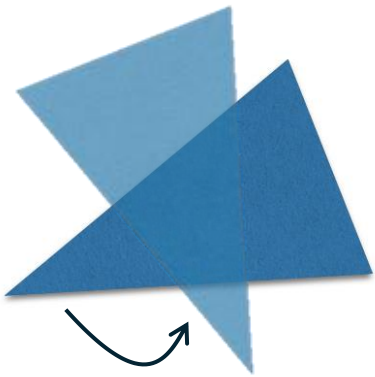
- The configuration of a robot is a complete specification of the position of every point of the robot
- Configuration space (C-space) is the n-dimensional space containing all possible configurations of the robot. In other words, C-space includes the set of all rigid-body transformations that can be applied to the robot.



The Configuration Space

- The configuration of a robot is a complete specification of the position of every point of the robot
- Configuration space (C-space) is the n-dimensional space containing all possible configurations of the robot. In other words, C-space includes the set of all rigid-body transformations that can be applied to the robot.

$$\mathbb{R}^2 \times \mathbb{S}^1$$



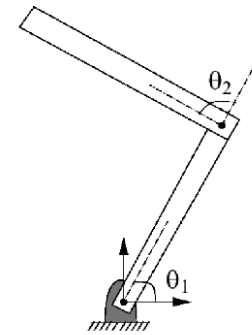
Triangle

$$\mathbb{R}^2 \times \mathbb{S}^1$$



Racecar

$$\mathbb{S}^1 \times \mathbb{S}^1$$



2-joint planar arm

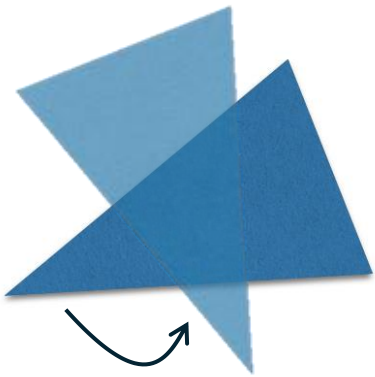
We can represent it with angle θ_1, θ_2

The Configuration Space

- The configuration of a robot is a complete specification of the position of every point of the robot
- Configuration space (C-space) is the n-dimensional space containing all possible configurations of the robot. In other words, C-space includes the set of all rigid-body transformations that can be applied to the robot.

We can represent it with angle $\theta_1, \theta_2, \dots, \theta_n$

$$\mathbb{R}^2 \times \mathbb{S}^1$$



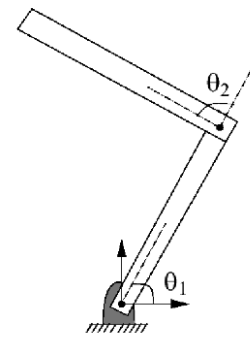
Triangle

$$\mathbb{R}^2 \times \mathbb{S}^1$$



Racecar

$$\mathbb{S}^1 \times \mathbb{S}^1$$



2-joint planar arm

$$\mathbb{S}^1 \times \dots \times \mathbb{S}^1$$



Manipulator

How to Specify Obstacles in the Configuration Space?

Robot operates in a 2D / 3D workspace $\mathcal{W} = \mathbb{R}^2$ or $\mathbb{R}^3$

Subset of this space is obstacles $\mathcal{O} \subset \mathcal{W}$

semi-algebraic models (polygons, polyhedra)

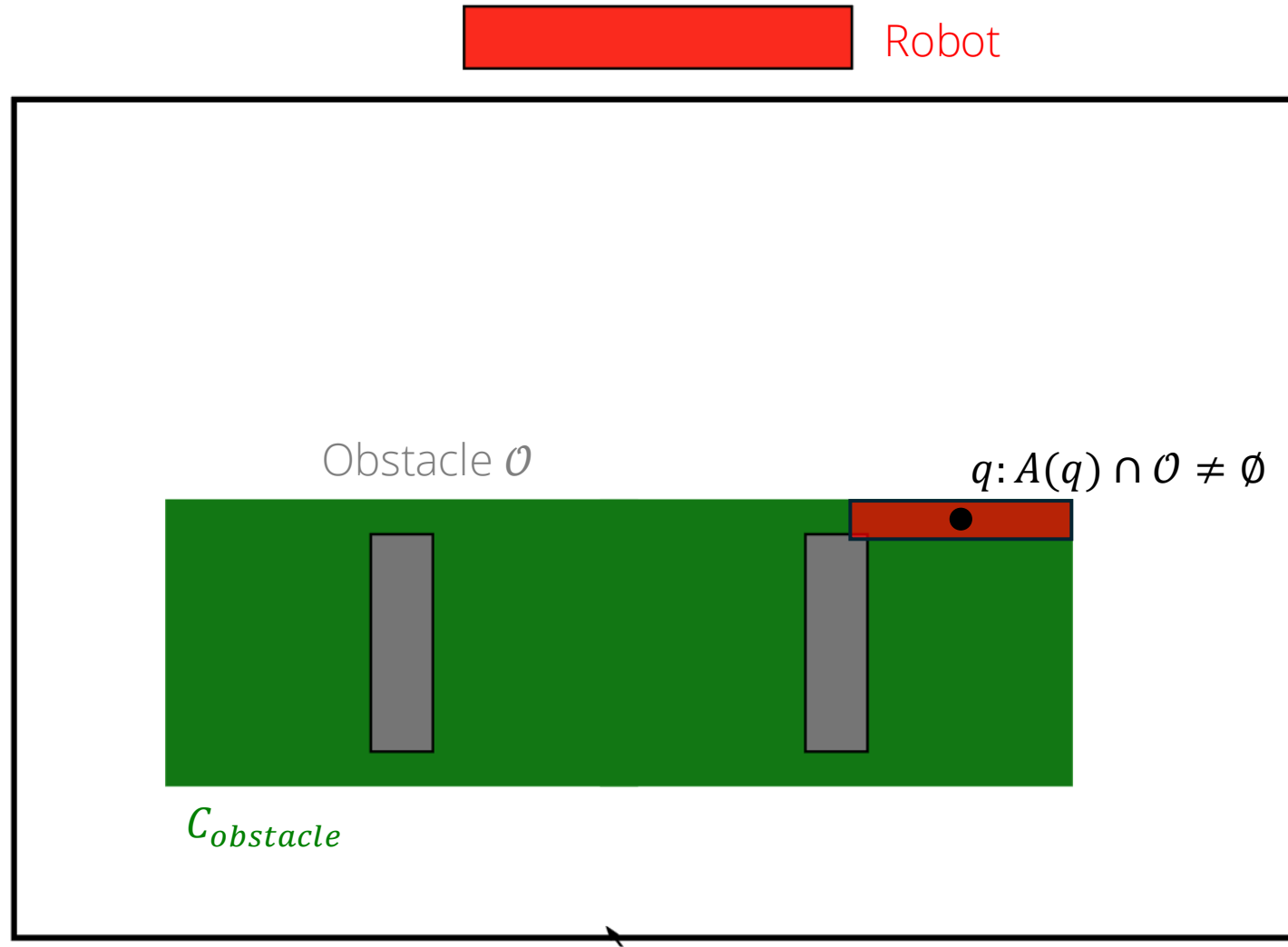
Geometric shape of the robot
(set of points occupied by robot at a config) $\mathcal{A}(q) \subset \mathcal{W}$

C-space obstacle region

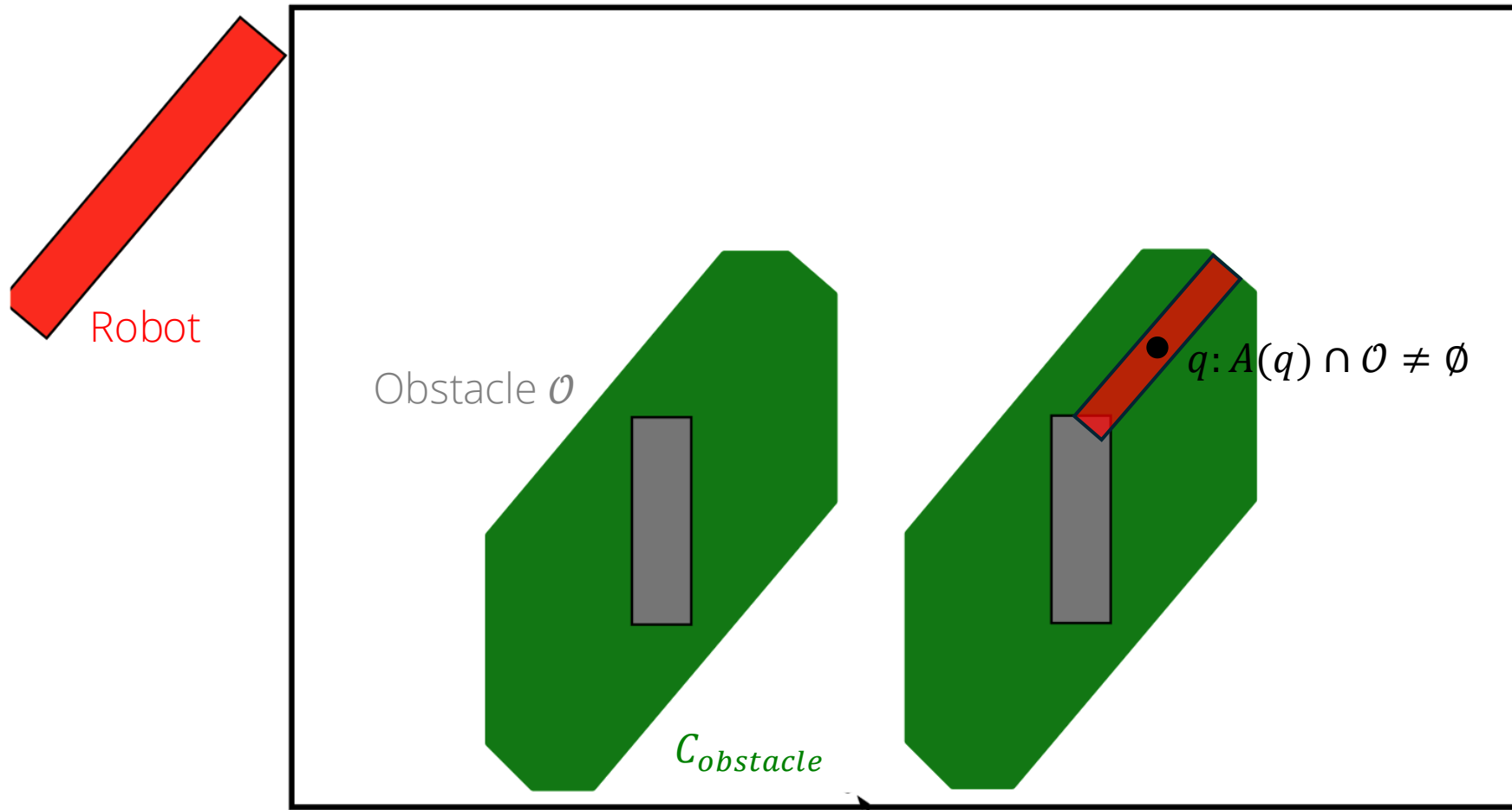
$$\mathcal{C}_{obs} = \{\mathbf{q} \in \mathcal{C} \mid \mathcal{A}(\mathbf{q}) \cap \mathcal{O} \neq \emptyset\}$$

$$\mathcal{C}_{free} = \mathcal{C} \setminus \mathcal{C}_{obs}$$

Example: Rotational Motion

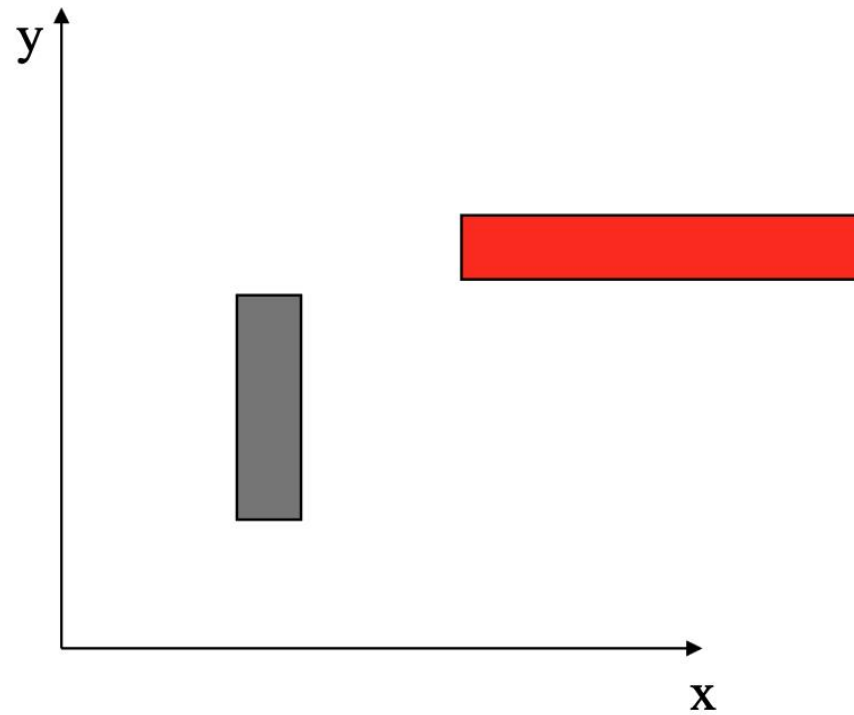


Example: Rotational Motion

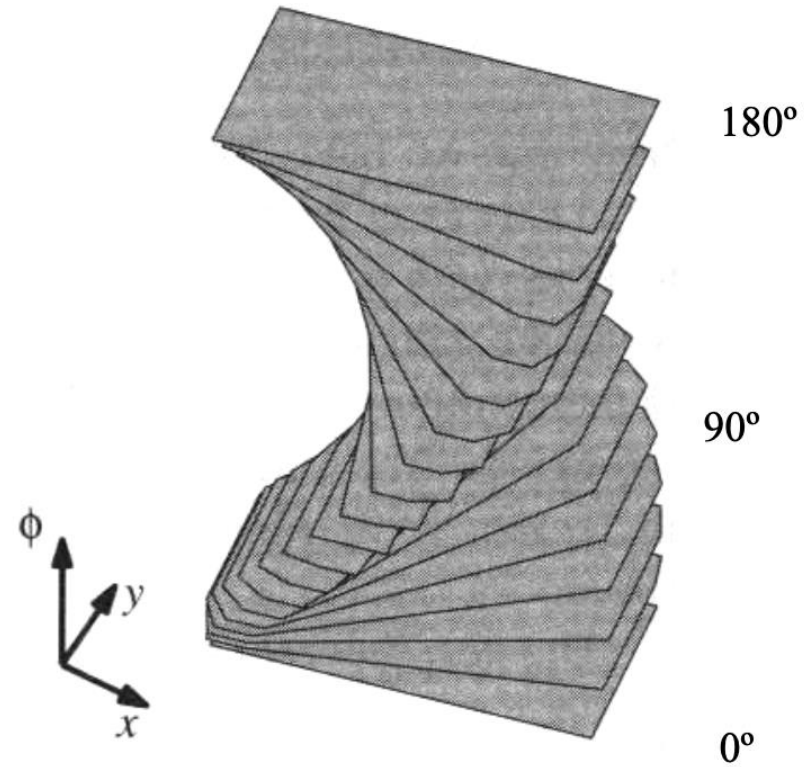


What would the configuration space of a rectangular robot (red) in this world look like?

(The obstacle is blue.)



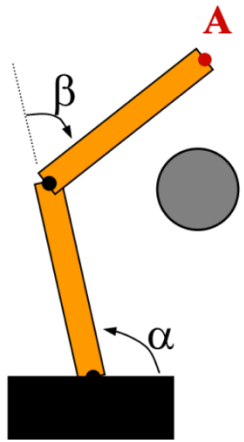
configuration space



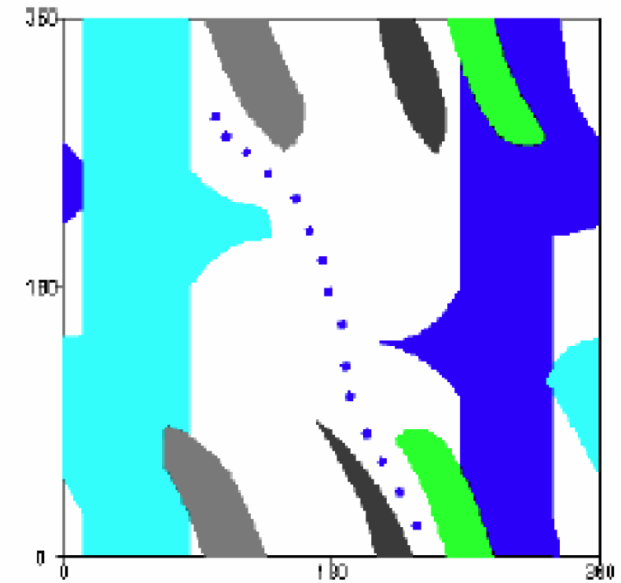
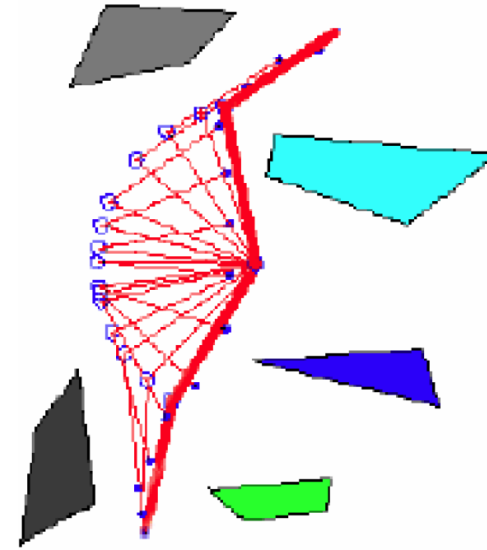
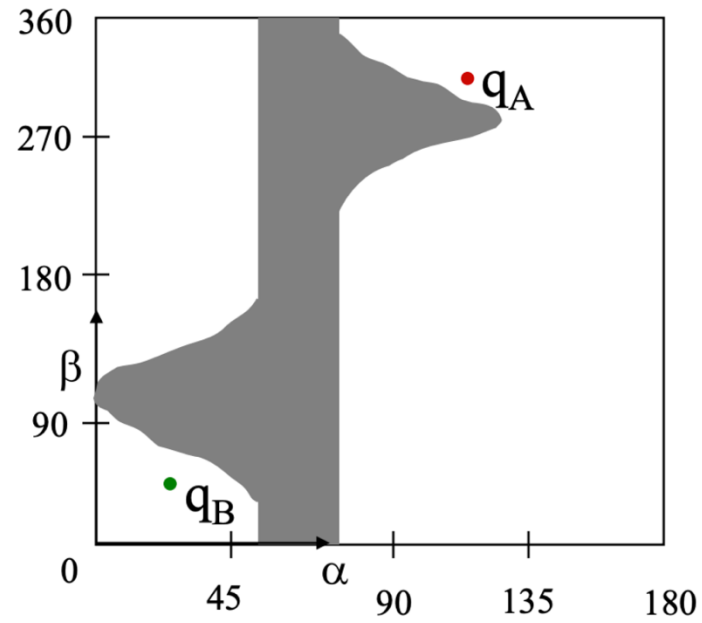
16-735, Howie Choset with slides from G.D. Hager and Z. Dodds

this is twisted...

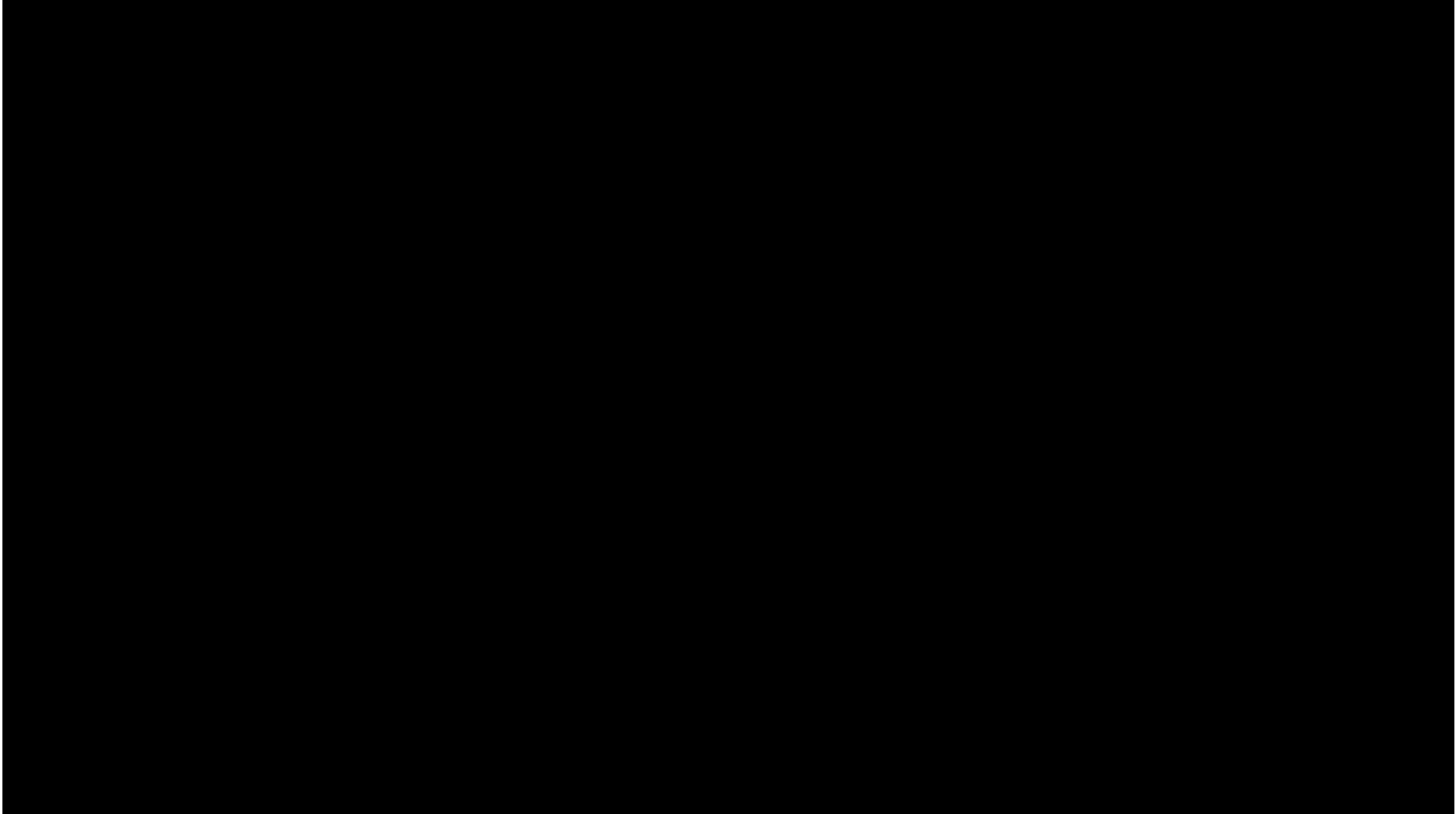
Example: 2-Link Planar Arm



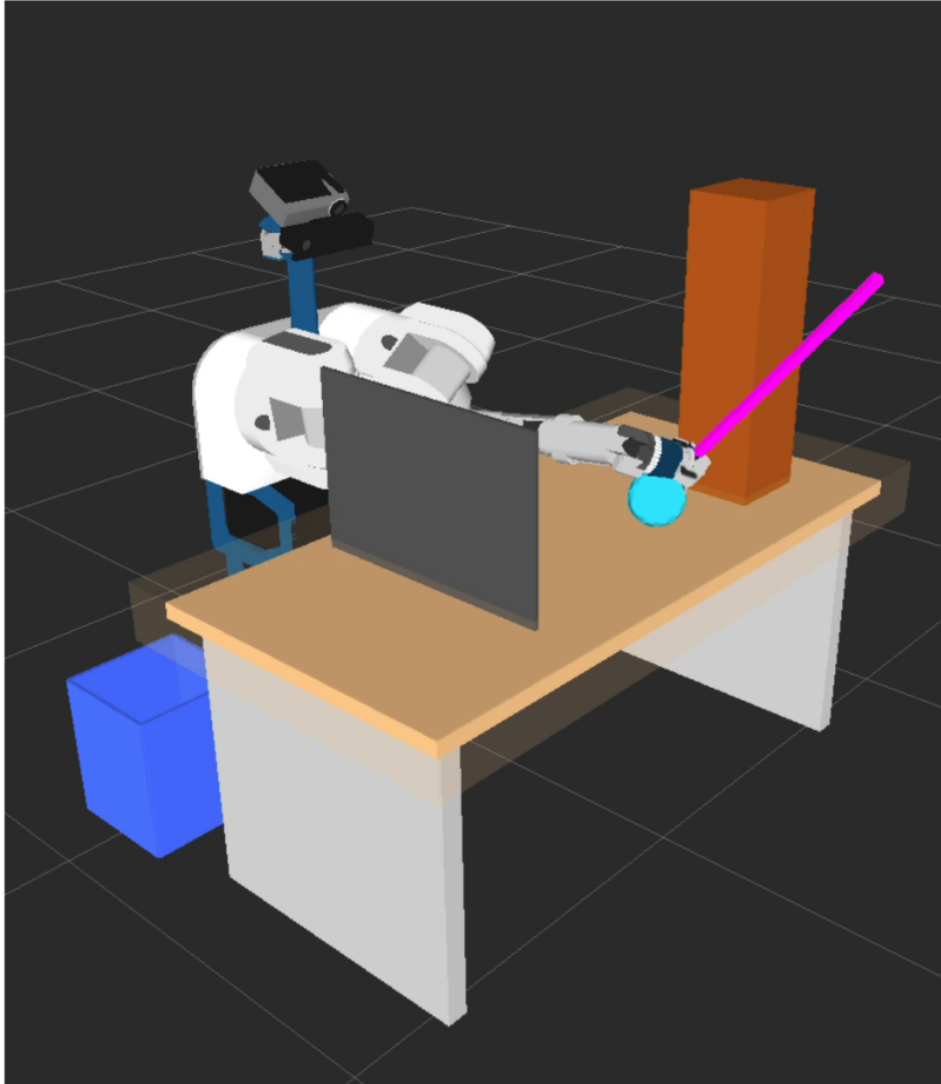
B



Motion Planning is Hard....



Geometric Path Planning Problem



Also known as
Piano Mover's Problem (Reif 79)

Given:

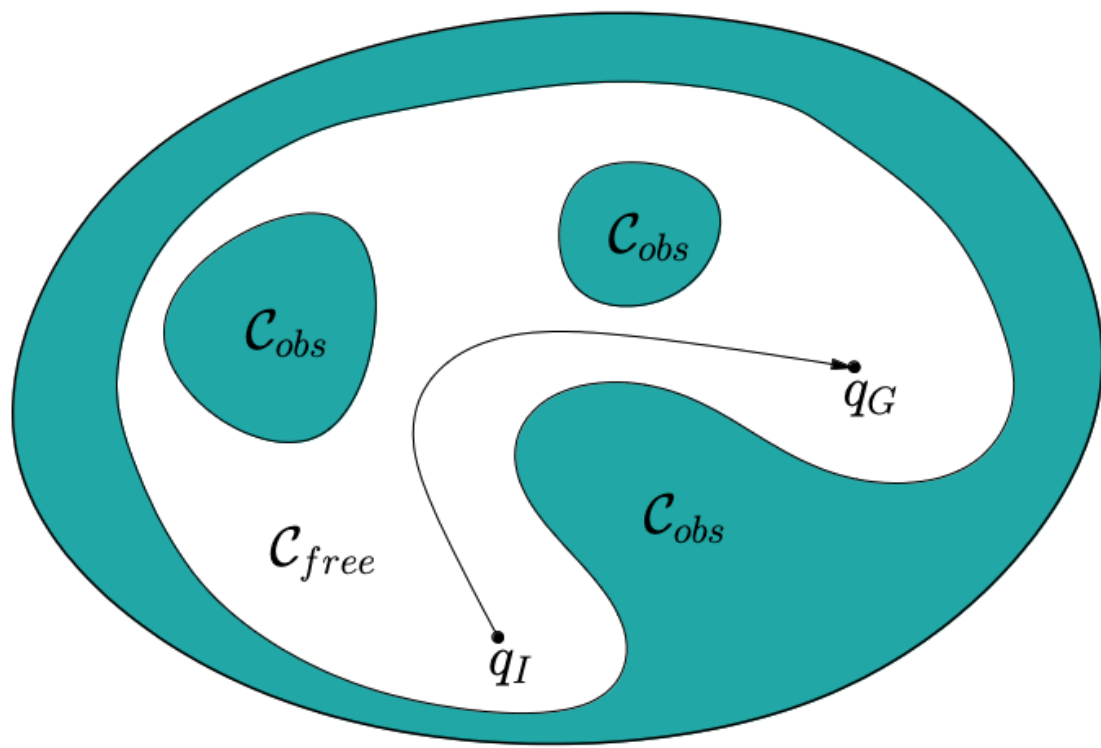
1. A *workspace* $\mathcal{W}$, where either $\mathcal{W} = \mathbb{R}^2$ or $\mathcal{W} = \mathbb{R}^3$.
2. An *obstacle region* $\mathcal{O} \subset \mathcal{W}$.
3. A *robot* defined in $\mathcal{W}$. Either a rigid body $\mathcal{A}$ or a collection of m links: $\mathcal{A}_1, \mathcal{A}_2, \dots, \mathcal{A}_m$.
4. The *configuration space* $\mathcal{C}$ ($\mathcal{C}_{obs}$ and $\mathcal{C}_{free}$ are then defined).
5. An *initial configuration* $\mathbf{q}_I \in \mathcal{C}_{free}$.
6. A *goal configuration* $\mathbf{q}_G \in \mathcal{C}_{free}$. The initial and goal configuration are often called a *query* $(\mathbf{q}_I, \mathbf{q}_G)$.

Compute a (continuous) path, $\tau : [0, 1] \rightarrow \mathcal{C}_{free}$, such that $\tau(0) = \mathbf{q}_I$ and $\tau(1) = \mathbf{q}_G$.

Also may want to minimize cost
 $c(\tau)$

Geometric Path Planning Problem

We can apply previous techniques (e.g. grid-search, visibility graph, cell decomposition) here!



Also known as

Piano Mover's Problem (Reif 79)

Given:

1. A *workspace* $\mathcal{W}$, where either $\mathcal{W} = \mathbb{R}^2$ or $\mathcal{W} = \mathbb{R}^3$.
2. An *obstacle region* $\mathcal{O} \subset \mathcal{W}$.
3. A *robot* defined in $\mathcal{W}$. Either a rigid body $\mathcal{A}$ or a collection of m links: $\mathcal{A}_1, \mathcal{A}_2, \dots, \mathcal{A}_m$.
4. The *configuration space* $\mathcal{C}$ ($\mathcal{C}_{obs}$ and $\mathcal{C}_{free}$ are then defined).
5. An *initial configuration* $q_I \in \mathcal{C}_{free}$.
6. A *goal configuration* $q_G \in \mathcal{C}_{free}$. The initial and goal configuration are often called a *query* (q_I, q_G) .

Compute a (continuous) path, $\tau : [0, 1] \rightarrow \mathcal{C}_{free}$, such that $\tau(0) = q_I$ and $\tau(1) = q_G$.

Also may want to minimize cost
 $c(\tau)$

Motion Planning is Hard....



10 vertices per dim

Dimension d	# vertices
2	100
3	1,000
6	1,000,000
8	100,000,000
10	10,000,000,000
15	1,000,000,000,000,000
20	100,000,000,000,000,000,000

- n-joint robot has n-dim C-space
- Assume we have M vertices per dim. The n-dim C-space has M^n vertices...
- In high dimensions:
 - Computing the C-space obstacle is hard
 - Planning is hard

Motion Planning is Hard....



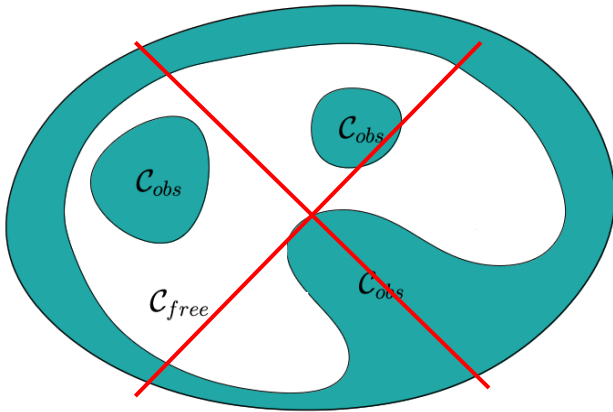
10 vertices per dim

- n-joint robot has n-dim C-space
 - Assume we have M vertices per dim. The n-dim C-space has M^n vertices...
 - In high dimensions:
 - Computing the C-space obstacle is hard
Solution: Don't compute C_{obs} explicitly, instead we query a collision detector
 - Planning is hard
Solution: Don't search a path from all vertices, but sampled vertices
 - Idea: Don't compute until being queried!
- n-joint robot has n-dim C-space
 - Assume we have M vertices per dim. The n-dim C-space has M^n vertices...
 - In high dimensions:
 - Computing the C-space obstacle is hard
Solution: Don't compute C_{obs} explicitly, instead we query a collision detector
 - Planning is hard
Solution: Don't search a path from all vertices, but sampled vertices
 - Idea: Don't compute until being queried!

Implicit C-Obstacle Representations

- Feasibility query:

$$Feasible(q) = \begin{cases} 1, & \text{if } q \text{ is in the free space} \\ 0, & \text{if } q \text{ is in the obstacle space} \end{cases}$$



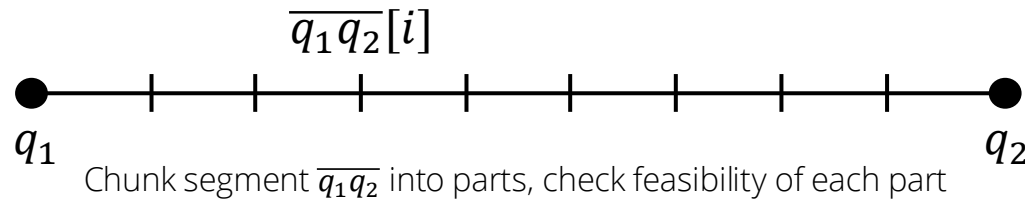
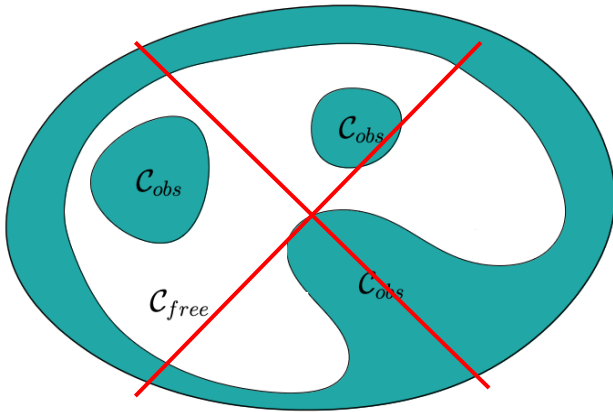
Implicit C-Obstacle Representations

- Feasibility query:

$$Feasible(q) = \begin{cases} 1, & \text{if } q \text{ is in the free space} \\ 0, & \text{if } q \text{ is in the obstacle space} \end{cases}$$

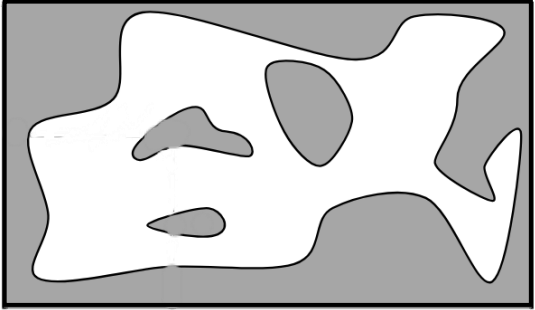
- Visibility query:

$$Visible(q_1, q_2) = \begin{cases} 1, & \text{if } \overline{q_1 q_2} \text{ is completely in the free space} \\ 0, & \text{if } \overline{q_1 q_2} \text{ intersects the obstacle space} \end{cases}$$



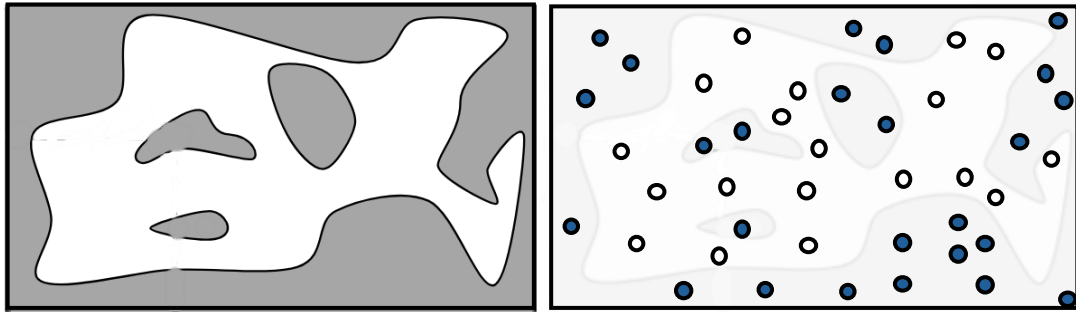
Probabilistic Roadmaps (PRM)

We don't have an explicit feasibility map. Just for reference...



RPM Algorithm:

Probabilistic Roadmaps (PRM)



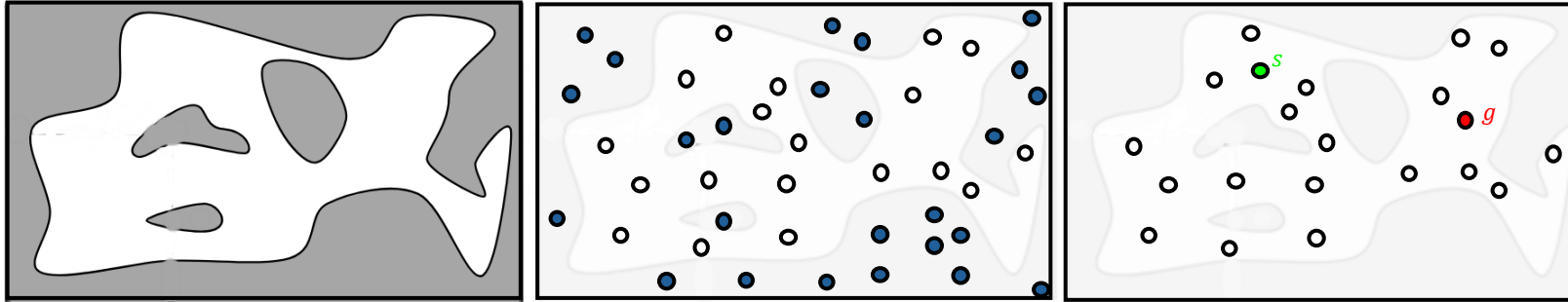
$Feasible(q)$

RPM Algorithm:

1. Sample random N configurations from the C-space and query their feasibilities

Probabilistic Roadmaps (PRM)

We don't have an explicit feasibility map. Just for reference...

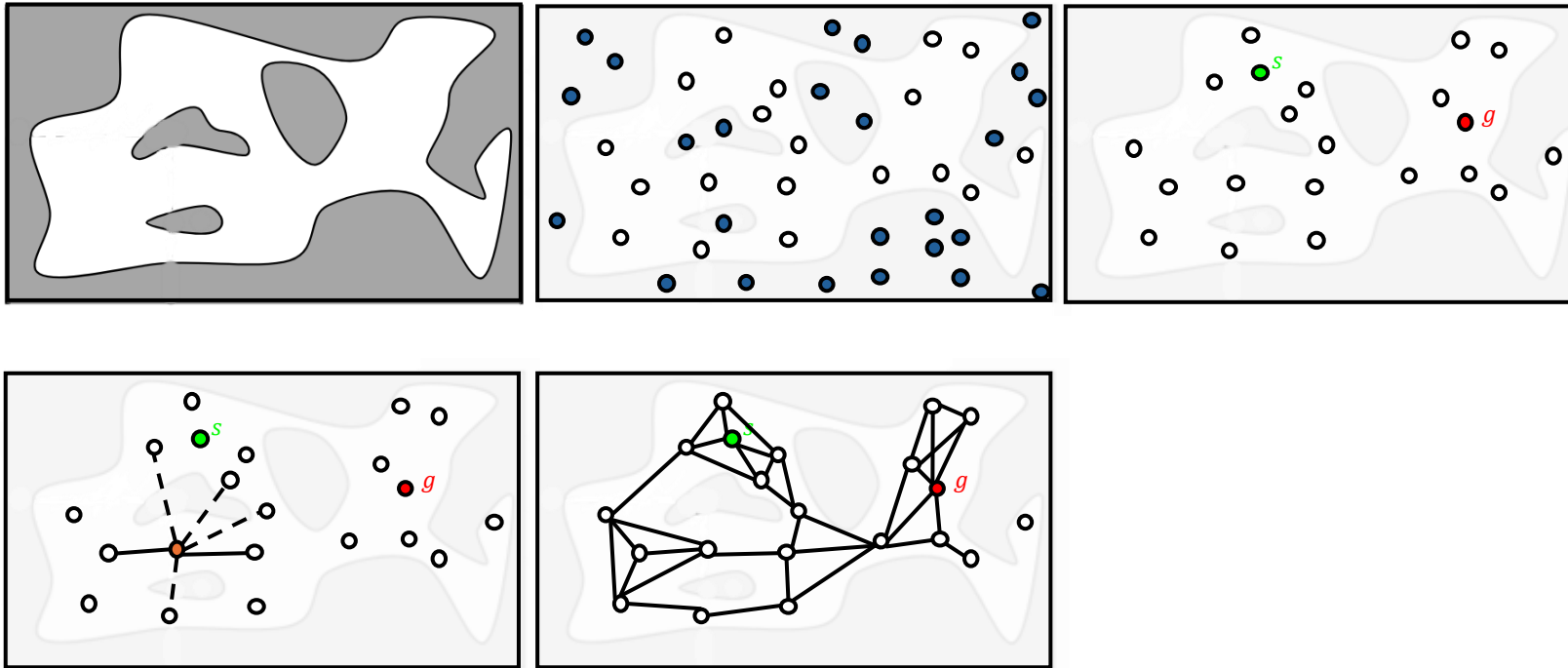


RPM Algorithm:

1. Sample random N configurations from the C-space and query their feasibilities
2. Add **milestones** (all feasible, the start and the goal configurations).

Probabilistic Roadmaps (PRM)

We don't have an explicit feasibility map. Just for reference...



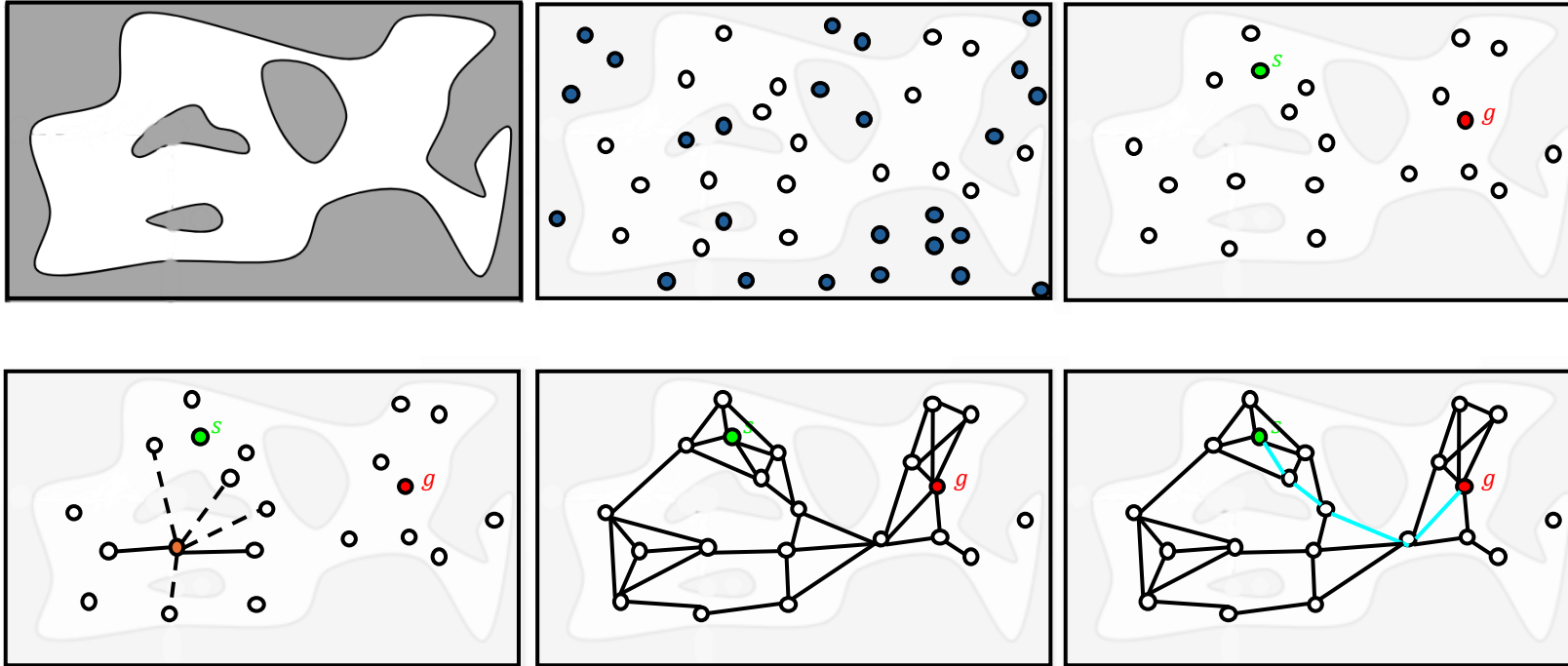
$Visible(q, p)$

RPM Algorithm:

1. Sample random N configurations from the C -space and query their feasibilities
2. Add **milestones** (all feasible, the start and the goal configurations).
3. Connect pairs of neighboring **milestones** if they are visible

Probabilistic Roadmaps (PRM)

We don't have an explicit feasibility map. Just for reference...



RPM Algorithm:

1. Sample random N configurations from the C -space and query their feasibilities
2. Add **milestones** (all feasible, the start and the goal configurations).
3. Connect pairs of neighboring **milestones** if they are visible
4. Search for a path from the start to the goal

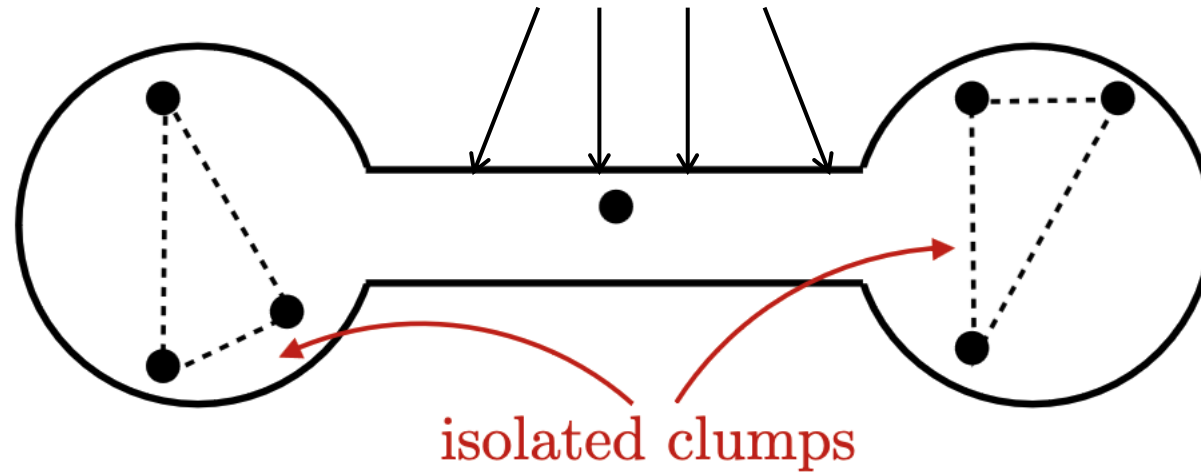
Probabilistic Roadmaps (PRM)

Algorithm Basic-PRM(s, g, N)

1. $V \leftarrow \{s, g\}$.
2. $E \leftarrow \{\}$.
3. **for** $i = 1, \dots, N$ **do**
4. $q \leftarrow \text{Sample}()$
5. **if** not Feasible(q) **then** return to Line 3.
6. Add q to V (add q as a new milestone)
7. **for** all $p \in \text{near}(q, V)$
8. **if** Visible(p, q) **then**
9. Add (p, q) to E .
10. Search $G = (V, E)$, with Cartesian distance as the edge cost, to connect s and g .
11. **return** the path if one is found.

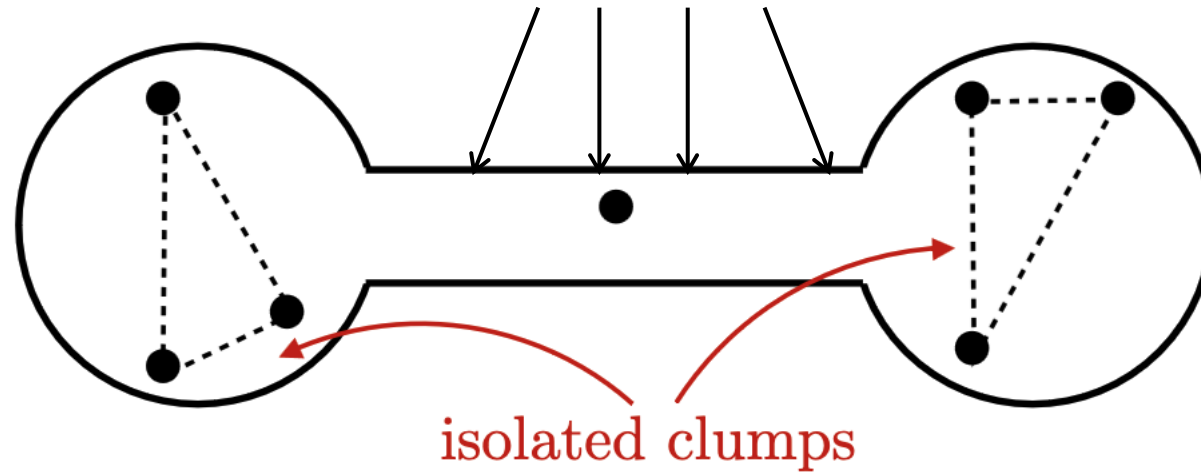
Is Uniformly Randomly Sampling Good Enough?

The narrow passage: We need more samples here



Is Uniformly Randomly Sampling Good Enough?

The narrow passage: We need more samples here

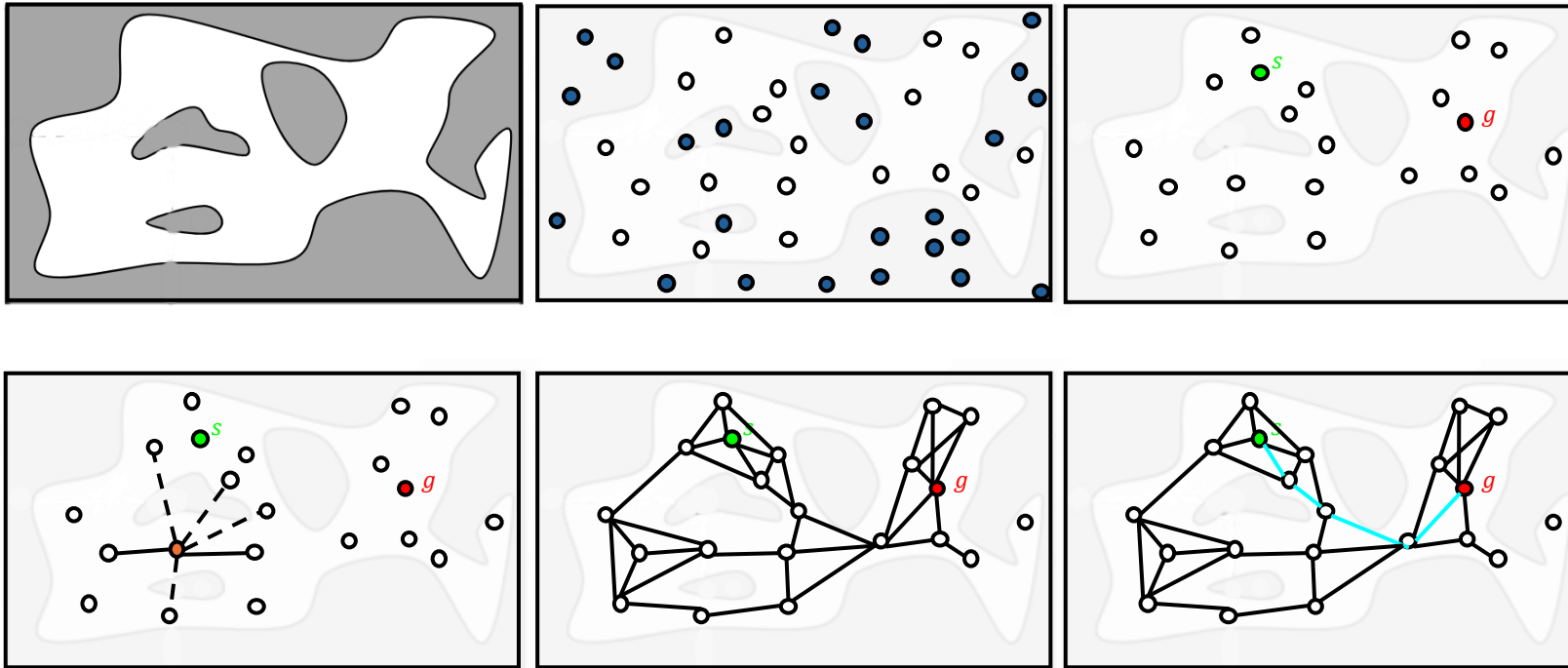


Solutions:

1. Sample near obstacle surface
2. Add samples that are in between two obstacles
3. Train a learner to detect the narrow passages

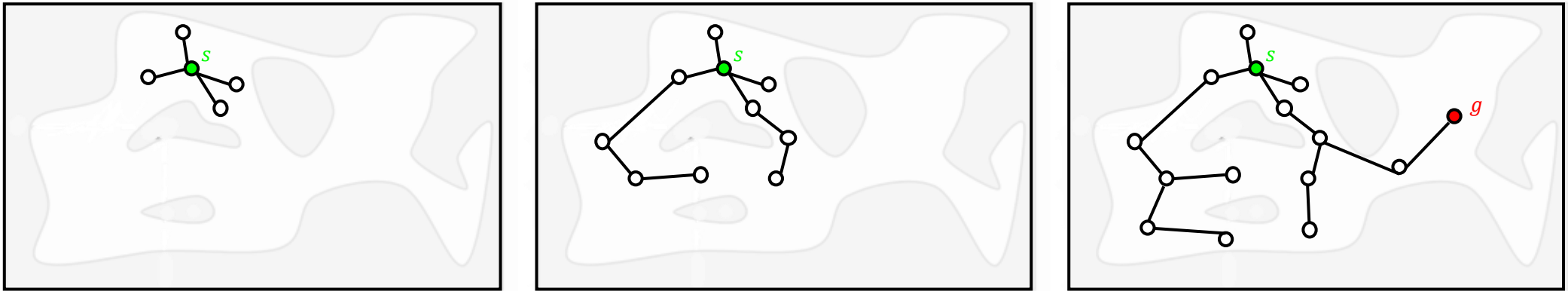
Quick summary of PRM

We don't have an explicit feasibility map. Just for reference...



- PRM randomly samples configurations to build a roadmap, hoping to cover the free space
- PRM is great when we want to reuse the graph. For example, plan many times with different start/goal pairs
- What if we just need to plan once? For example, a robot only needs to *pick up* the object once

Rapidly Exploring Random Trees (RRTs)



Grow a tree of feasible paths from the **start** to the **goal** configuration, instead of building a graph.

Rapidly Exploring Random Trees (RRTs)

BUILD_RRT(q_{init})

1 $\mathcal{T}.\text{init}(q_{init});$

2 **for** $k = 1$ **to** K **do**

3 $q_{rand} \leftarrow \text{RANDOM_CONFIG}();$

4 EXTEND($\mathcal{T}, q_{rand}$);

5 **Return** $\mathcal{T}$

Sample random configuration with probability p , and the goal with probability $1 - p$

Rapidly Exploring Random Trees (RRTs)

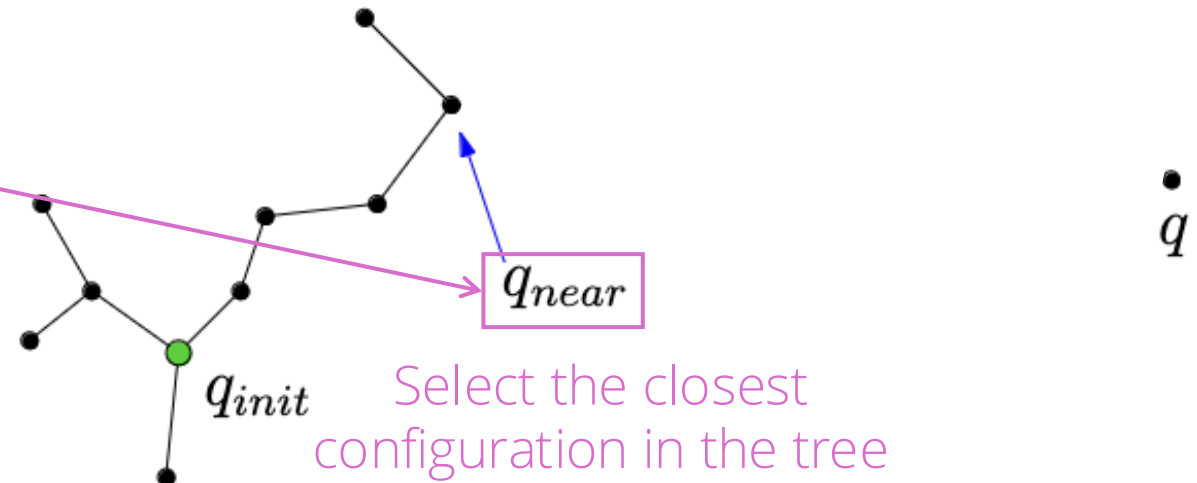
BUILD_RRT(q_{init})

```
1   $\mathcal{T}.$ init( $q_{init}$ );  
2  for  $k = 1$  to  $K$  do  
3       $q_{rand} \leftarrow$  RANDOM_CONFIG();  
4      EXTEND( $\mathcal{T}, q_{rand}$ );  
5  Return  $\mathcal{T}$ 
```

EXTEND($\mathcal{T}, q$)

```
1   $q_{near} \leftarrow$  NEAREST_NEIGHBOR( $q, \mathcal{T}$ );  
2  if NEW_CONFIG( $q, q_{near}, q_{new}$ ) then  
3       $\mathcal{T}.$ add_vertex( $q_{new}$ );  
4       $\mathcal{T}.$ add_edge( $q_{near}, q_{new}$ );  
5      if  $q_{new} = q$  then  
6          Return Reached;  
7      else  
8          Return Advanced;  
9  Return Trapped;
```

How to expand the tree? Expand from the node that is the closest to the configured q



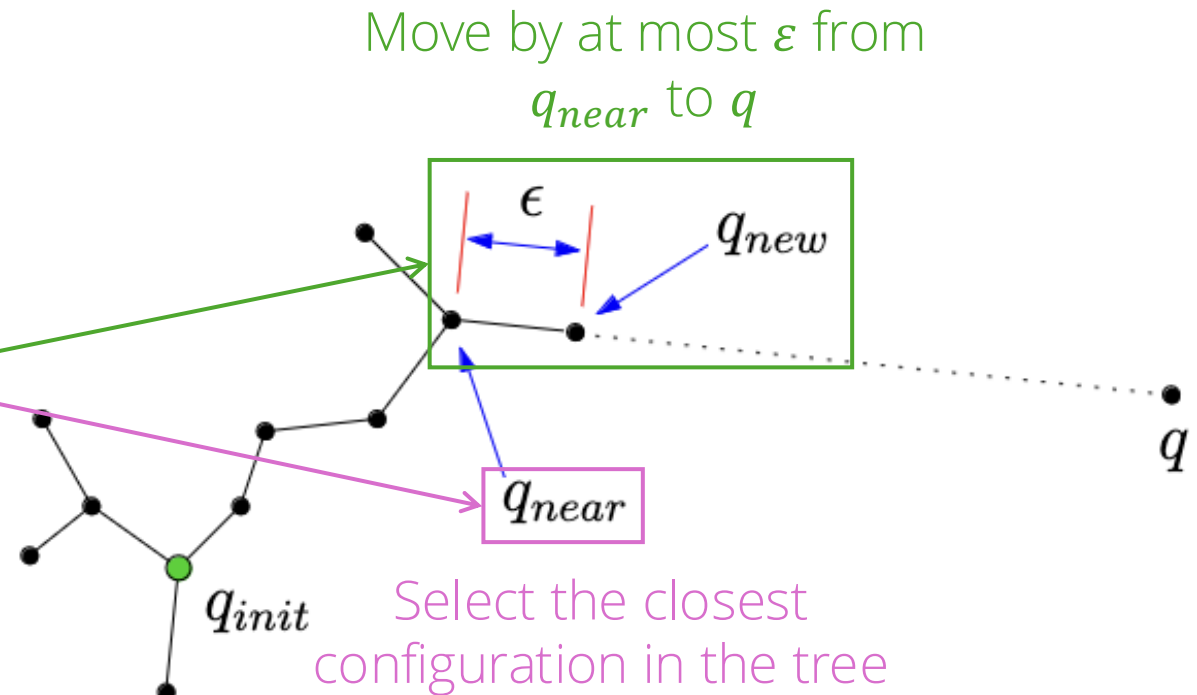
Rapidly Exploring Random Trees (RRTs)

BUILD_RRT(q_{init})

```
1   $\mathcal{T}.\text{init}(q_{init});$   
2  for  $k = 1$  to  $K$  do  
3       $q_{rand} \leftarrow \text{RANDOM\_CONFIG}();$   
4       $\text{EXTEND}(\mathcal{T}, q_{rand});$   
5  Return  $\mathcal{T}$ 
```

EXTEND($\mathcal{T}, q$)

```
1   $q_{near} \leftarrow \text{NEAREST\_NEIGHBOR}(q, \mathcal{T});$   
2  if  $\text{NEW\_CONFIG}(q, q_{near}, q_{new})$  then  
3       $\mathcal{T}.\text{add\_vertex}(q_{new});$   
4       $\mathcal{T}.\text{add\_edge}(q_{near}, q_{new});$   
5      if  $q_{new} = q$  then  
6          Return Reached;  
7      else  
8          Return Advanced;  
9  Return Trapped;
```



Expansion Strategy

BUILD_RRT(q_{init})

1 $\mathcal{T}.init(q_{init});$

2 **for** $k = 1$ **to** K **do**

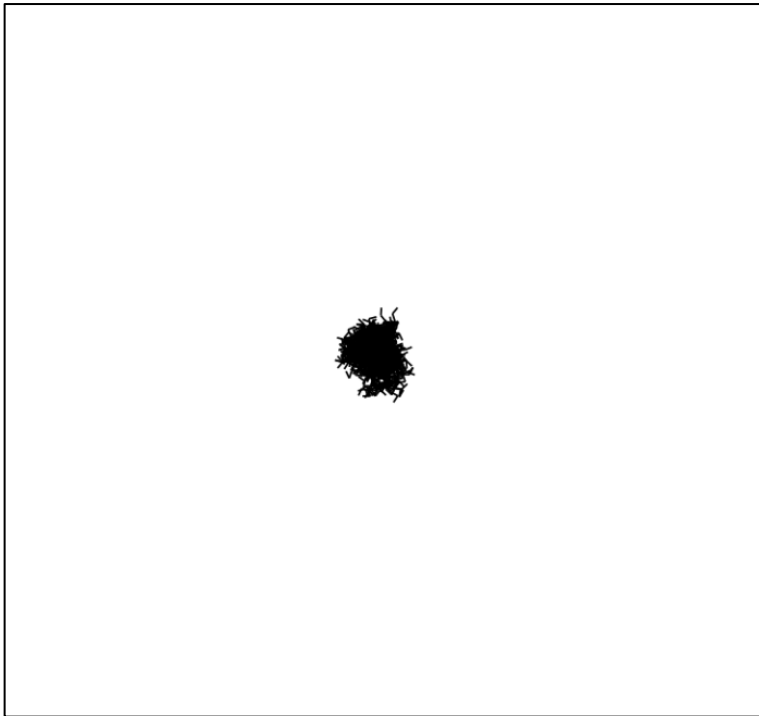
3 $q_{rand} \leftarrow$ RANDOM_CONFIG();

4 EXTEND($\mathcal{T}, q_{rand}$);

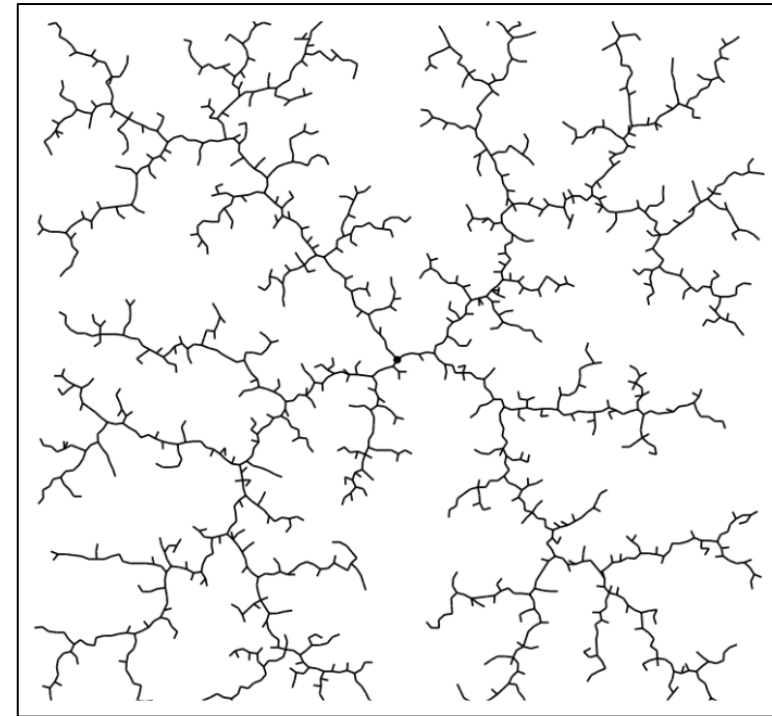
5 **Return** $\mathcal{T}$

Sample random configuration with probability p , and the goal with probability $1 - p$

Randomly uniformly sampling

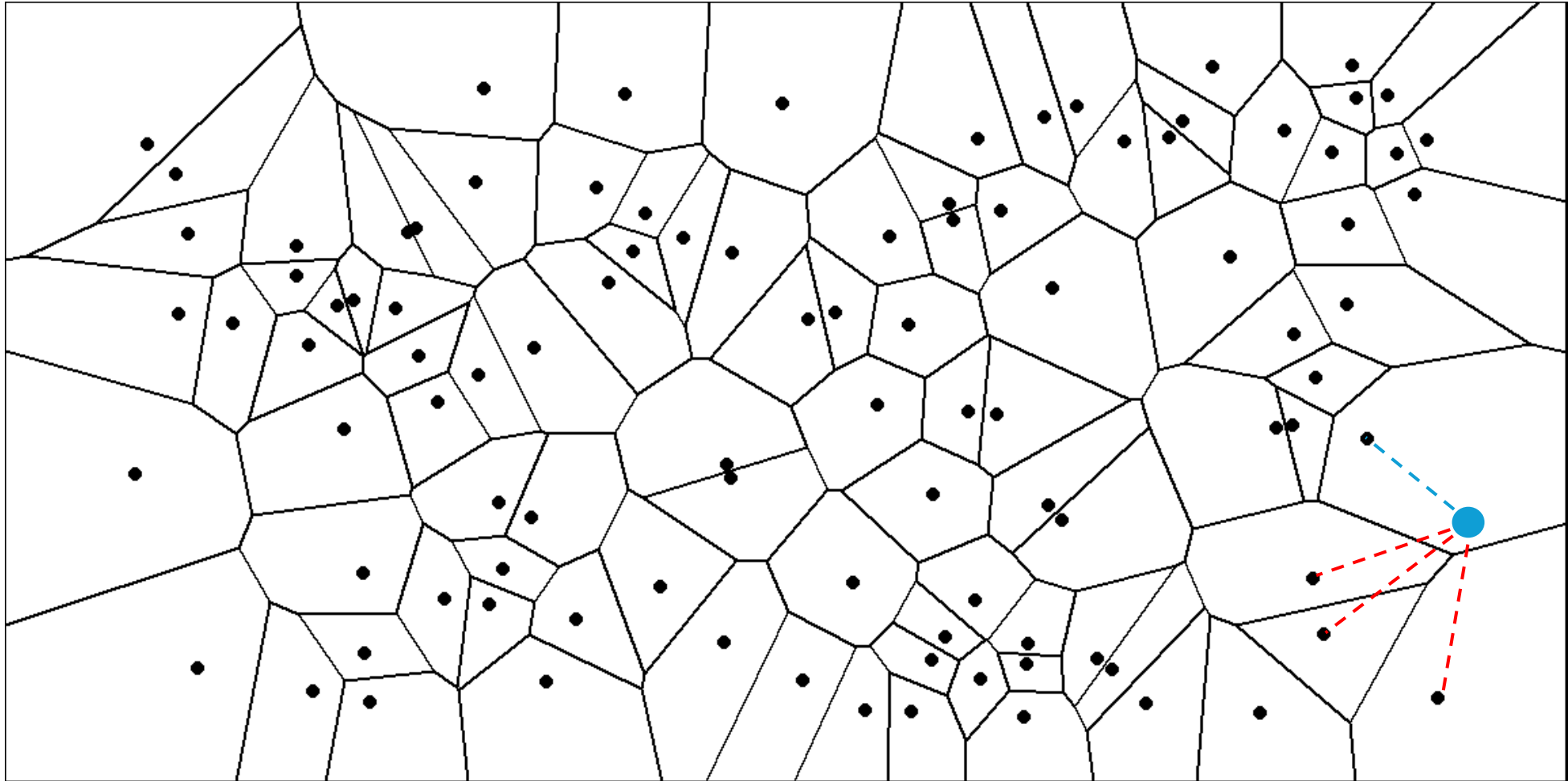


Biased sampling toward
unexplored regions

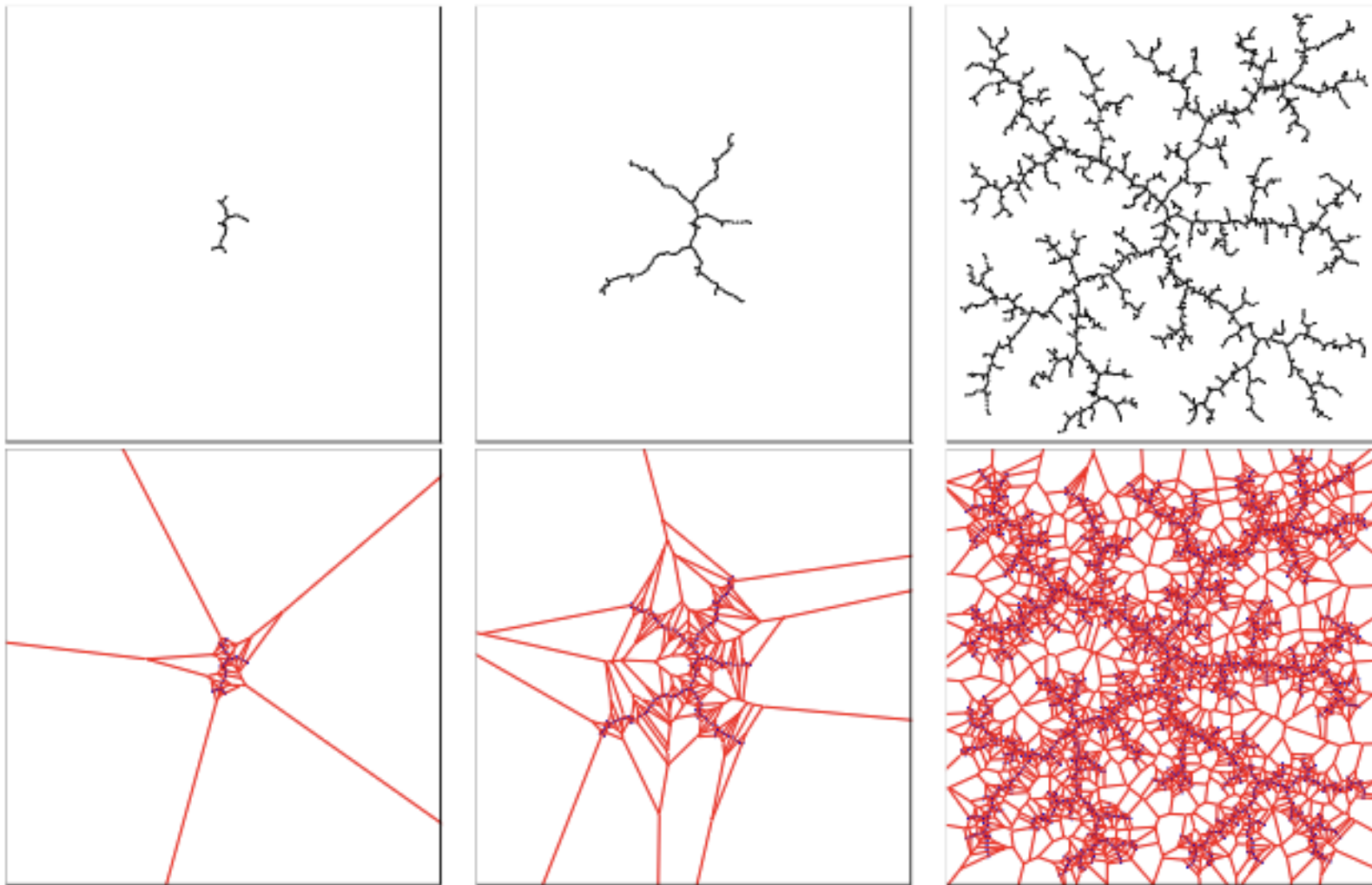


Voronoi Diagram

Voronoi diagram: nearest-neighbor segmentation. Assign each pixel to the nearest node. We can check the area of each region and explore more in the large ones.



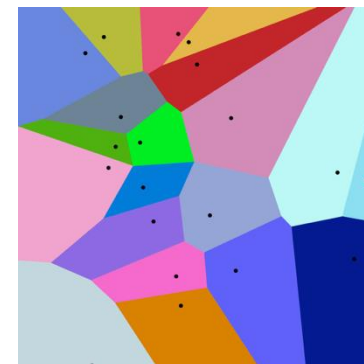
Voronoi Bias Strategy



Sample random configuration with probability p , which is proportional to the volume of its Voronoi cell.

- Bias sampling toward unexplored areas.

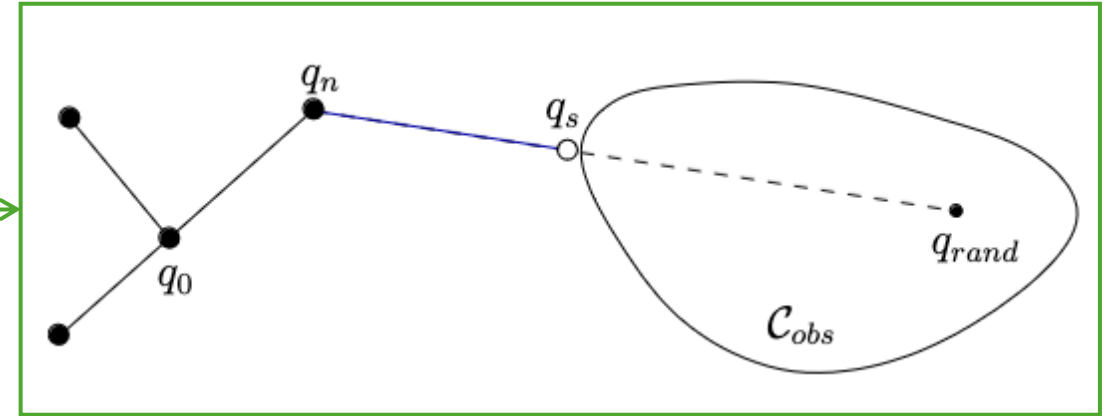
Voronoi diagram: nearest-neighbor segmentation



RRT-Connect: Bi-direction RRTs

```
CONNECT( $\mathcal{T}, q$ )  
1  repeat  
2     $S \leftarrow \text{EXTEND}(\mathcal{T}, q)$ ;  
3  until not ( $S = \text{Advanced}$ ) # new configuration is added  
4  Return  $S$ ;
```

```
RRT_CONNECT_PLANNER( $q_{init}, q_{goal}$ )  
1   $\mathcal{T}_a.\text{init}(q_{init})$ ;  $\mathcal{T}_b.\text{init}(q_{goal})$ ;  
2  for  $k = 1$  to  $K$  do  
3     $q_{rand} \leftarrow \text{RANDOM\_CONFIG}()$ ;  
4    if not ( $\text{EXTEND}(\mathcal{T}_a, q_{rand}) = \text{Trapped}$ ) then  
5      if ( $\text{CONNECT}(\mathcal{T}_b, q_{new}) = \text{Reached}$ ) then  
6        Return  $\text{PATH}(\mathcal{T}_a, \mathcal{T}_b)$ ;  
7    SWAP( $\mathcal{T}_a, \mathcal{T}_b$ );  
8  Return Failure
```

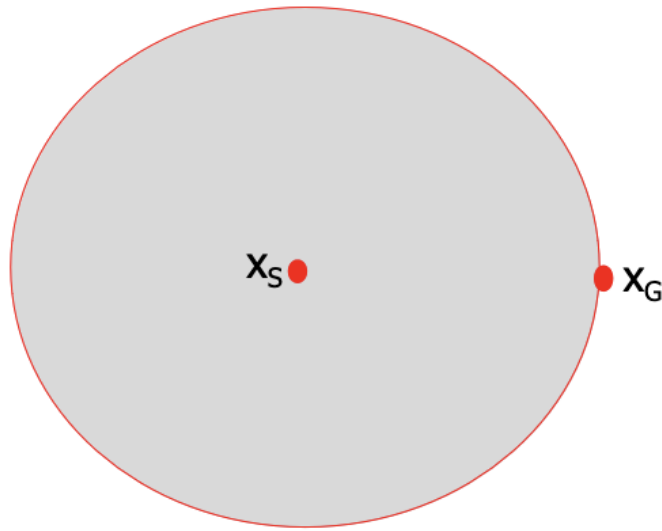


Greedy move from q_{near} to q_{rand}

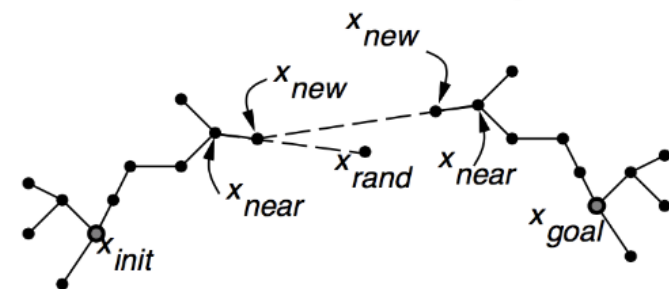
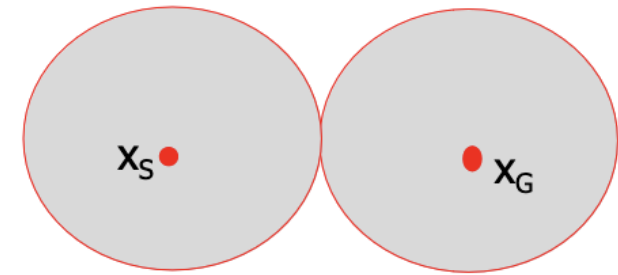
Grow two trees, one from the start and another from the goal

Single Tree vs. Double Trees

- Volume swept out by unidirectional RRT:

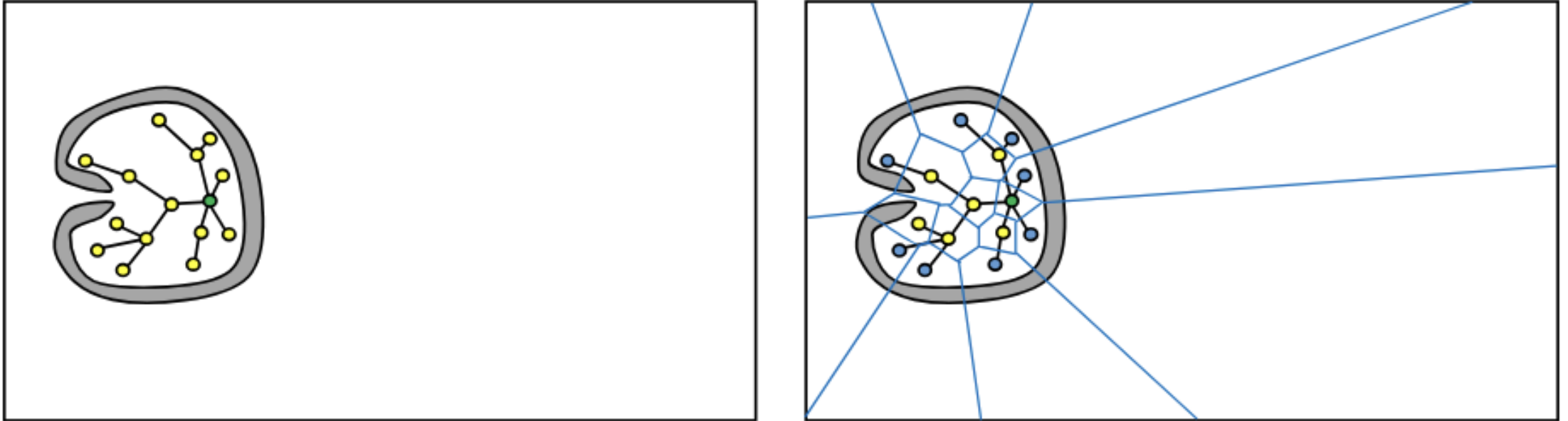


- Volume swept out by bi-directional RRT:



- Difference more and more pronounced as dimensionality increases

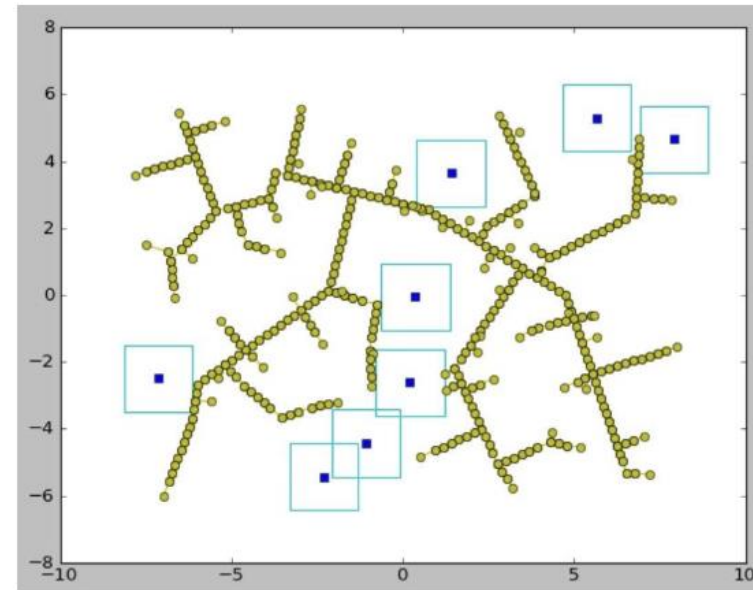
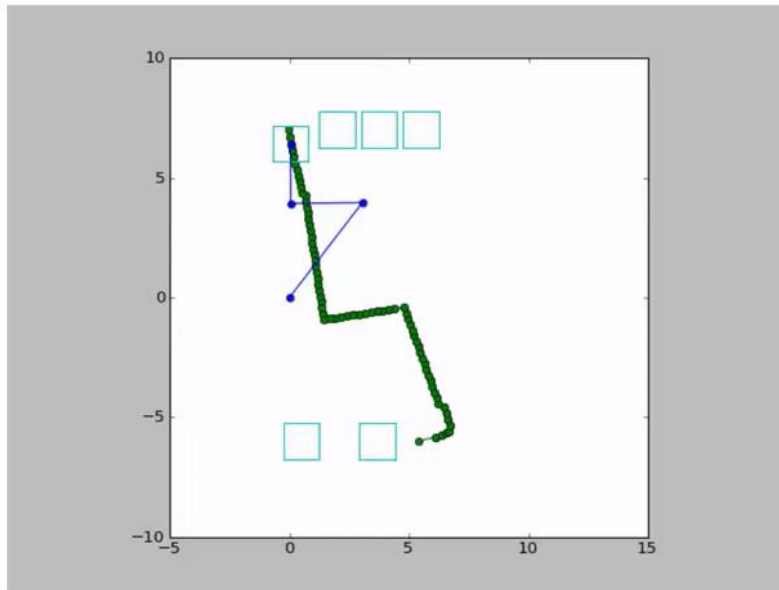
RRT Still Has Problems...



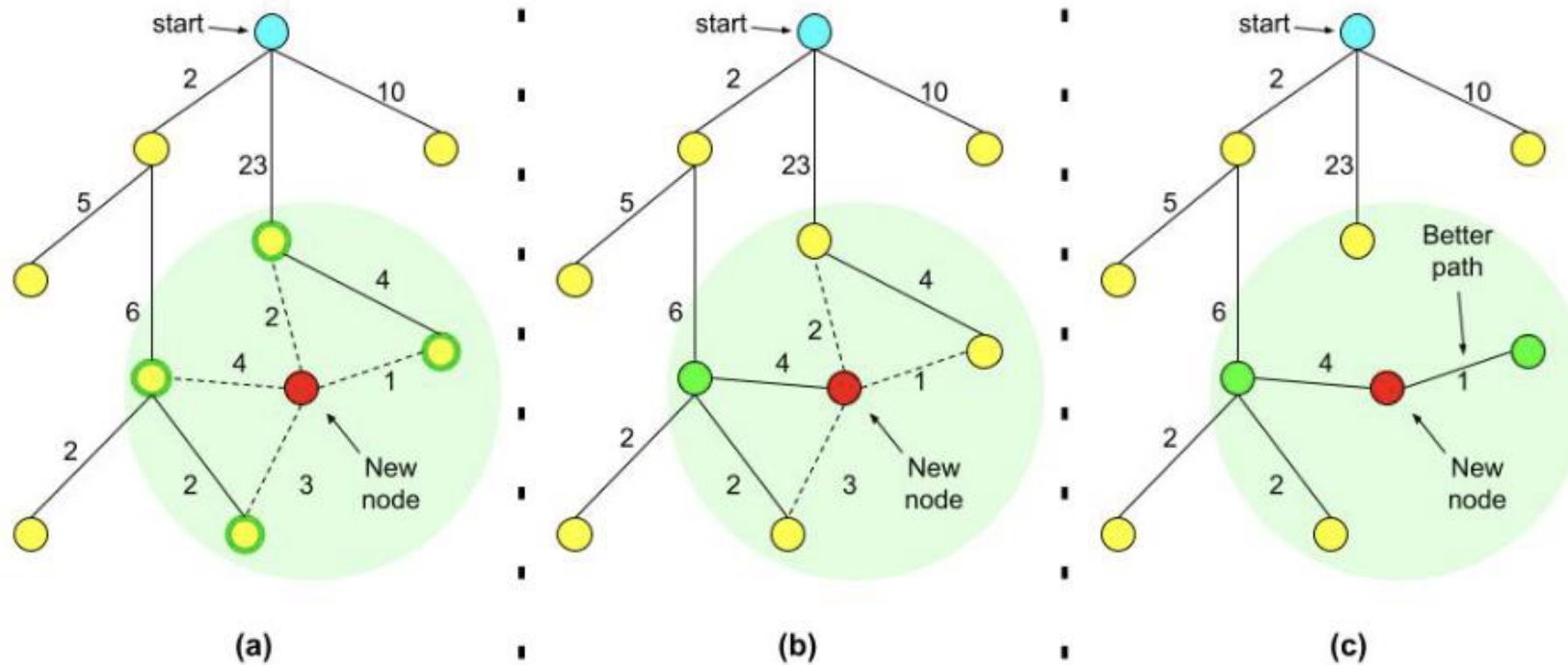
- The “bugtrap” problem: due to the Voronoi bias, RRT frequently attempts infeasible extensions
- To escape the mouth of a bugtrap, we need to sample a very carefully chosen sequence of milestones within the general area that it has already explored
- A tradeoff between exploring new regions and refine the roadmap of explored areas

RRT Still Has Problems...

- RRT guarantees probabilistic completeness but not optimality (shortest path)
 - In practice leads to paths that are very roundabout and non-direct -> not shortest paths



RRT*: RRT + Re-Wiring



- Can we find more optimal path passing through q_{new} ?
- Can we find more optimal path to q_{new} ?

RRT*: RRT + Re-Wiring

Algorithm RRT*

1. $T \leftarrow \{s\}..$
2. **for** $i = 1, \dots, N$ **do**
3. $q_{rand} \leftarrow \text{Sample}()$
4. $q_e \leftarrow \text{Extend-Tree}(T, q_{rand}, \delta)$
5. **if** $q_e \neq \text{nil}$ **then** $\text{Rewire}(T, q_e, |T|)$
6. **if** $d(q_e, g) \leq \delta$ and $\text{Visible}(q_e, g)$ **then**
7. Add edge $q_e \rightarrow g$ to T
8. $c(g) = \text{cost of optimal path from } s \text{ to } g, \text{ if } g \text{ is connected, and } \infty \text{ otherwise}$
9. **return** "no path"

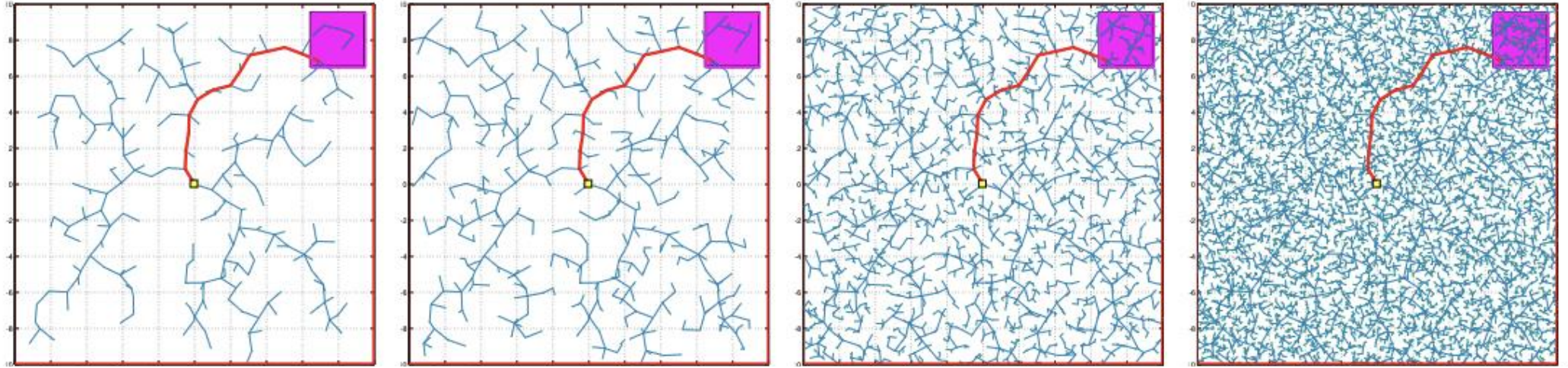
RRT*: RRT + Re-Wiring

Algorithm Rewire(T, q_{new}, n)

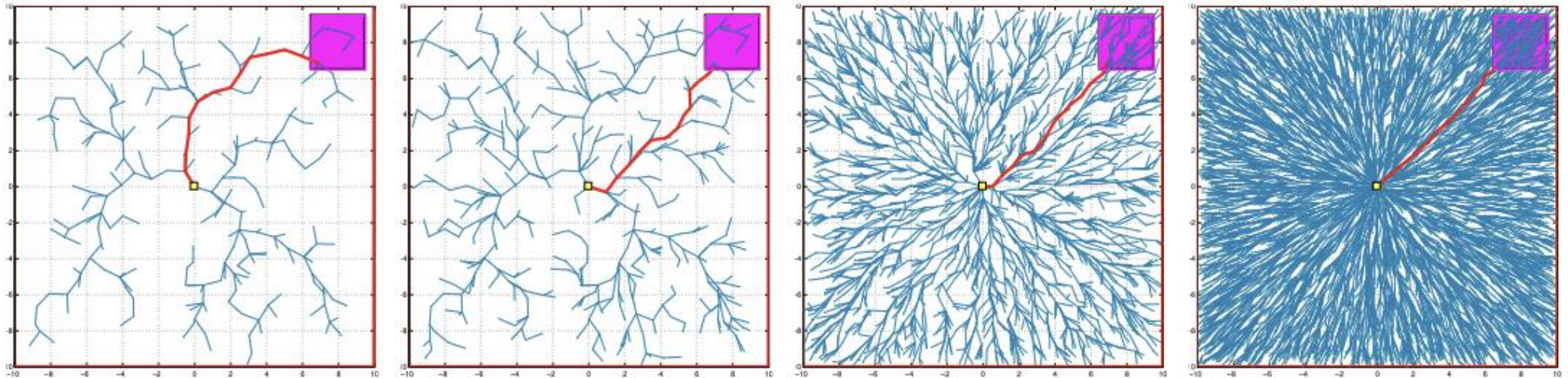
1. Neighbors $\leftarrow$ Set of $k^*(n)$ -nearest neighbors in T , or points in $R^*(n)$ -neighborhood.
2. **for** $q \in \text{Neighbors}$ sorted by increasing $c(q)$ **do**
3. **if** $c(q_{new}) + d(q_{new}, q) < c(q)$ **then** (optimal path to q passes through q_{new})
4. $c(q) \leftarrow c(q_{new}) + d(q_{new}, q)$
5. Update costs of descendants of q .
6. **if** $c(q) + d(q, q_{new}) < c(q_{new})$ **then** (optimal path to q_{new} passes through q)
7. $c(q_{new}) \leftarrow c(q) + d(q, q_{new})$
8. Set parent of q_{new} to q .
9. Revise costs and parents of descendants of q_{new} .

RRT vs. RRT*

RRT

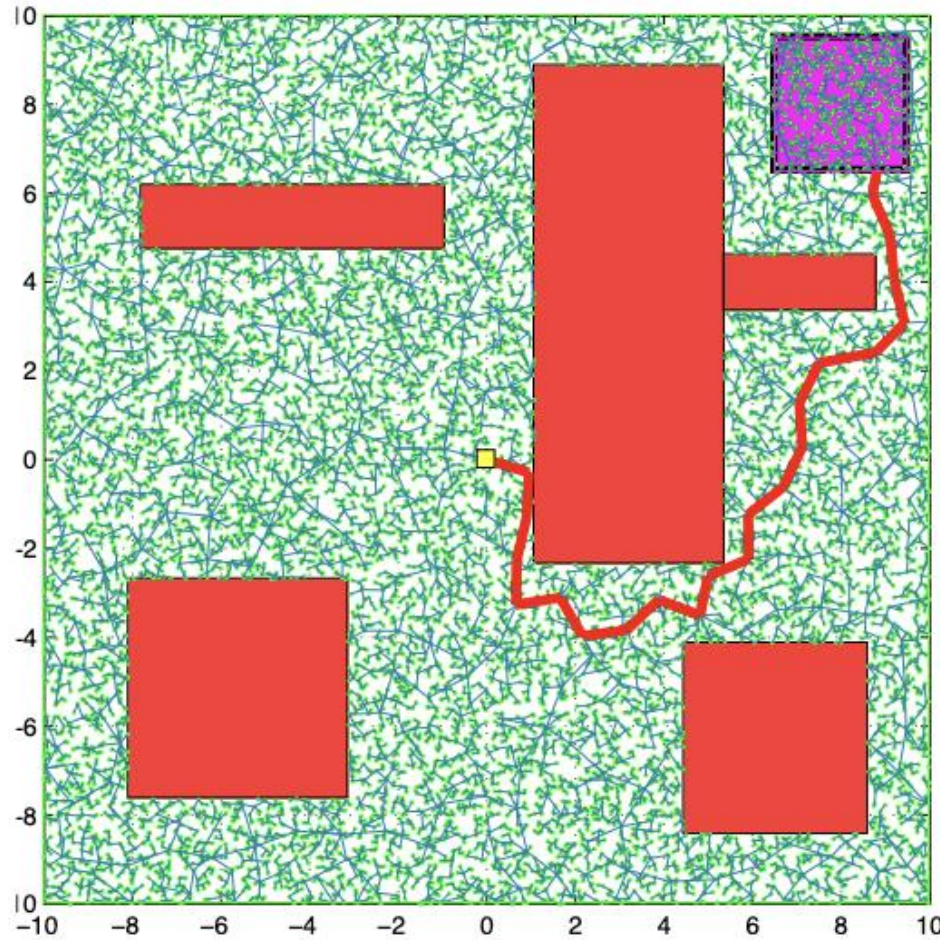


RRT*

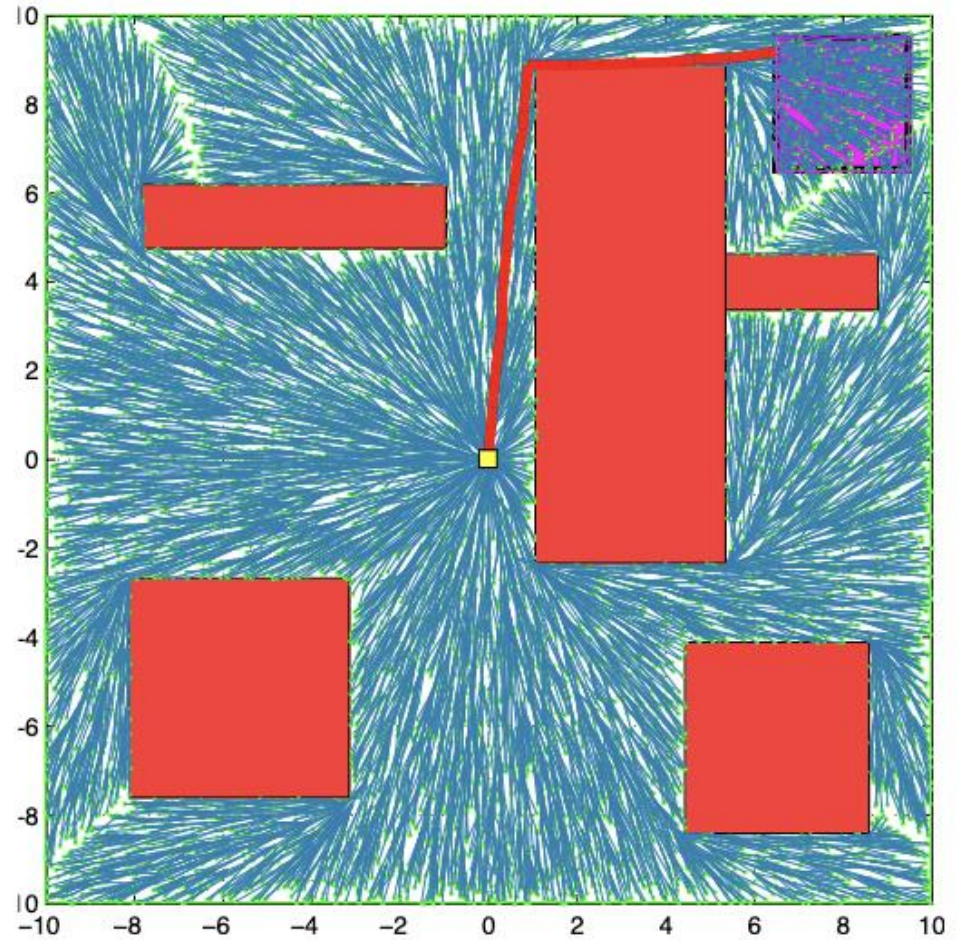


RRT vs. RRT*

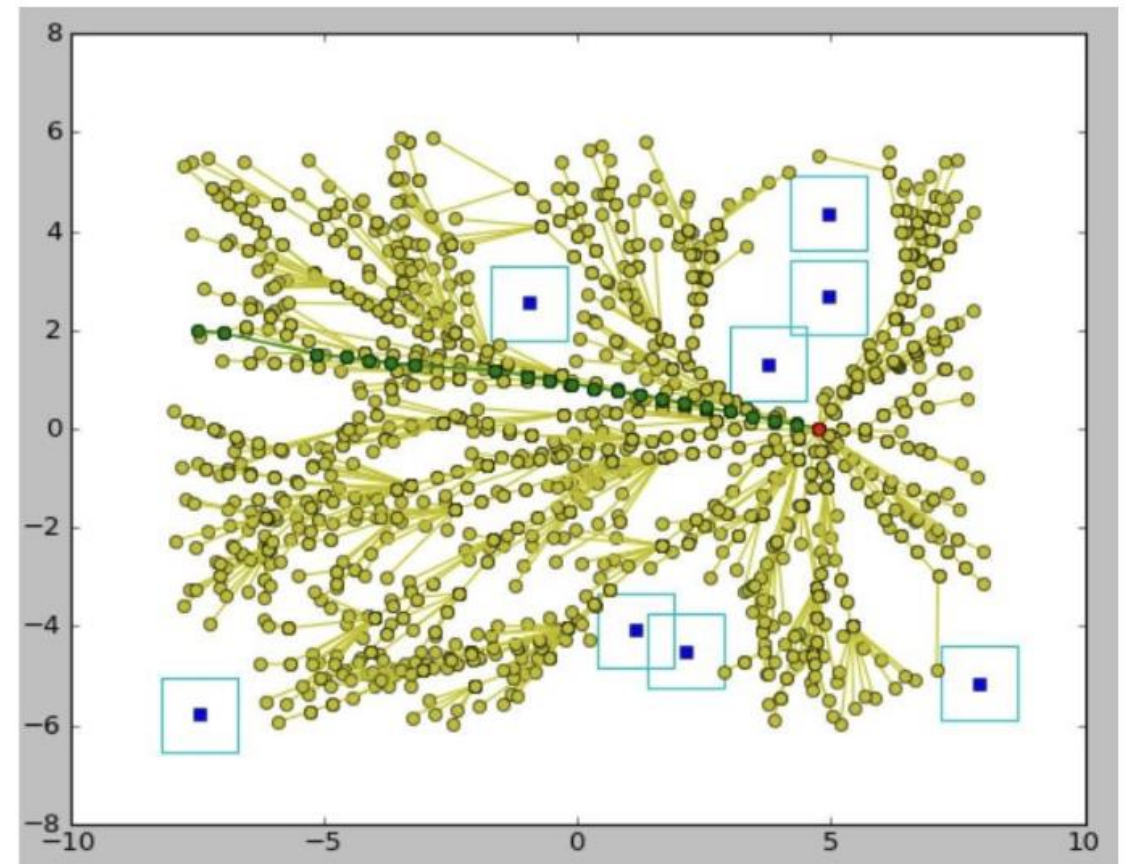
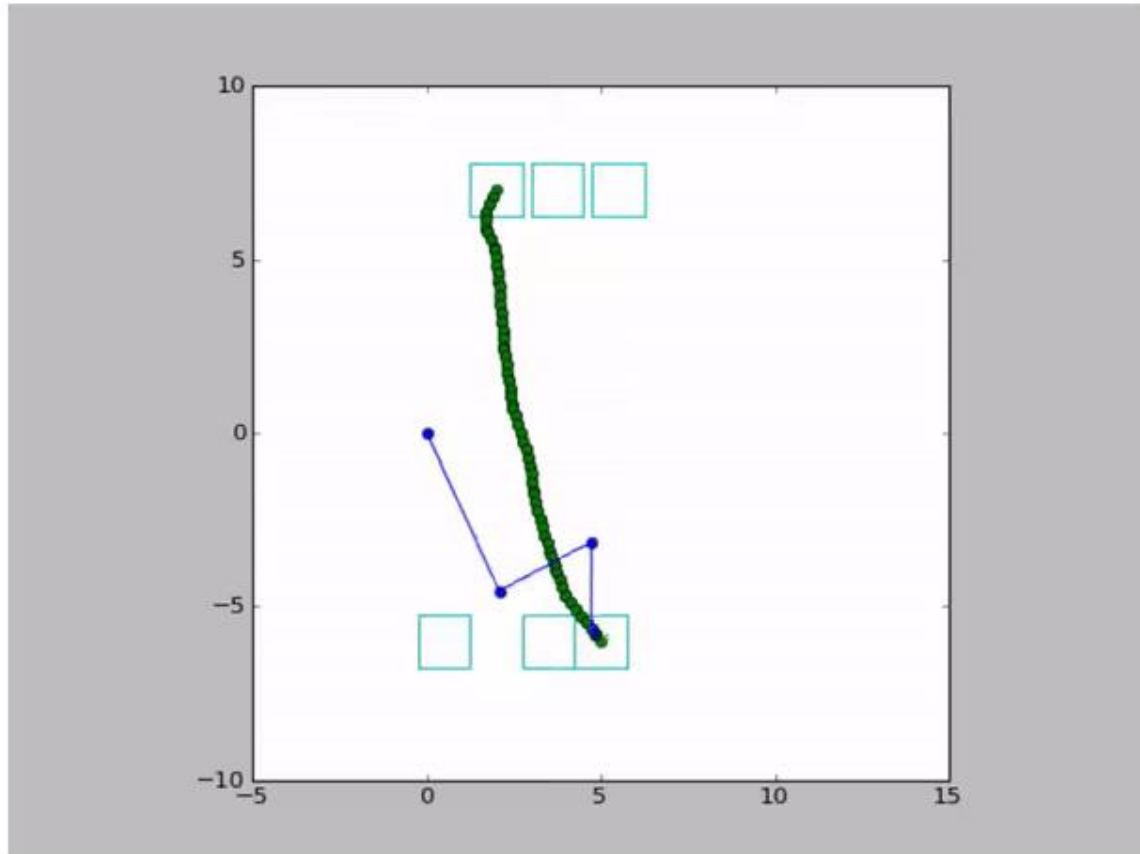
RRT



RRT*



RRT*



Virtual Potential Fields

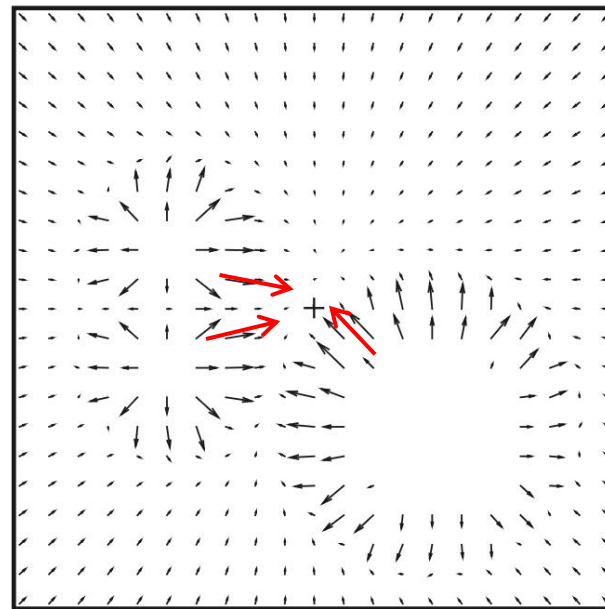
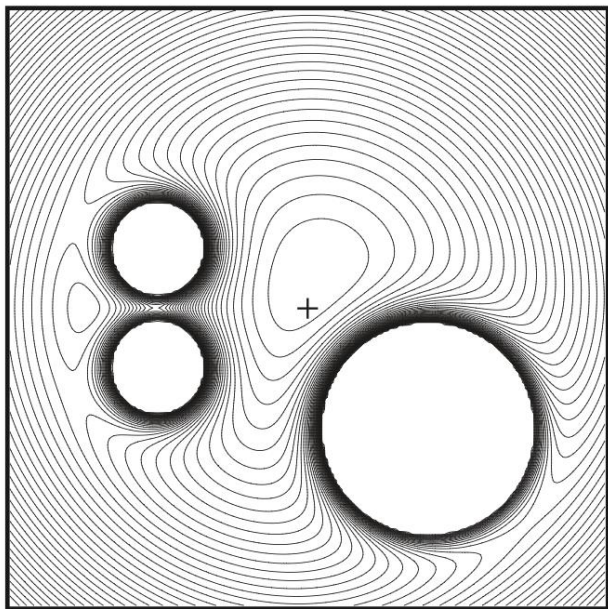
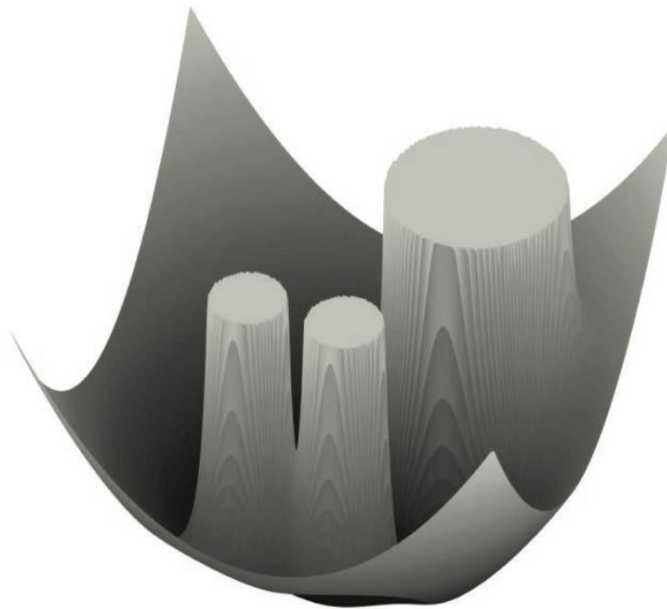
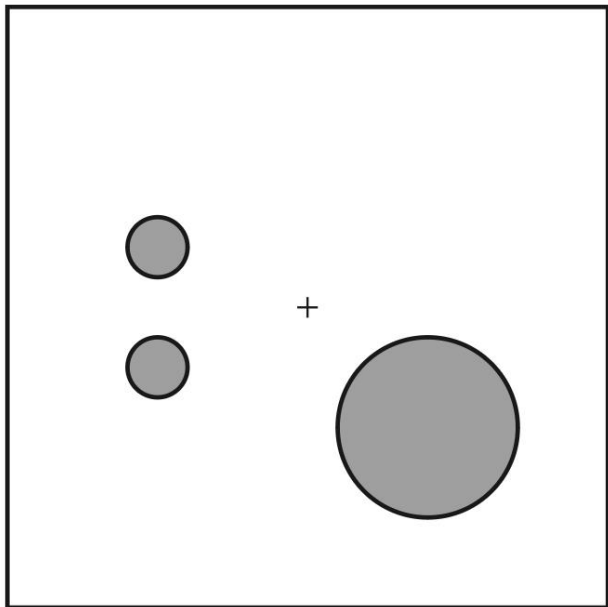
- From physics we know that a potential field $P(q)$ defined over $\mathcal{C}$ induces a force $F = -\frac{\partial P}{\partial q}$ that drives an object from high to low potential.
- In robotics, we can define a potential field and derive the corresponding force, with which we drive the configuration q from high to low potential.

- The potential field of reaching a goal:

$$\mathcal{P}_{\text{goal}}(q) = \frac{1}{2}(q - q_{\text{goal}})^T K (q - q_{\text{goal}}), \quad F_{\text{goal}}(q) = -\frac{\partial \mathcal{P}_{\text{goal}}}{\partial q} = K(q_{\text{goal}} - q);$$

- The potential field induced by a $\mathcal{C}$ -obstacle β

$$\mathcal{P}_{\mathcal{B}}(q) = \frac{k}{2d^2(q, \mathcal{B})} \quad F_{\mathcal{B}}(q) = -\frac{\partial \mathcal{P}_{\mathcal{B}}}{\partial q} = \frac{k}{d^3(q, \mathcal{B})} \frac{\partial d}{\partial q}.$$



Pushing configuration
to regions with low
potential

$$q_{t+1} = q_t + \frac{\Delta t}{m} \Sigma F$$

Non-Linear Optimization

- We can consider path planning as an optimization problem

$$\text{find} \quad u(t), q(t), T \quad (10.6)$$

$$\text{minimizing} \quad J(u(t), q(t), T) \quad (10.7)$$

$$\text{subject to} \quad \dot{x}(t) = f(x(t), u(t)), \quad \forall t \in [0, T], \quad (10.8)$$

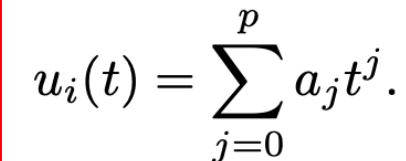
$$u(t) \in \mathcal{U}, \quad \forall t \in [0, T], \quad (10.9)$$

$$q(t) \in \mathcal{C}_{\text{free}}, \quad \forall t \in [0, T], \quad (10.10)$$

$$x(0) = x_{\text{start}}, \quad (10.11)$$

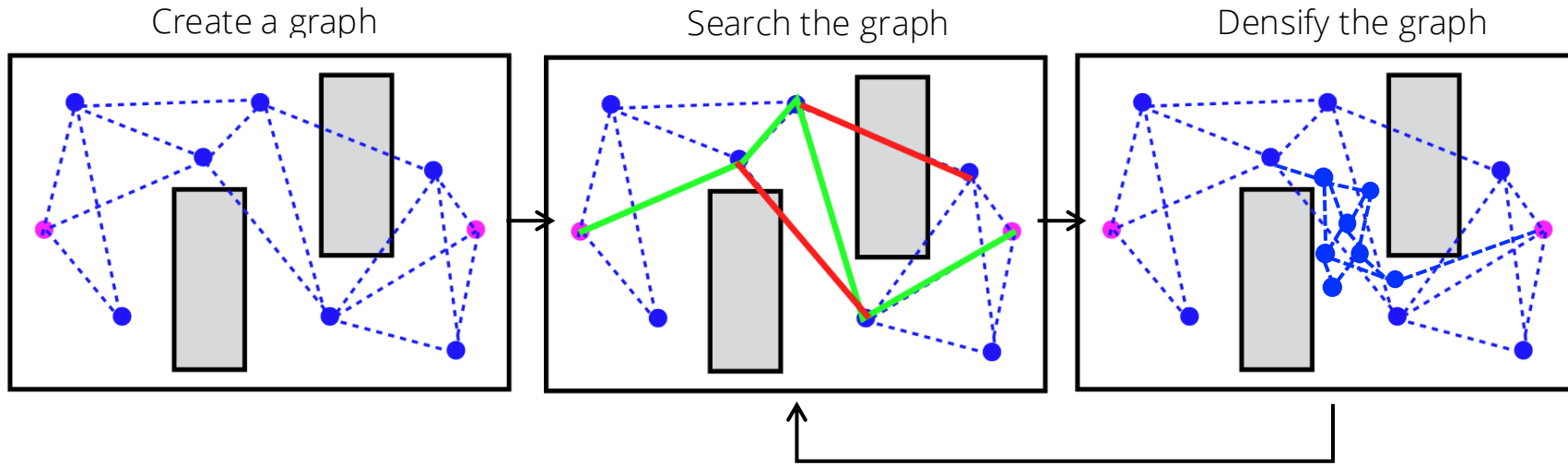
$$x(T) = x_{\text{goal}}. \quad (10.12)$$

- We can parametrize control $u(t)$ and path $q(t)$ with the coefficients of (1) a polynomial, (2) a truncated Fourier series, (3) spline, (4) wavelet, or (5) piecewise constant acceleration segments in time.


$$u_i(t) = \sum_{j=0}^p a_j t^j.$$

Demo Time

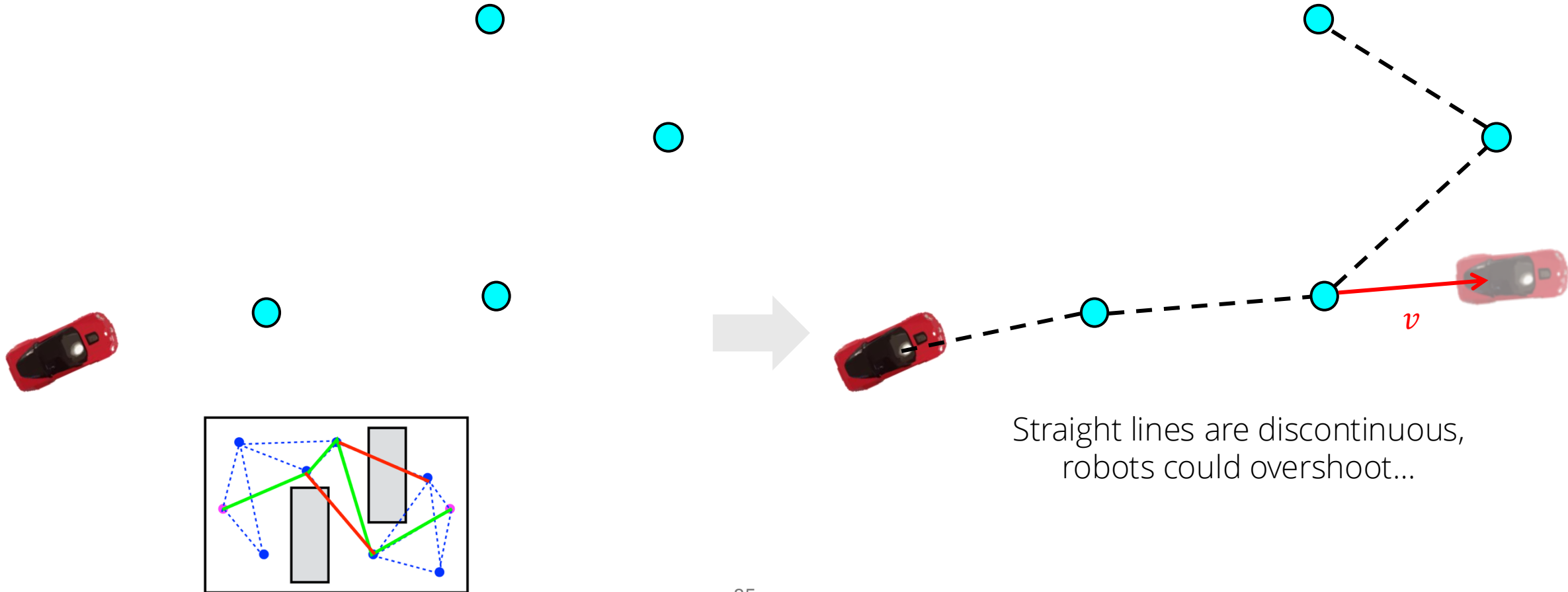
What are Missing? We Ignore Motion Constraints



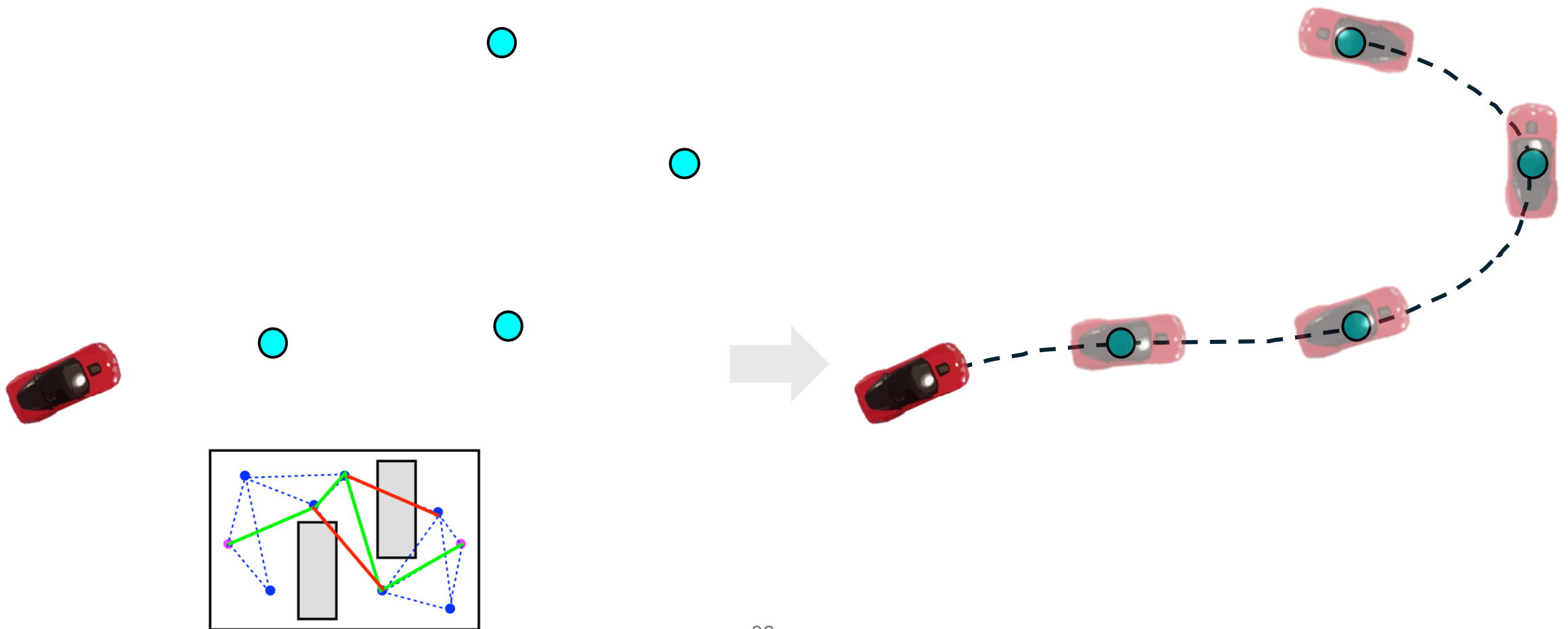
- We have only considered **path** and **obstacle**, but ignore:
 - How to traverse from one node to another



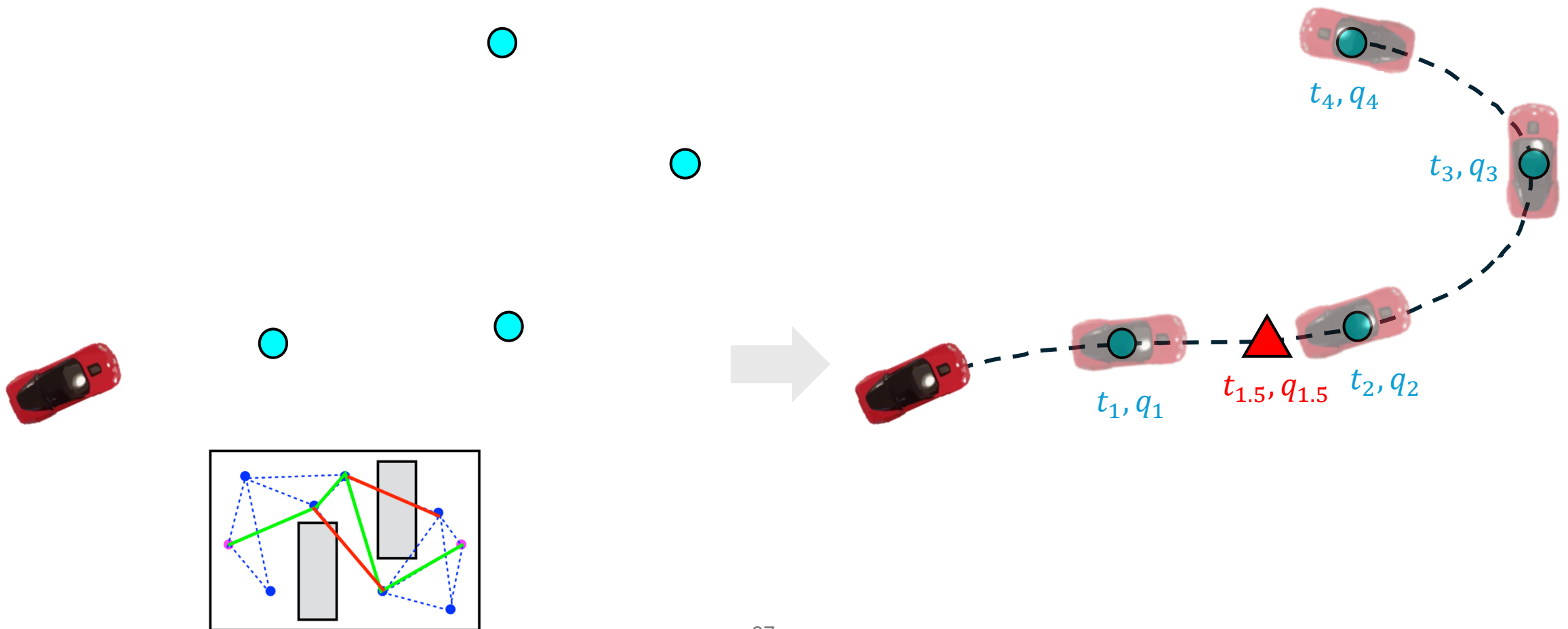
Should We Traverse the Path in Straight Lines?



Traverse in Smooth Curves

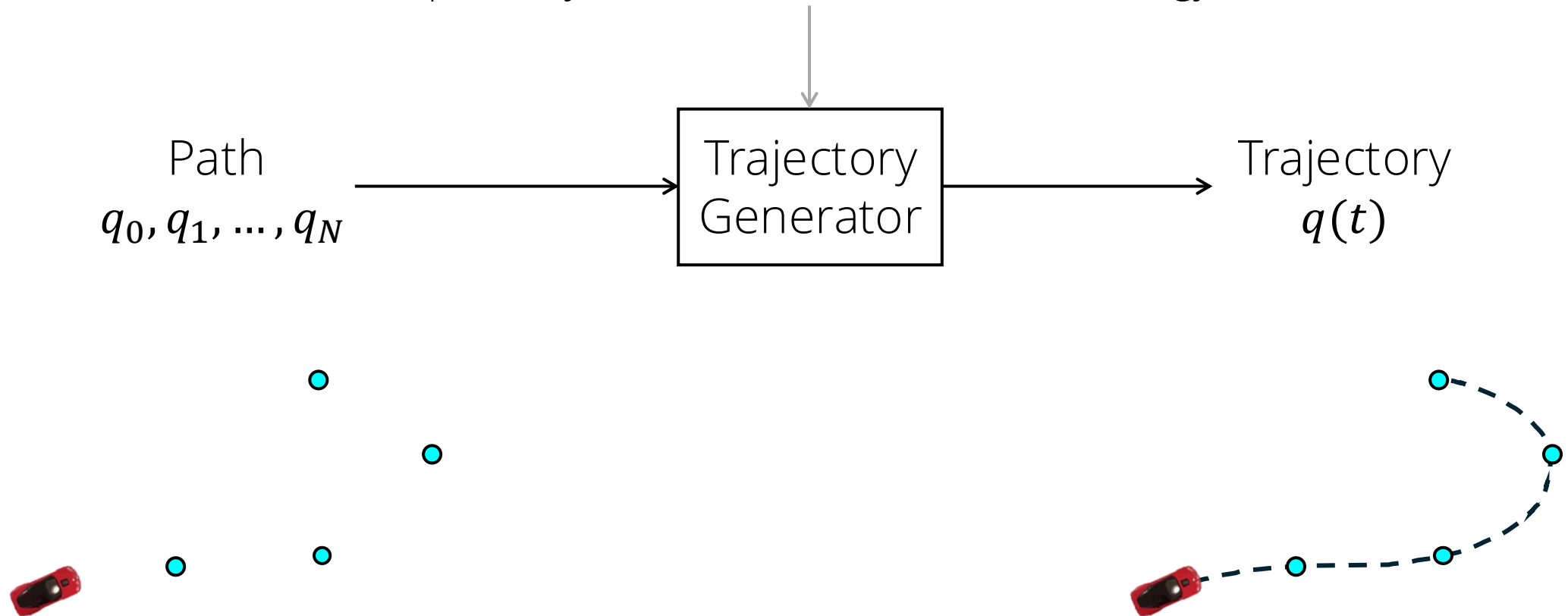


Traverse in Smooth Trajectories



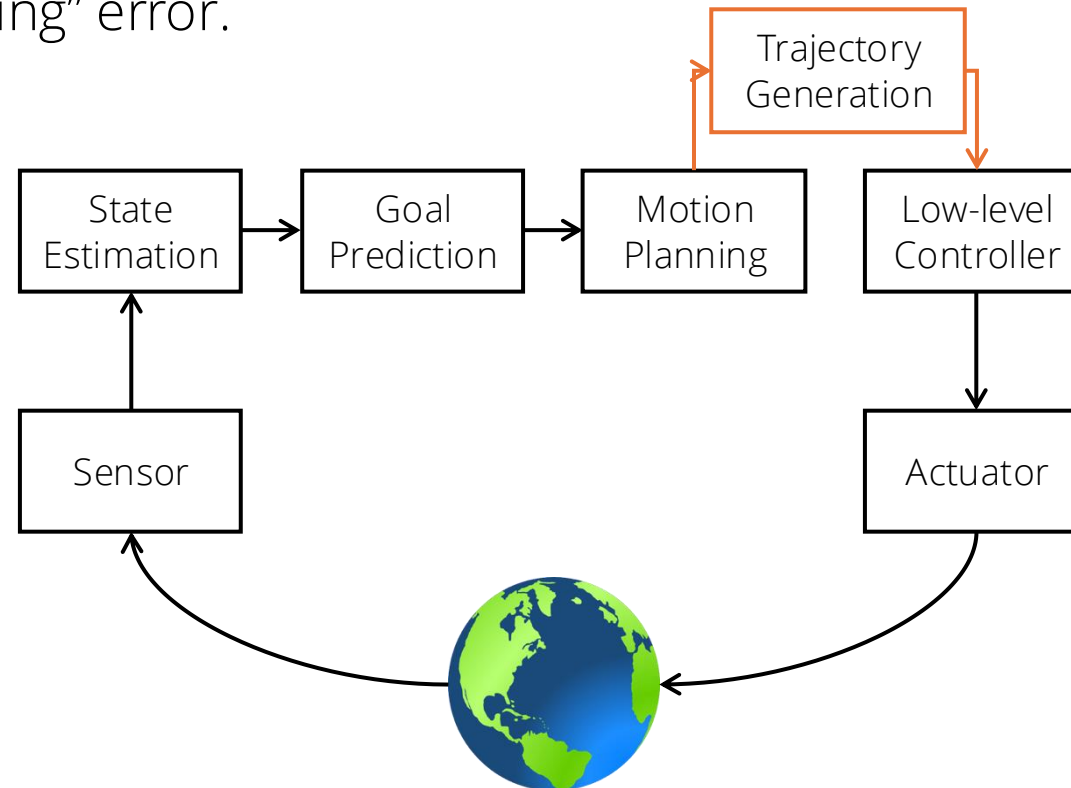
Trajectory Generation

- Constraints as continuity, smoothness, velocity/acceleration limit, kinematic,
- optimality criteria (min transfer time / energy...)



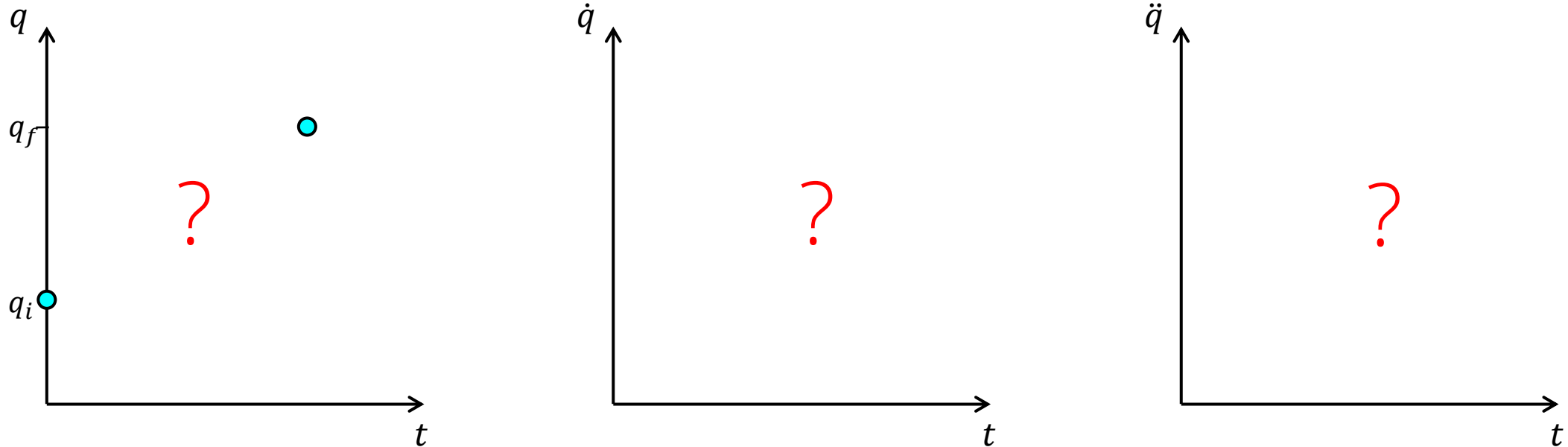
Basic Recipe of Planning and Control

1. Motion planning: compute a feasible path
 - grid search / PRM / RRT ...
2. **Trajectory generalization:** time-scale the path into a trajectory
3. Controller: predict a control to follow the path or minimize the "tracking" error.



Point-to-Point Trajectory Generation

- Let's start with an easy example: point-to-point trajectory generation

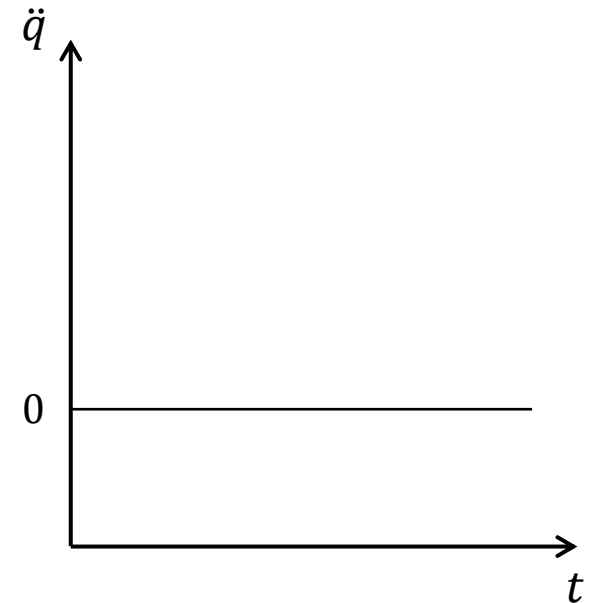
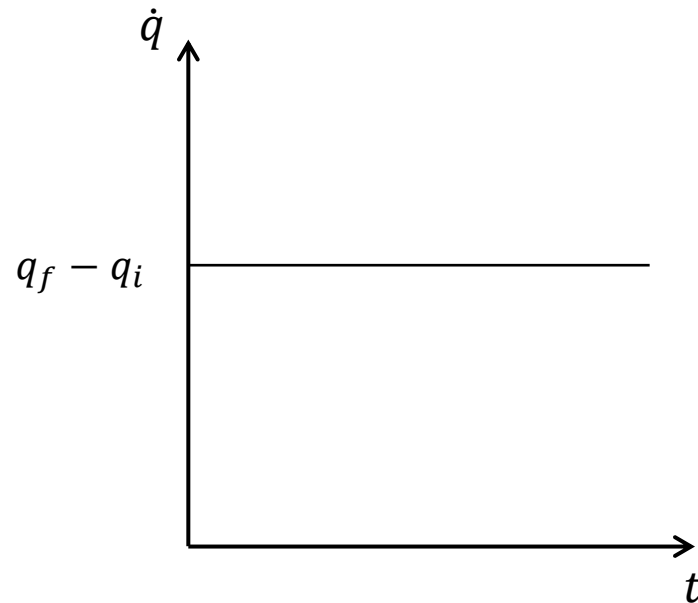
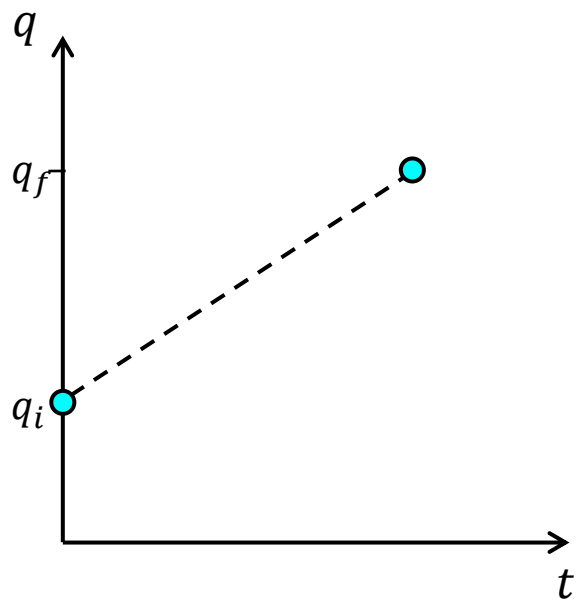


Point-to-Point Trajectory Generation with Straight Lines

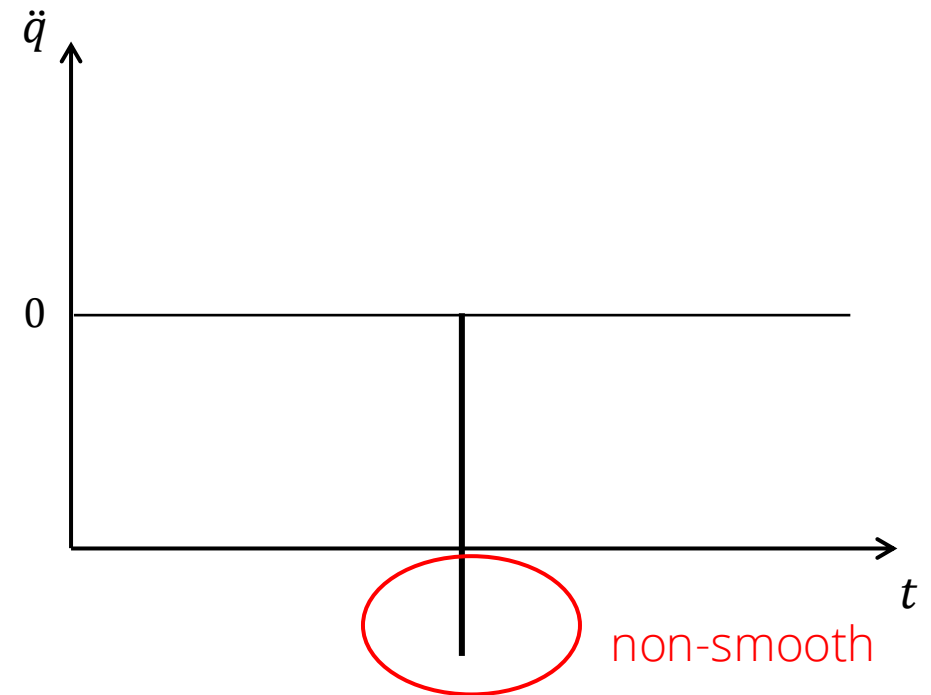
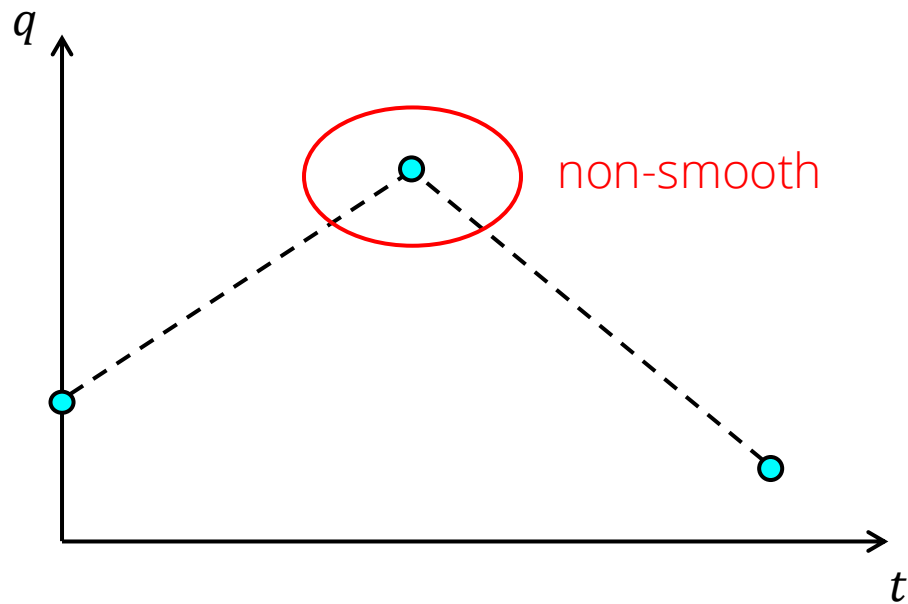
- Straight-line paths:

$$q(t) = q_i + t(q_f - q_i)$$

$$\dot{q}(t) = \frac{q_f - q_i}{t} \text{ and } \ddot{q}(t) = 0$$



Point-to-Point Trajectory Generation with Straight Lines



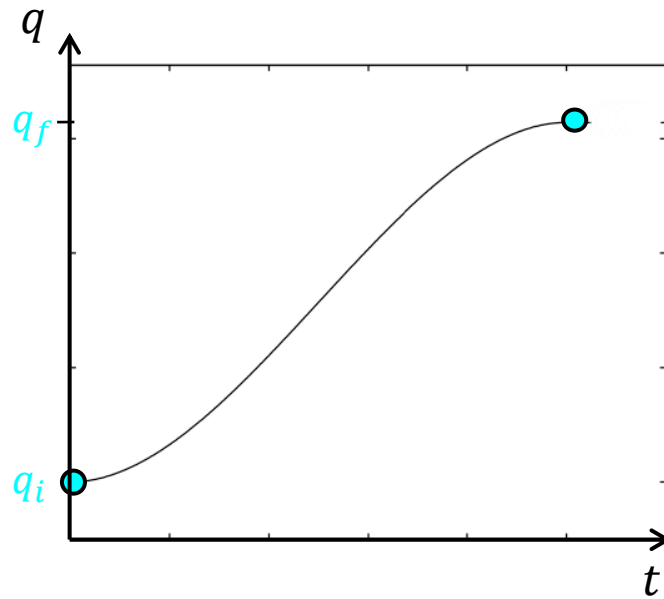
Point-to-Point Trajectory Generation with Cubic Polynomials

- Cubic polynomial paths:

$$q(t) = a_3 t^3 + a_2 t^2 + a_1 t + a_0$$

$$\dot{q}(t) = 3a_3 t^2 + 2a_2 t + a_1$$

$$\ddot{q}(t) = 6a_3 t + 2a_2$$



Cubic Polynomial Paths

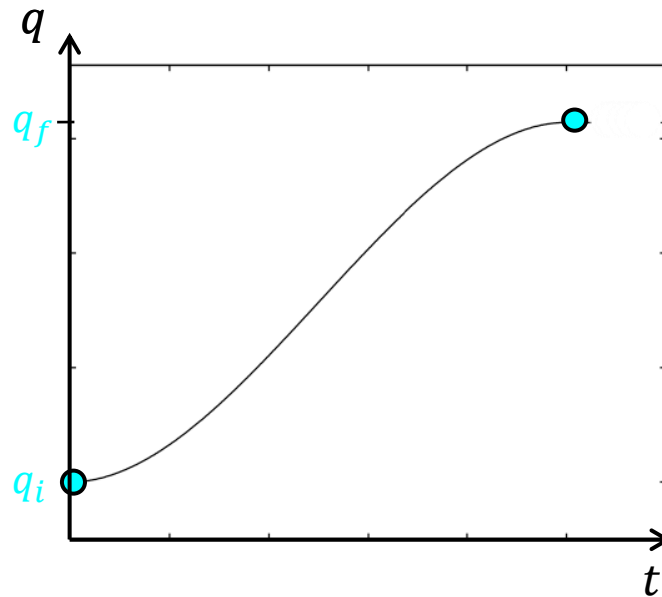
- Equations:

$$q(t) = a_3 t^3 + a_2 t^2 + a_1 t + a_0$$

$$\dot{q}(t) = 3a_3 t^2 + 2a_2 t + a_1$$

$$\ddot{q}(t) = 6a_3 t + 2a_2$$

Solve the equation



- when $t = 0$

$$a_3 0^3 + a_2 0^2 + a_1 0 + a_0 = q_i \Rightarrow a_0 = q_i$$

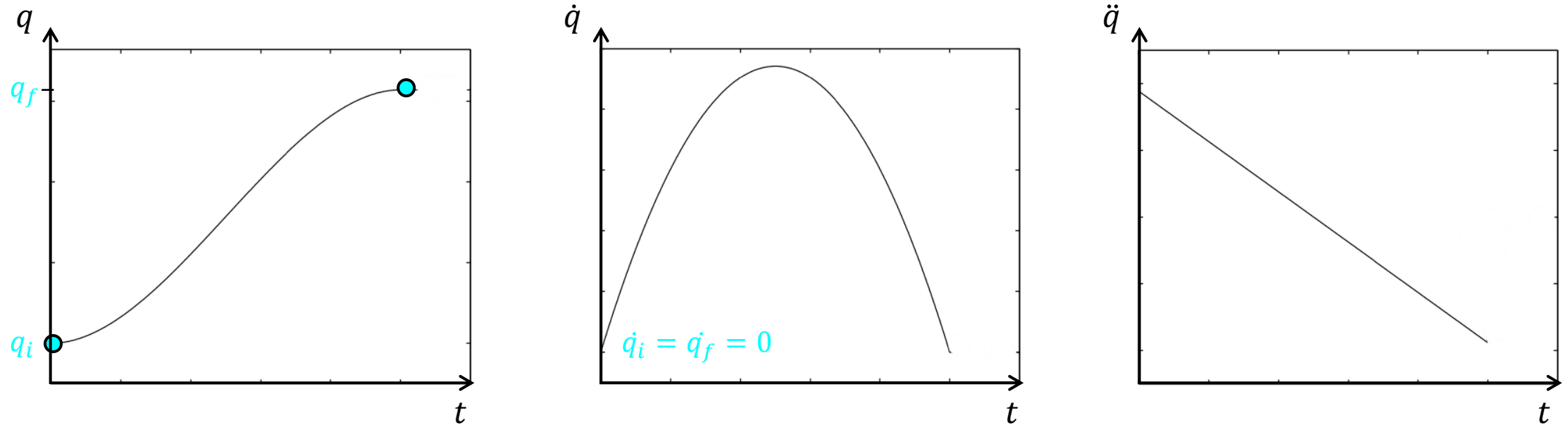
$$3a_3 0^2 + 2a_2 0 + a_1 = \dot{q}_i \Rightarrow a_1 = \dot{q}_i$$

- when $t = t_f$

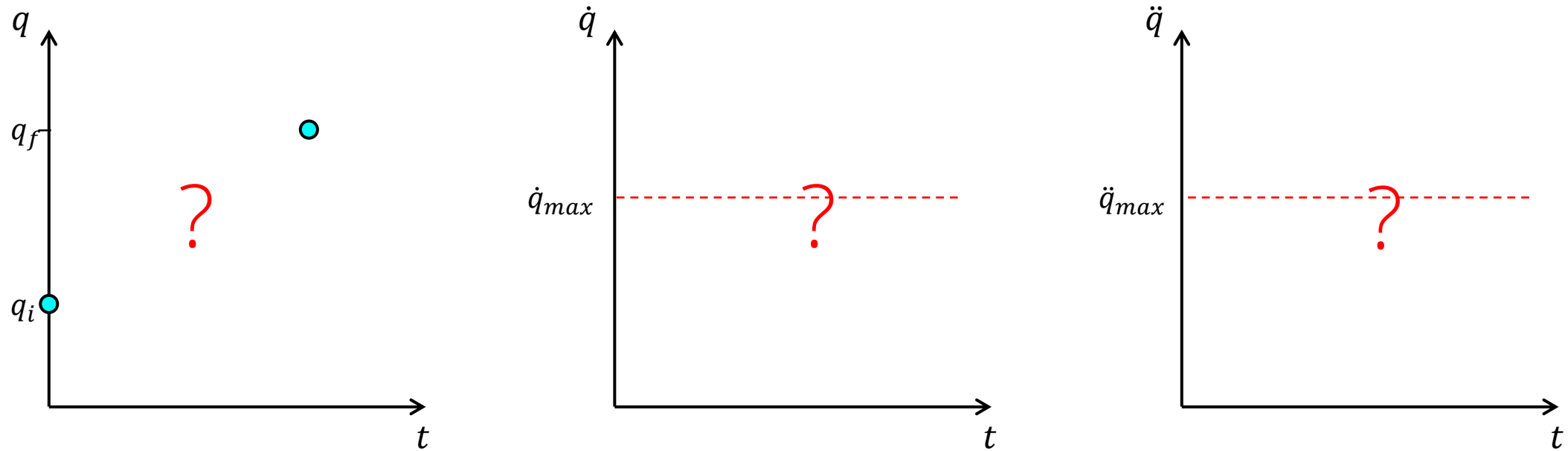
$$a_3 t_f^3 + a_2 t_f^2 + a_1 t_f + a_0 = q_f$$

$$3a_3 t_f^2 + 2a_2 t_f + a_1 = \dot{q}_f$$

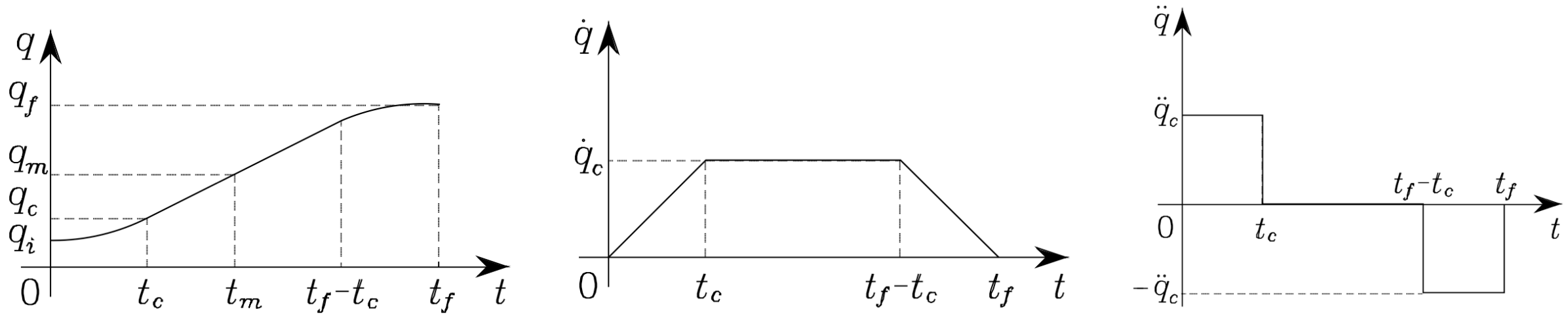
Point-to-Point Trajectory Generation with Cubic Polynomials



What if We Want to Impose Velocity/Acceleration Constraints?

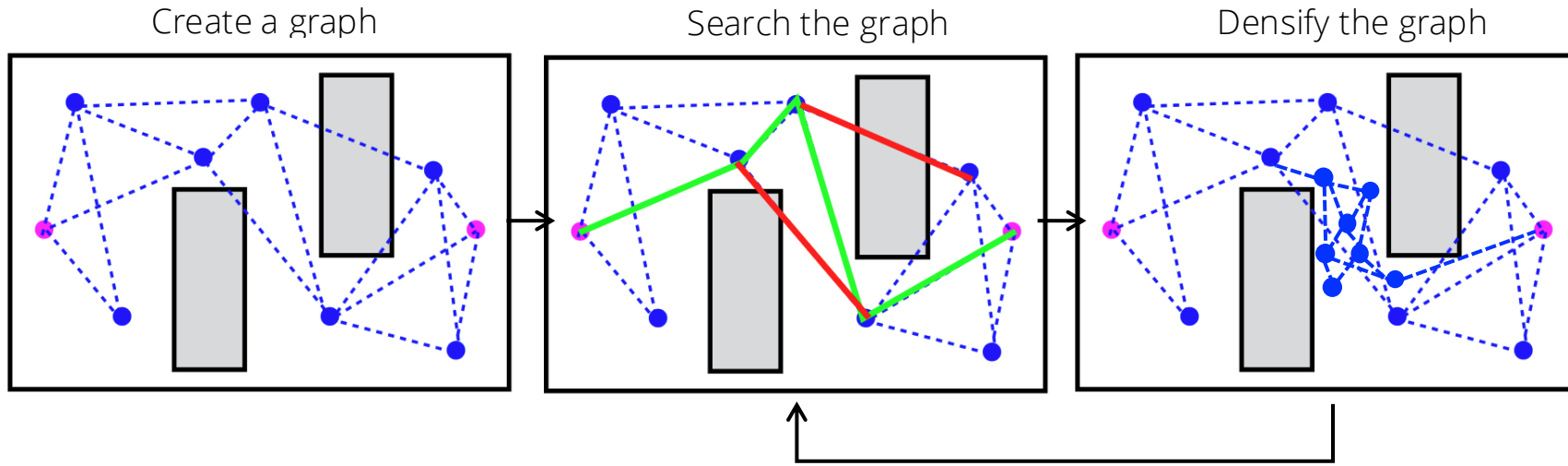


Point-to-Point Trajectory Generation with Trapezoidal Motion Profiles



Check the Appendix for more the detailed derivation

What are Missing? We Ignore Motion Constraints

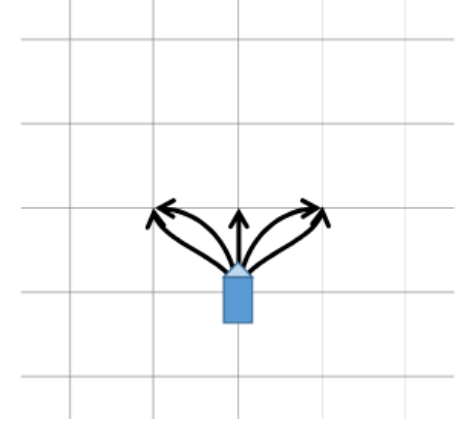
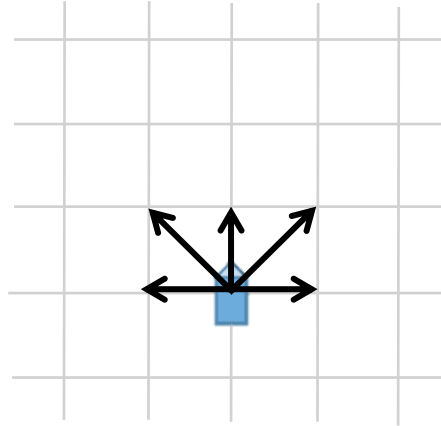


- We have only considered **path** and **obstacle**, but ignore:
 - How to traverse from one node to another
 - Motion (velocity/acceleration) constraints
 - Safety constraints

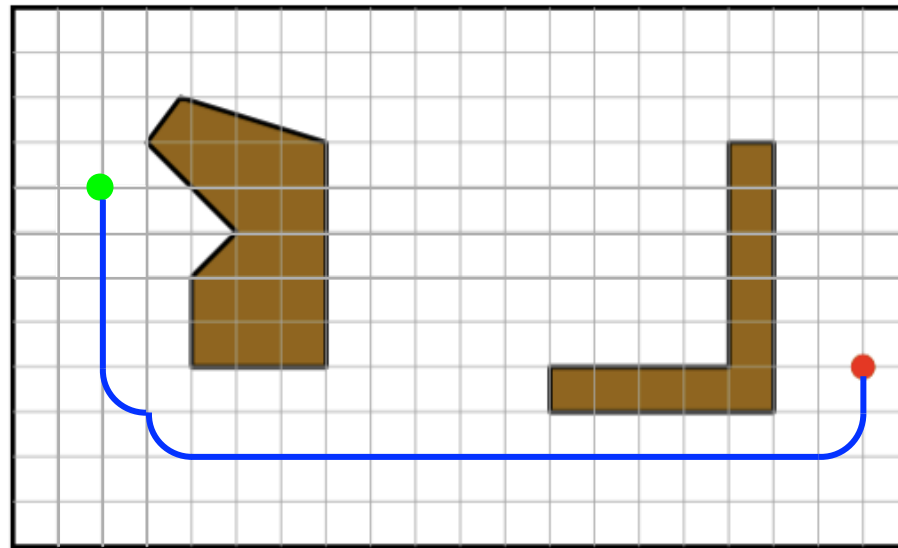
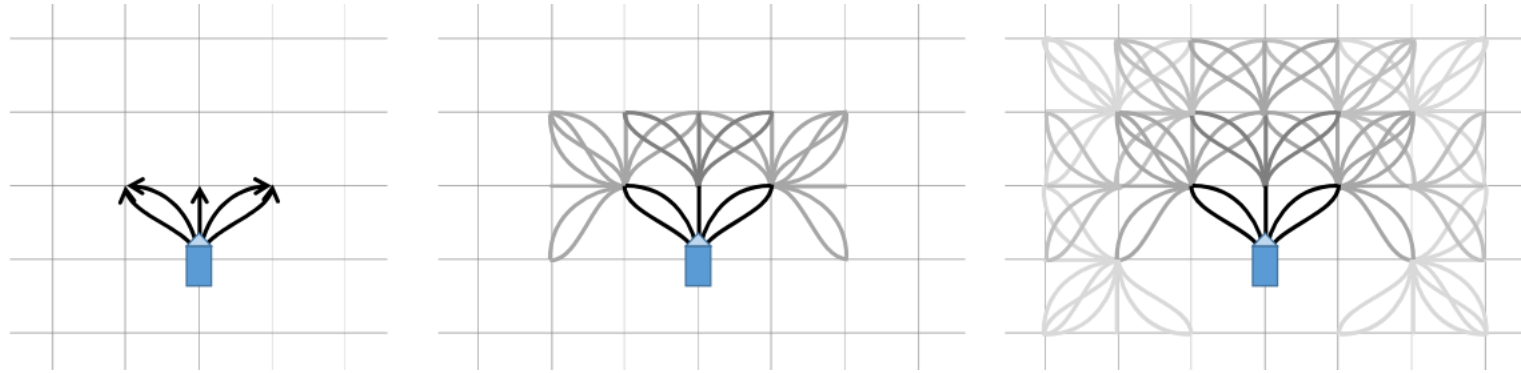


Can We Generate Paths that Respect Directional Constraints?

- Take *car* as an example. How can we reformulate the roadmap planners?

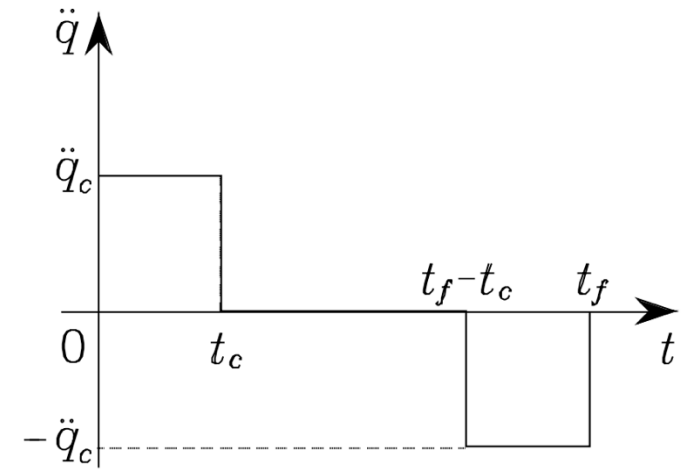
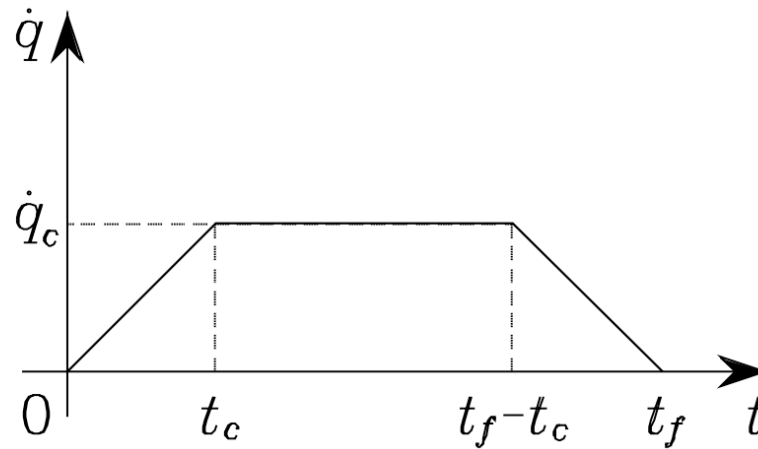
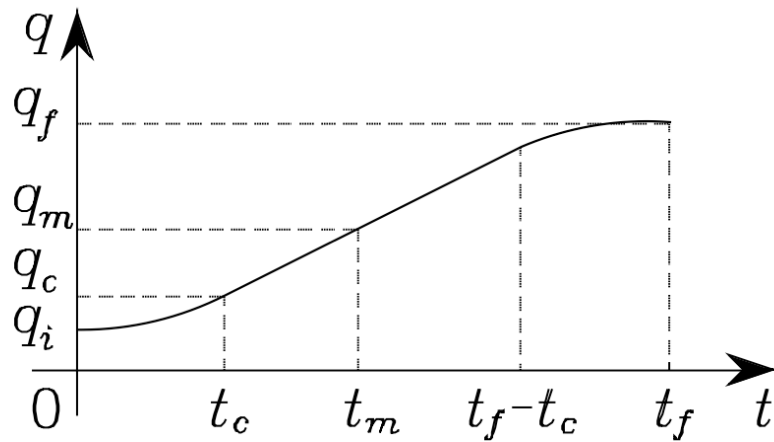


Path Planning with Directional Constraints



Appendix

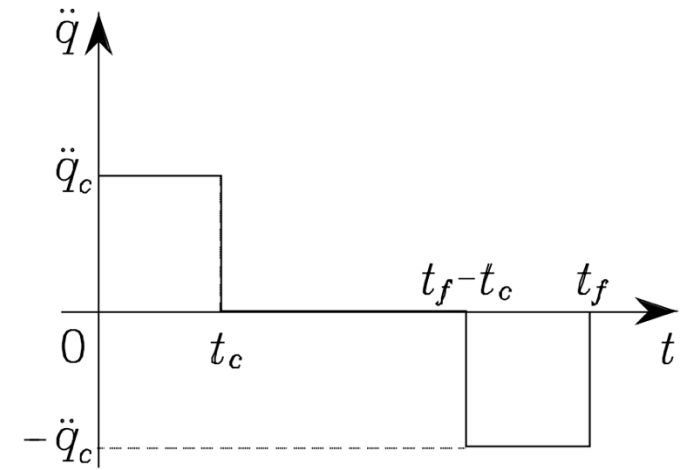
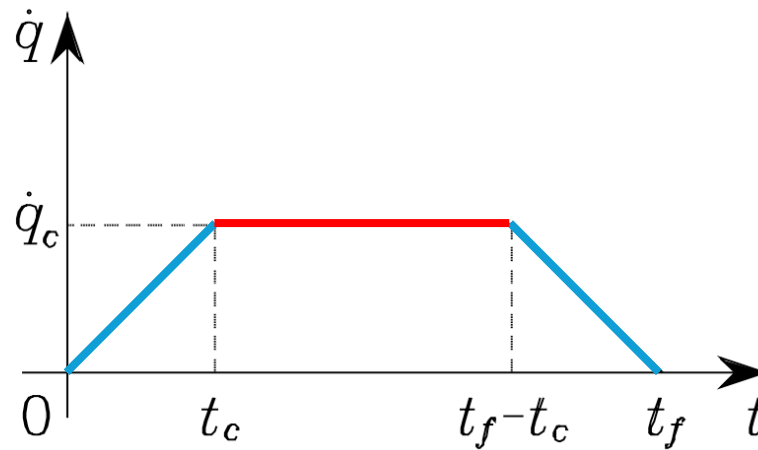
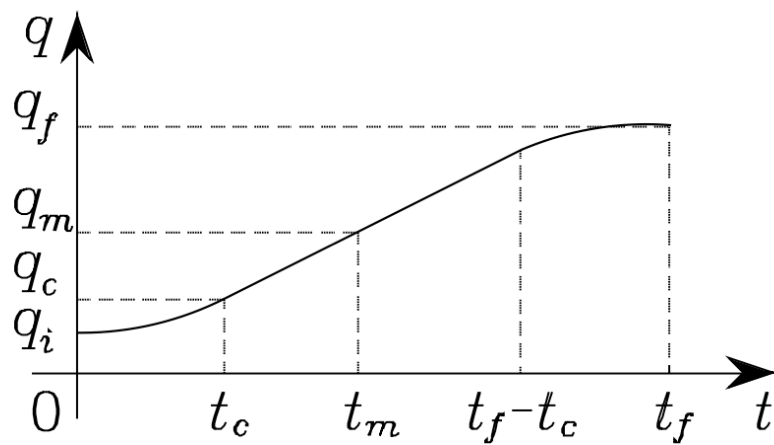
Point-to-Point Trajectory Generation with Trapezoidal Motion Profiles

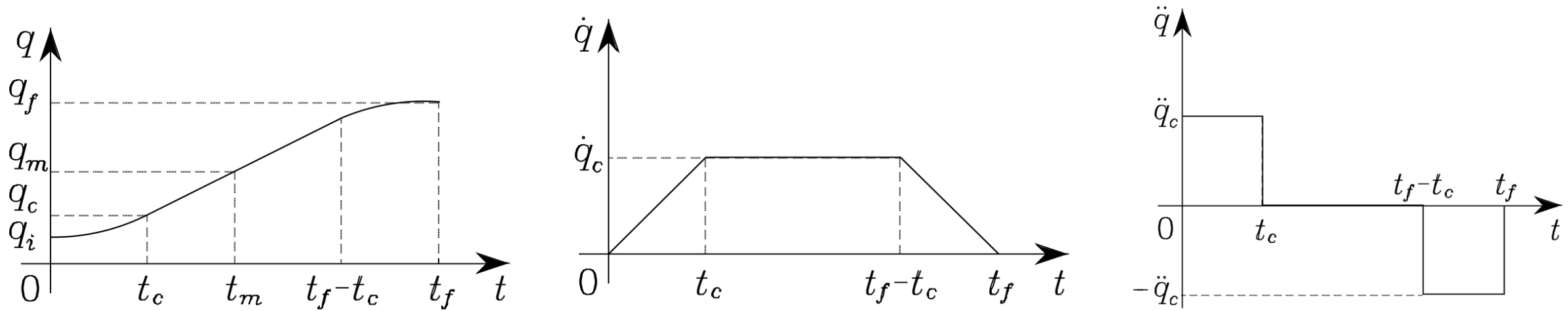


Point-to-Point Trajectory Generation with Trapezoidal Motion Profiles

Also known as Linear Segments with Parabolic Blends:

1. Fit with quadratic polynomials (linear ramp velocity)
2. At the blend time, switch to a linear function (constant velocity)





- Average point: $q_m = (q_f + q_i)/2$ at $t_m = t_f/2$
- Velocity:

$$\ddot{q}_c t_c = \frac{q_m - q_c}{t_m - t_c}$$

$$q_c = q_i + \frac{1}{2} \ddot{q}_c t_c^2$$

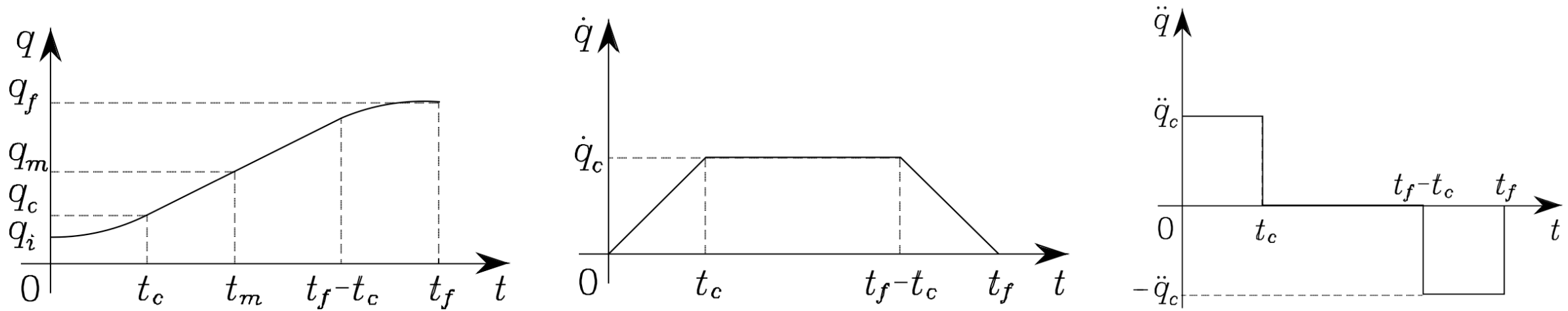
$$\ddot{q}_c t_c^2 - \ddot{q}_c t_f t_c + q_f - q_i = 0$$

↓ We can specify $\ddot{q}_c$

$$t_c = \frac{t_f}{2} - \frac{1}{2} \sqrt{\frac{t_f^2 \ddot{q}_c - 4(q_f - q_i)}{\ddot{q}_c}}.$$

Constraints

$$|\ddot{q}_c| \geq \frac{4|q_f - q_i|}{t_f^2}$$



- Average point: $q_m = (q_f + q_i)/2$ at $t_m = t_f/2$
- Velocity:

$$\ddot{q}_c t_c = \frac{q_m - q_c}{t_m - t_c}$$

$$q_c = q_i + \frac{1}{2} \ddot{q}_c t_c^2$$

$$\ddot{q}_c t_c^2 - \ddot{q}_c t_f t_c + q_f - q_i = 0$$

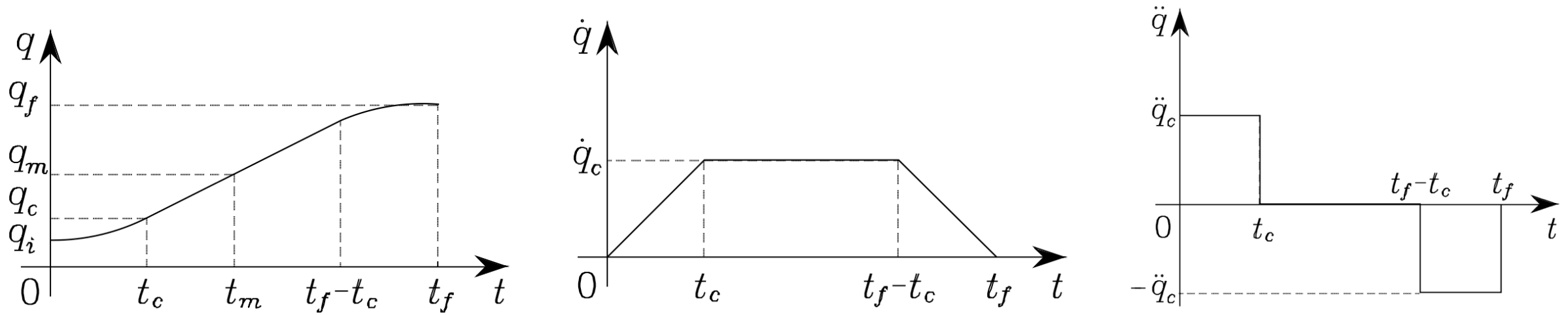
↓ We can specify $\dot{q}_c$

$$t_c = \frac{q_i - q_f + \dot{q}_c t_f}{\dot{q}_c}$$

Constraints

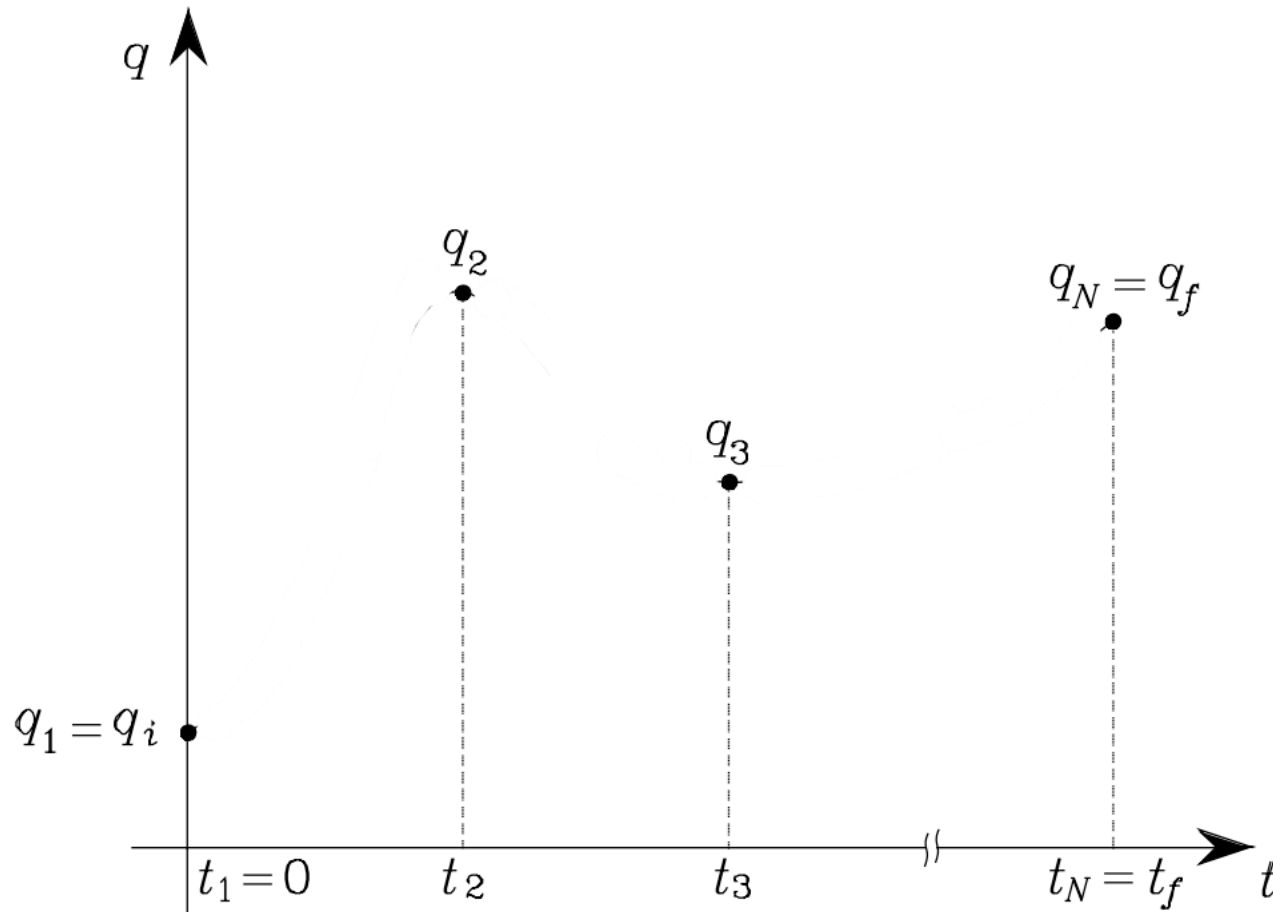
$$\frac{|q_f - q_i|}{t_f} < |\dot{q}_c| \leq \frac{2|q_f - q_i|}{t_f}$$

Point-to-Point Trajectory Generation with Trapezoidal Motion Profiles



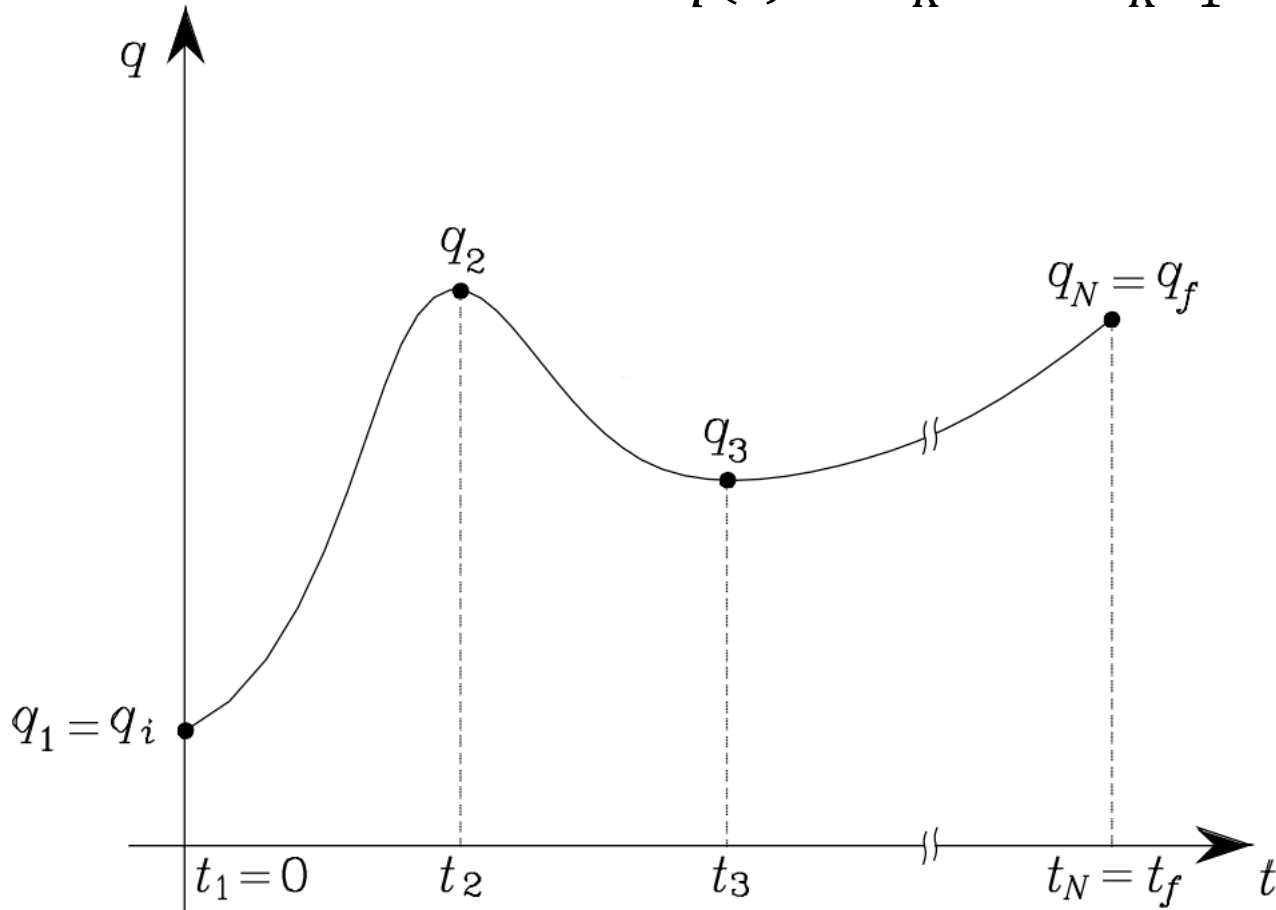
$$q(t) = \begin{cases} q_i + \frac{1}{2}\ddot{q}_c t^2 & 0 \leq t \leq t_c \\ q_i + \dot{q}_c(t - t_c/2) & t_c < t \leq t_f - t_c \\ q_f - \frac{1}{2}\ddot{q}_c(t_f - t)^2 & t_f - t_c < t \leq t_f. \end{cases}$$

What if We Want to Generate Trajectory from Multiple Waypoints?



Can We Fit a High-Order Polynomials?

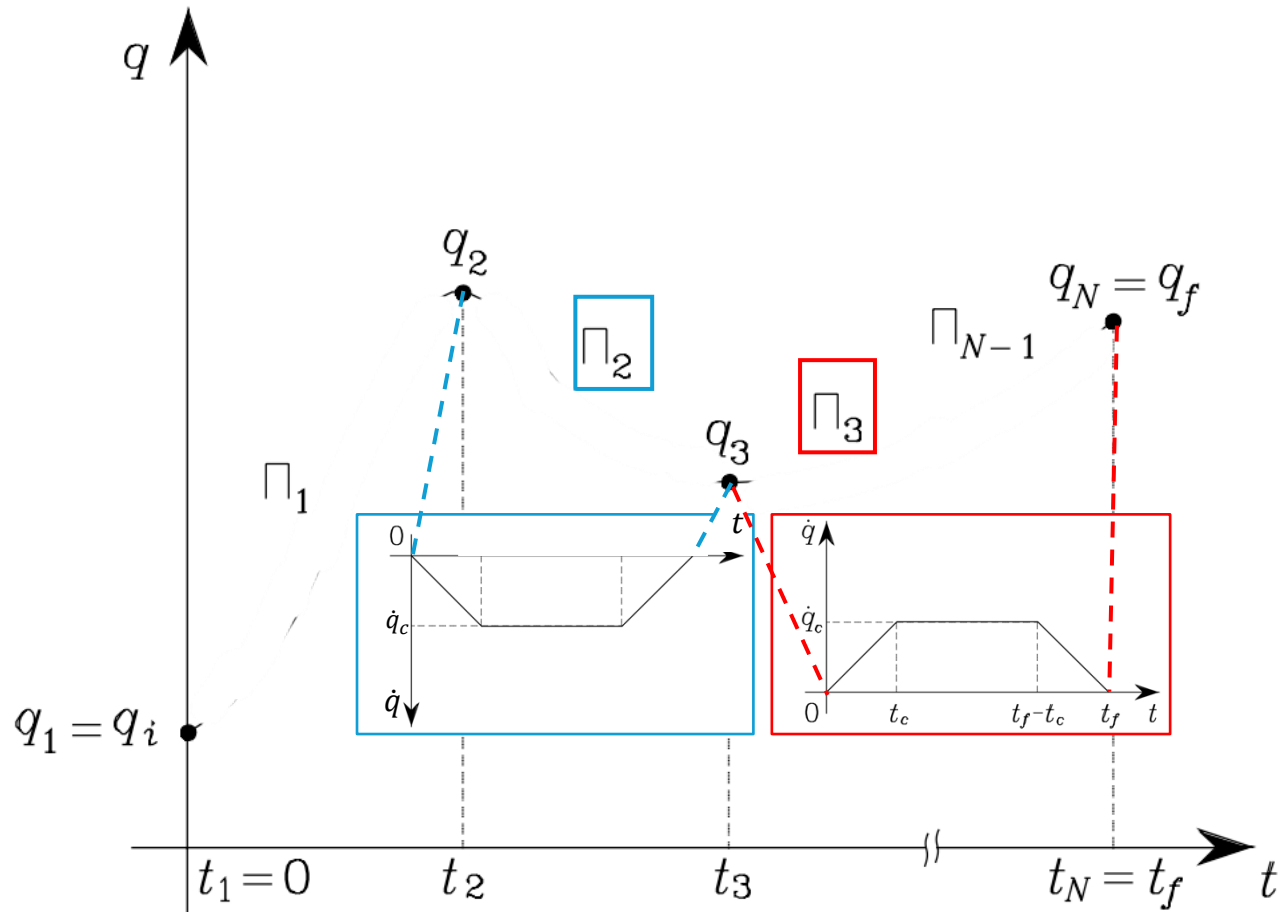
$$q(t) = a_K t^K + a_{K-1} t^{K-1} + \dots + a_1 t + a_0$$



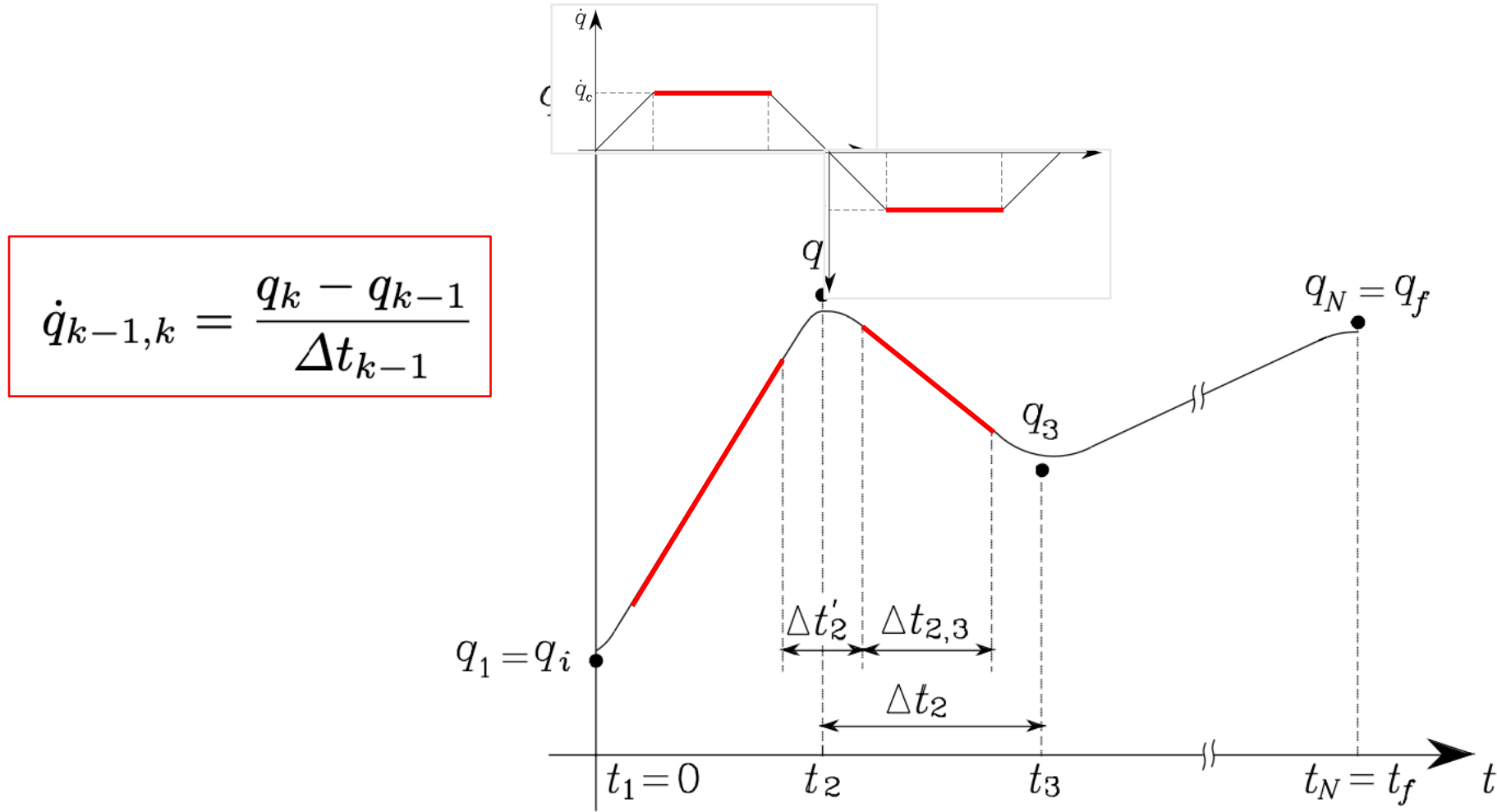
Drawbacks:

- The polynomial function overfits, resulting in oscillatory curves
- The system of constraint equations is heavy to solve

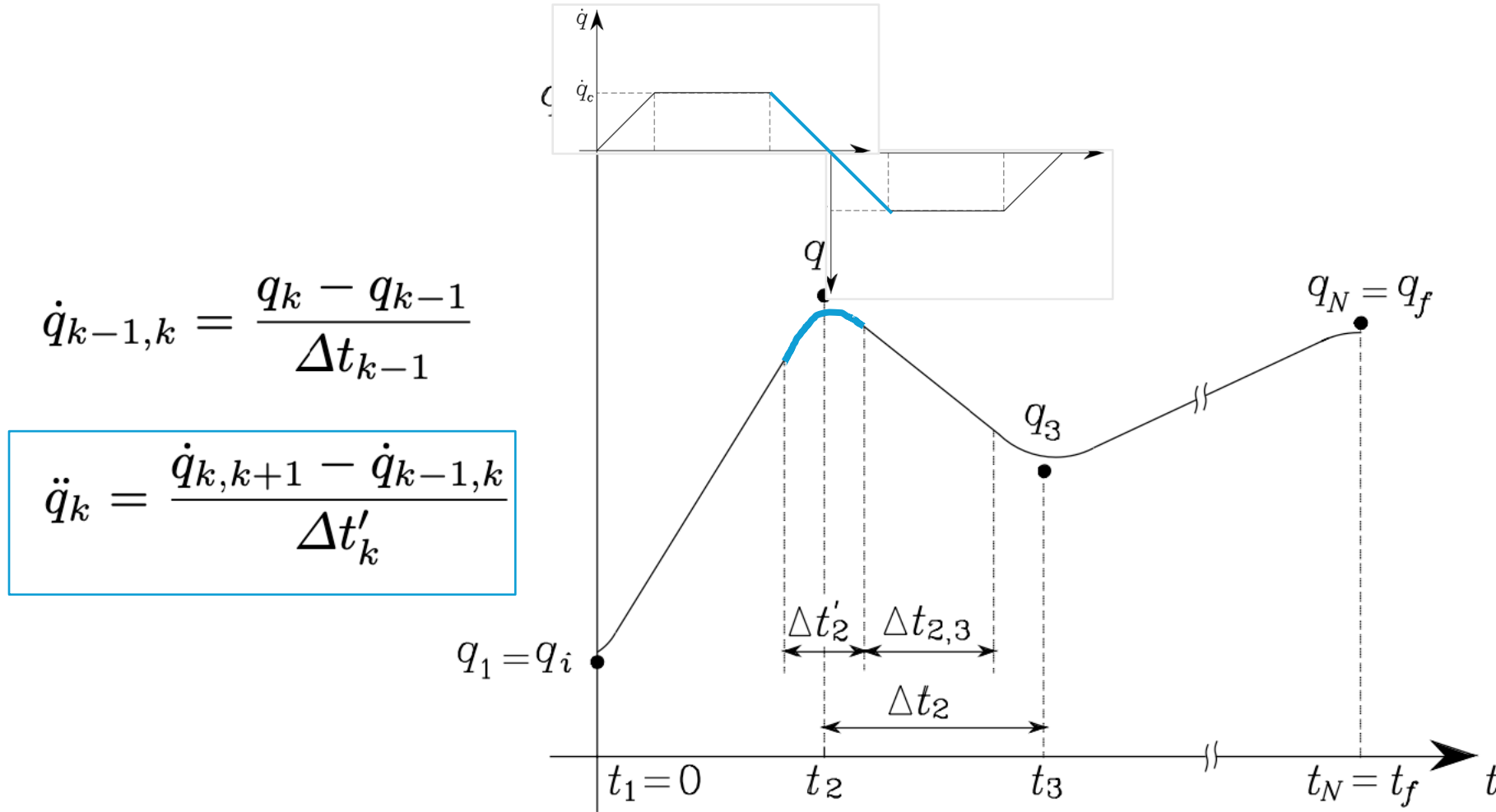
Can We Fit with Multiple Trapezoidal Motion Profiles?



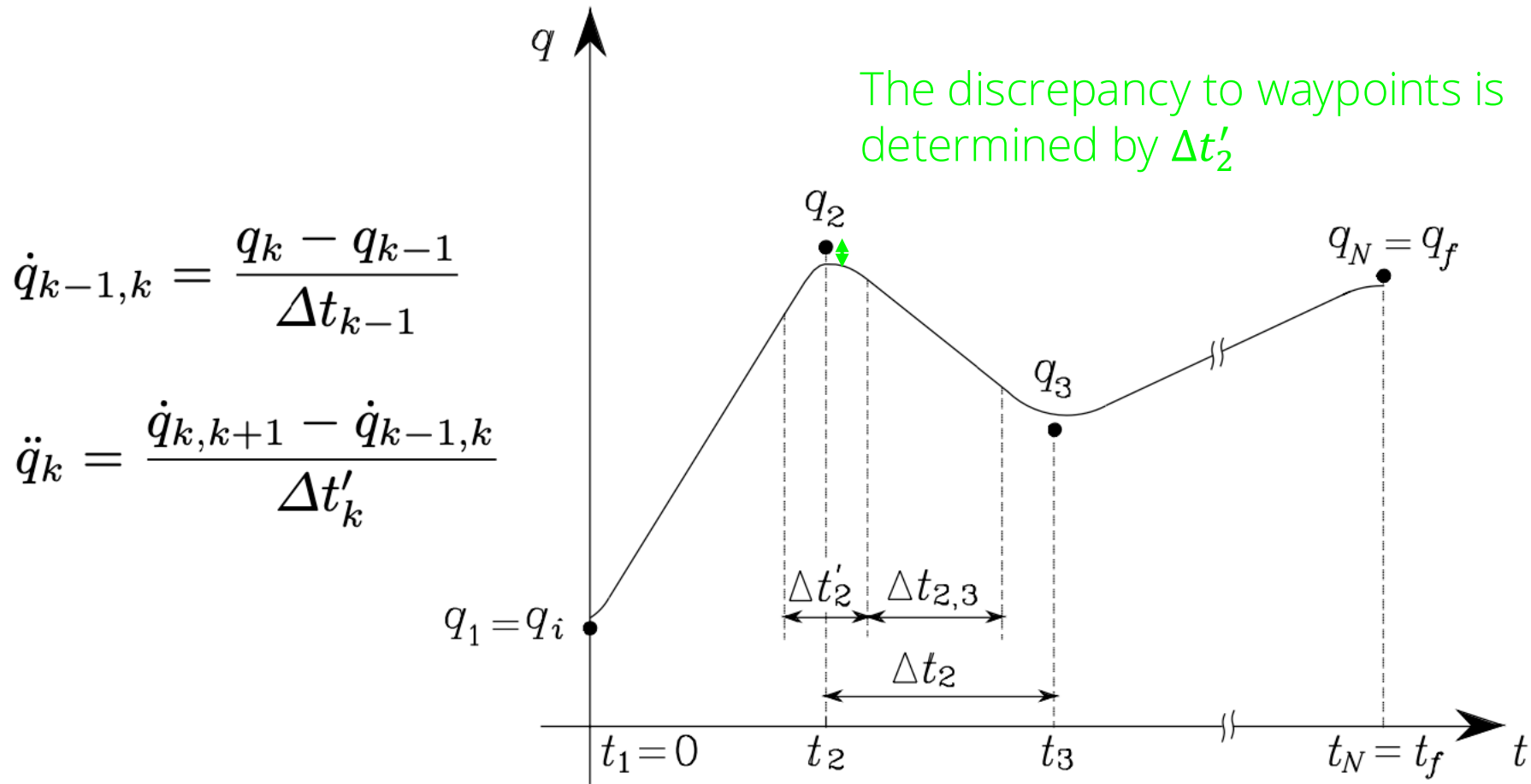
Can We Fit with Multiple Trapezoidal Motion Profiles?



Can We Fit with Multiple Trapezoidal Motion Profiles?



Can We Fit with Multiple Trapezoidal Motion Profiles?



See "Turning paths into Trajectories Using Parabolic Blends" by T. Kunz and M. Stilman for more details